

Vehicle Interface Volume

For

A-GRA

5.0a

21 APR 2026

DISCLAIMER: The discussion of non-federal entities, methods, products, or services does not imply any endorsement by the United States Government, the Department of Defense, or the Department of the Air Force.

Revision History

Version	Description	Generation Date
5.0a	Initial Public ASK 5.0a Release	21 APR 2026

Table of Contents

1	Vehicle Interface	11
1.1	Interface Description	11
1.2	Interface Interactions.....	12
1.2.1	Contingencies Package.....	12
1.2.2	Control and Tasking Package	18
1.2.3	COP Package.....	39
1.2.4	Data Validation Package	41
1.2.5	Route Plan Behaviors Package.....	47
1.2.6	Status Package	58
1.2.7	Weapon Employment Package	82
1.3	Interface Compliance.....	84
1.3.1	VI MMS	84
1.3.2	VI UCI Extensions	86
1.4	Mission and Flight Autonomy Capabilities V5	128

Table of Figures

Figure 1-1: Vehicle Interface Interactions	12
Figure 1-2: Collision Avoidance	13
Figure 1-3: Intra-Vehicle Comms Failure	14
Figure 1-4: MA Failsafe.....	15
Figure 1-5: Mechanical Damage Reporting.....	16
Figure 1-6: Sensor Failure	17
Figure 1-7: Control by Curve Following.....	20
Figure 1-8: Control by HSA/CSA Command	23
Figure 1-9: Control by Waypoint Following	26
Figure 1-10: Control Mode Authorization	28
Figure 1-11: MA-VI Command Task	30
Figure 1-12: Modify Capabilities.....	32
Figure 1-13: Receive Control Request.....	34
Figure 1-14: Unpair Control Assignment.....	36
Figure 1-15: Update C2 Control Designations	38
Figure 1-16: VI Updates to COP	40
Figure 1-17: Checksum Validation	42
Figure 1-18: Query for Missing Data	44
Figure 1-19: Route Plan Data Validation.....	46
Figure 1-20: Activate Route	48
Figure 1-21: Convert and Upload Route	51
Figure 1-22: Prepare for Route Activation.....	53
Figure 1-23: Receive Deactivate Route	54
Figure 1-24: Validate Route Plan	56
Figure 1-25: VI Deactivate Route.....	57
Figure 1-26: Exchange Heartbeat - Subsystem Status Reports.....	59
Figure 1-27: Publish Control Status	61
Figure 1-28: Query Airfield Update	63
Figure 1-29: Query Route Plan	66
Figure 1-30: Receive Barometric Pressure	68
Figure 1-31: Receive Execution Status.....	70
Figure 1-32: Receive Vehicle Performance Values.....	72

Figure 1-33: Receive Vehicle State Data	74
Figure 1-34: Request Terrain Data	77
Figure 1-35: Vehicle Status Reporting	79
Figure 1-36: VI Responds to Query for Flight Capabilities	81
Figure 1-37: Validate Release Envelope.....	83
Figure 1-38: Vehicle Interface Interactions	84

Table of Tables

Table 1-1: Collision Avoidance.....	13
Table 1-2: Intra-Vehicle Comms Failure	14
Table 1-3: MA Failsafe	15
Table 1-4: Mechanical Damage Reporting.....	16
Table 1-5: Sensor Failure	17
Table 1-6: Control by Curve Following	20
Table 1-7: Control by HSA/CSA Command	24
Table 1-8: Control by Waypoint Following	26
Table 1-9: Control Mode Authorization	29
Table 1-10: MA-VI Command Task.....	31
Table 1-11: Modify Capabilities.....	32
Table 1-12: Receive Control Request	35
Table 1-13: Unpair Control Assignment	36
Table 1-14: Update C2 Control Designations.....	38
Table 1-15: VI Updates to COP	41
Table 1-16: Checksum Validation	43
Table 1-17: Query for Missing Data	45
Table 1-18: Route Plan Data Validation	47
Table 1-19: Activate Route.....	48
Table 1-20: Convert and Upload Route	52
Table 1-21: Prepare for Route Activation	53
Table 1-22: Receive Deactivate Route	54
Table 1-23: Validate Route Plan	56
Table 1-24: VI Deactivate Route	57
Table 1-25: Exchange Heartbeat - Subsystem Status Reports.....	60
Table 1-26: Publish Control Status	61
Table 1-27: Query Airfield Update.....	64
Table 1-28: Query Route Plan	67
Table 1-29: Receive Barometric Pressure	68
Table 1-30: Receive Execution Status	71
Table 1-31: Receive Vehicle Performance Values	72
Table 1-32: Receive Vehicle State Data	74

Table 1-33: Request Terrain Data.....	78
Table 1-34: Vehicle Status Reporting	79
Table 1-35: VI Responds to Query for Flight Capabilities	81
Table 1-36: Validate Release Envelope	83
Table 1-37: Vehicle Interface Compliance Requirements	84
Table 1-38: VI MMS	85
Table A-1-39: Vehicle Interface MA_FlightCapabilityMT UCI Extension	87
Table A-1-40: Vehicle Interface MA_FlightCapabilityMT - HSA_CSA_PerformanceProfile UCI Extension Field	88
Table A-1-41: Vehicle Interface MA_FlightCapabilityMT - WaypointFollowingPerformanceProfile UCI Extension Field	89
Table A-1-42: Vehicle Interface MA_FlightCapabilityMT - CurveFollowingPerformanceProfile UCI Extension Field	90
Table A-1-43: Vehicle Interface MA_FlightCapabilityMT - AirspeedLimit UCI Extension Field	90
Table A-1-44: Vehicle Interface MA_FlightCapabilityMT - AccelerationLimit UCI Extension Field	91
Table A-1-45: Vehicle Interface MA_FlightCapabilityMT - AccelerationLimitPair UCI Extension Field	91
Table A-1-46: Vehicle Interface MA_FlightCapabilityMT - BodyReferenceOrientationRate UCI Extension Field	92
Table A-1-47: Vehicle Interface MA_FlightCapabilityMT - OrientationRateLimits UCI Extension Field	92
Table A-1-48: Vehicle Interface MA_FlightCapabilityMT - AirspeedPair UCI Extension Field	93
Table A-1-49: Vehicle Interface MA_FlightCommandMT UCI Extension	93
Table A-1-50: Vehicle Interface MA_FlightCommandMT - Command UCI Extension Field	94
Table A-1-51: Vehicle Interface MA_FlightCommandMT - Capability UCI Extension Field	95
Table A-1-52: Vehicle Interface MA_FlightCommandMT - HSA_CSA UCI Extension Field	96
Table A-1-53: Vehicle Interface MA_FlightCommandMT - WaypointFollowing UCI Extension Field ..	96
Table A-1-54: Vehicle Interface MA_FlightCommandMT - CurveFollowing UCI Extension Field	96
Table A-1-55: Vehicle Interface MA_FlightCommandMT - Speed UCI Extension Field	97
Table A-1-56: Vehicle Interface MA_FlightCommandMT - Heading UCI Extension Field	97
Table A-1-57: Vehicle Interface MA_FlightCommandMT - SpeedChoice UCI Extension Field	97
Table A-1-58: Vehicle Interface MA_FlightCommandMT - SpeedValue UCI Extension Field	98
Table A-1-59: Vehicle Interface MA_FlightCommandMT - CurveFitMethod UCI Extension Field	98
Table A-1-60: Vehicle Interface MA_FlightCommandMT - Reference UCI Extension Field	98
Table A-1-61: Vehicle Interface MA_FlightCommandMT - ControlPoint UCI Extension Field	99
Table A-1-62: Vehicle Interface MA_FlightCommandMT - RelativeAccelertaion UCI Extension Field ..	99

Table A-1-63: Vehicle Interface MA_FlightCommandMT - RelativeVelocity UCI Extension Field.....	100
Table A-1-64: Vehicle Interface MA_FlightCommandMT - OrientationRate UCI Extension Field	100
Table A-1-65: Vehicle Interface MA_FlightCommandMT - Direction UCI Extension Field	100
Table A-1-66: Vehicle Interface MA_FlightCommandMT - Course UCI Extension Field.....	101
Table A-1-67: Vehicle Interface MA_FlightCommandStatusMT UCI Extension	101
Table A-1-68: Vehicle Interface MA_FlightCommandStatusMT - MessageData UCI Extension Field	102
Table A-1-69: Vehicle Interface MA_FlightCommandStatusMT - CannotComplyDetails UCI Extension Field	102
Table A-1-70: Vehicle Interface MA_FlightCommandStatusMT - Severity UCI Extension Field	103
Table A-1-71: Vehicle Interface MA_FlightActivityMT UCI Extension	103
Table A-1-72: Vehicle Interface MA_FlightActivityMT - VehicleCommandState UCI Extension Field	104
Table A-1-73: Vehicle Interface MA_FlightActivityMT - Activity UCI Extension Field	105
Table A-1-74: Vehicle Interface MA_FlightActivityMT - Speed UCI Extension Field	105
Table A-1-75: Vehicle Interface MA_FlightActivityMT - Heading UCI Extension Field	106
Table A-1-76: Vehicle Interface MA_PositionReportDetailedMT UCI Extension.....	106
Table A-1-77: Vehicle Interface MA_PositionReportDetailedMT - PositionReportData UCI Extension Field	107
Table A-1-78: Vehicle Interface MA_PositionReportDetailedMT - MessageData UCI Extension Field	107
Table A-1-79: Vehicle Interface MA_FaultMT UCI Extension	108
Table A-1-80: Vehicle Interface MA_FaultMT - MessageData UCI Extension Field	108
Table A-1-81: Vehicle Interface MA_FaultMT - FaultInformation UCI Extension Field.....	109
Table A-1-82: Vehicle Interface MA_FaultMT - Severity UCI Extension Field.....	110
Table A-1-83: Vehicle Interface MA_TaskMT UCI Extension.....	110
Table A-1-84: Vehicle Interface MA_TaskMT - TaskType UCI Extension Field.....	112
Table A-1-85: Vehicle Interface MA_TaskMT - TaskPlurality UCI Extension Field	112
Table A-1-86: Vehicle Interface MA_TaskMT - ConstraintID UCI Extension Field.....	113
Table A-1-87: Vehicle Interface MA_TaskMT - HSA_CSA UCI Extension Field	113
Table A-1-88: Vehicle Interface MA_TaskMT - WaypointFollowing UCI Extension Field.....	113
Table A-1-89: Vehicle Interface MA_TaskMT - CurveFollowing UCI Extension Field	114
Table A-1-90: Vehicle Interface MA_TaskMT - Speed UCI Extension Field	114
Table A-1-91: Vehicle Interface MA_TaskMT - Heading UCI Extension Field	115
Table A-1-92: Vehicle Interface MA_TaskMT - SpeedChoice UCI Extension Field	115

Table A-1-93: Vehicle Interface MA_TaskMT - SpeedValue UCI Extension Field	115
Table A-1-94: Vehicle Interface MA_TaskMT - CurveFitMethod UCI Extension Field	116
Table A-1-95: Vehicle Interface MA_RoutePlanMT UCI Extension.....	116
Table A-1-96: Vehicle Interface MA_RoutePlanMT - MessageData UCI Extension Field.....	116
Table A-1-97: Vehicle Interface MA_SystemManagementRequestMT UCI Extension	117
Table A-1-98: Vehicle Interface MA_SystemManagementRequestStatusMT UCI Extension	117
Table A-1-99: Vehicle Interface MA_ControlRequestMT UCI Extension	117
Table A-1-100: Vehicle Interface MA_ControlRequestMT - MessageData UCI Extension Field.....	118
Table A-1-101: Vehicle Interface MA_ControlRequestMT - ControlAssignment UCI Extension Field	118
Table A-1-102: Vehicle Interface MA_ControlRequestMT - RequestType UCI Extension Field	119
Table A-1-103: Vehicle Interface MA_ControlRequestStatusMT UCI Extension	119
Table A-1-104: Vehicle Interface MA_ControlRequestStatusMT - MessageData UCI Extension Field	119
Table A-1-105: Vehicle Interface MA_ControlAssignmentMT UCI Extension	120
Table A-1-106: Vehicle Interface MA_ControlAssignmentMT - MessageData UCI Extension Field ..	120
Table A-1-107: Vehicle Interface MA_ControlAssignmentMT - ControlAssignment UCI Extension Field	120
Table A-1-108: Vehicle Interface MA_MissionPlanActivationCommandMT UCI Extension	121
Table A-1-109: Vehicle Interface MA_MissionPlanActivationCommandStatusMT UCI Extension	121
Table A-1-110: Vehicle Interface MA_FlightCapabilityStatusMT UCI Extension.....	121
Table A-1-111: Vehicle Interface MA_FlightCapabilityStatusMT - Capability UCI Extension Field ...	122
Table A-1-112: Vehicle Interface MA_FlightCapabilityStatusMT - MessageData UCI Extension Field	122
Table A-1-113: Vehicle Interface MA_SystemNotificationMT UCI Extension.....	123
Table A-1-114: Vehicle Interface MA_SystemNotificationMT - MessageData UCI Extension Field..	124
Table A-1-115: Vehicle Interface MA_SystemNotificationMT - NotificationDetail UCI Extension Field	124
Table A-1-116: Vehicle Interface MA_ResponseMT UCI Extension	124
Table A-1-117: Vehicle Interface MA_ResponseMT - MessageData UCI Extension Field	126
Table A-1-118: Vehicle Interface MA_MissionPlanExecutionStatusMT UCI Extension	126
Table A-1-119: Vehicle Interface MA_TaskCommandStatusMT UCI Extension	126
Table A-1-120: Vehicle Interface MA_TaskCommandStatusMT - MessageData UCI Extension Field	127
Table A-1-121: Vehicle Interface MA_TaskCommandMT UCI Extension	127

Table A-1-122: Vehicle Interface MA_TaskCommandMT - MessageData UCI Extension Field	127
Table A-1-123: Vehicle Interface MA_ActionStatusMT UCI Extension	128
Table A-1-124: Mission and Flight Autonomy Capabilities V5	128

1 VEHICLE INTERFACE

The VI defines how MA commands vehicle motion to the platform. From MA's perspective, the VI is a system that includes guidance, navigation, high- and low-level flight control algorithms, and any flight safety-related functionality, some of which may be beyond the scope of a traditional VMS. MA focuses purely on mission execution by providing commands to the VI, but MA does not handle safety-critical functions; the VI must accept or reject commands as necessary to ensure safe operation.

The interface between MA and VI should act as an outgrowth of the VI. While MA is not responsible for safety-critical tasks, it is responsible for execution of mission-critical tasks, and the interface between MA and the VI is the primary gateway for control commands, vehicle state data, and other important parameters.

1.1 Interface Description

The Vehicle Interface (VI) defines the interface and architectural components required for interactions between Mission Autonomy (MA) and the vehicle, including any vendor-specific VI with vehicle control, command validation, and other capabilities. Focus is given to defining the boundary between MA and the VI and establishing the messaging patterns between the two.

The VI defines a set of messages and interactions for UCI communication between the MA and the VI OMS Isolator. This isolator is the primary conduit for data: all UCI communications from the OMS Mission Package to the safety-critical enclave are routed through it, and the vehicle will also pass data to MA without a specific request via the same component. The VI OMS Isolator is responsible for publishing and subscribing to the topics and messages detailed in the VI ICD section.

The separation between non-safety-critical systems (like the OMS Mission Package) and the safety-critical enclave is called the "Airworthiness Boundary". Systems assumed to be within this enclave could include the Vehicle Management System (VMS), Flight Autonomy (FA), or any other onboard system deemed safety-critical. MA makes several assumptions about functionality within this architecture; however, the VI ICD outlines interactions that require functionality resident within the VI OMS Isolator, the safety-critical enclave, or both. The definition of the external-facing boundary of the VI OMS Isolator is a platform-specific consideration potentially subject to certification. A nominal OMS Mission Package configuration containing MA and the VI OMS Isolator is depicted in the figure below.

It is assumed that the VI OMS Isolator will contain functions like a message validator and a platform adapter to interface with non-UCI compliant systems. However, the message validation strategy could be federated between the VI OMS Isolator and Flight Autonomy (FA). While this does not preclude MA from doing its own message validation, this validation is not considered safety-critical. A nominal breakdown of the validation functions is shown in the figure below.

The MA-FA interface is contained within the MA-VI interface, where specific safety-critical functionality within FA is considered read-only for MA. For example, FA is responsible for adhering to geofences while MA is responsible for adhering to dynamically managed geozones. The vehicle's FA handles a set of safety-critical, vehicle-specific, and last-second contingencies that can be reported to MA. Flight Autonomy notifies MA of contingencies and remedies taken that affect mission outcomes, such as fuel range or endurance, reduced speed/range, sensor failures, etc. Additional functionality, such as JPALS, is considered safety-critical, is located within FA, and does not interact directly with MA.

To ensure A-GRA compliance, vehicle manufacturers, VI providers, and MA software implementers should refer to this section to ensure that their systems support the required messages and interactions.

1.2 Interface Interactions

The VI interactions with MA have been scoped as shown in the below figure. Within interaction, there is a baseline set of functionalities that must be supported by VI and MA for a compliant system.

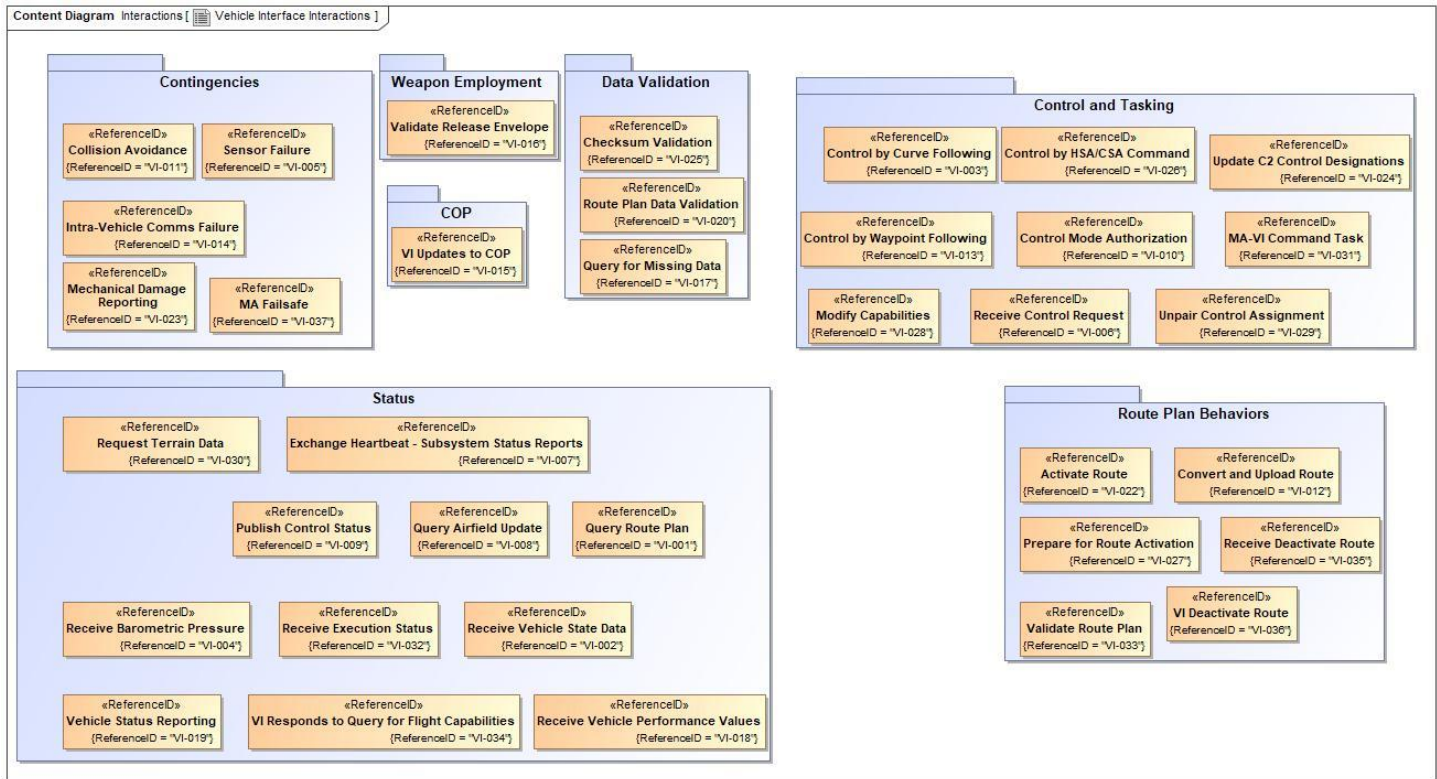


Figure 1-1: Vehicle Interface Interactions

1.2.1 Contingencies Package

1.2.1.1 Collision Avoidance

If VI is required to take safety critical action, VI will perform the necessary actions and inform MA that new actions are not being accepted due to evasive maneuvers being performed through the *MA_FlightCapabilityStatusMT* message in the *CapabilityStatus.AvailabilityInfo.Availability* field. VI will also provide the reason for denying new commands in the *MA_FlightCapabilityStatusMT.AvailabilityInfo.AvailabilityReason.Reason* field filled with *CONSTRAINT_COLLISION_AVOIDANCE*. After evasive maneuvers are performed, VI will inform MA of actions performed and resume accepting necessary tasking/messages from MA through the *MA_FlightCapabilityMT*.

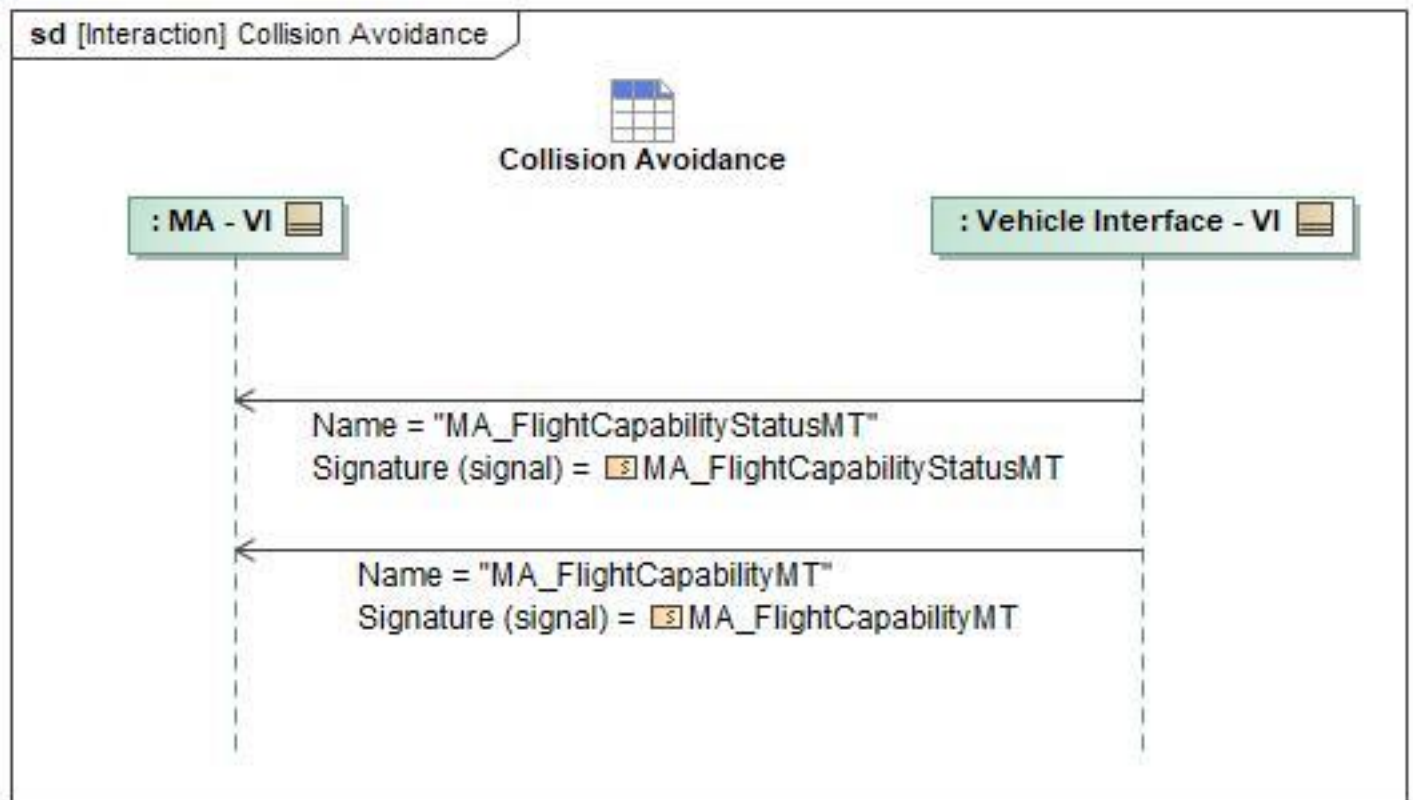


Figure 1-2: Collision Avoidance

Table 1-1: Collision Avoidance

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	AvailabilityReason: MA_CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.1.2 Intra-Vehicle Comms Failure

Vehicle Interface (VI) will provide regular status to Mission Autonomy (MA) via the *SubsystemStatusMT_1* UCI message. If MA doesn't receive a status from VI within the designated time frame, MA will request status from VI via the *SubsystemStatusDataRequestMT* UCI message. VI will continue to attempt to report status via the *SubsystemStatusMT_2* UCI message. If communication between MA and VI is not repaired within a specified time window, MA will refer to and follow the on-board contingency plan associated with a loss of communication with VI.

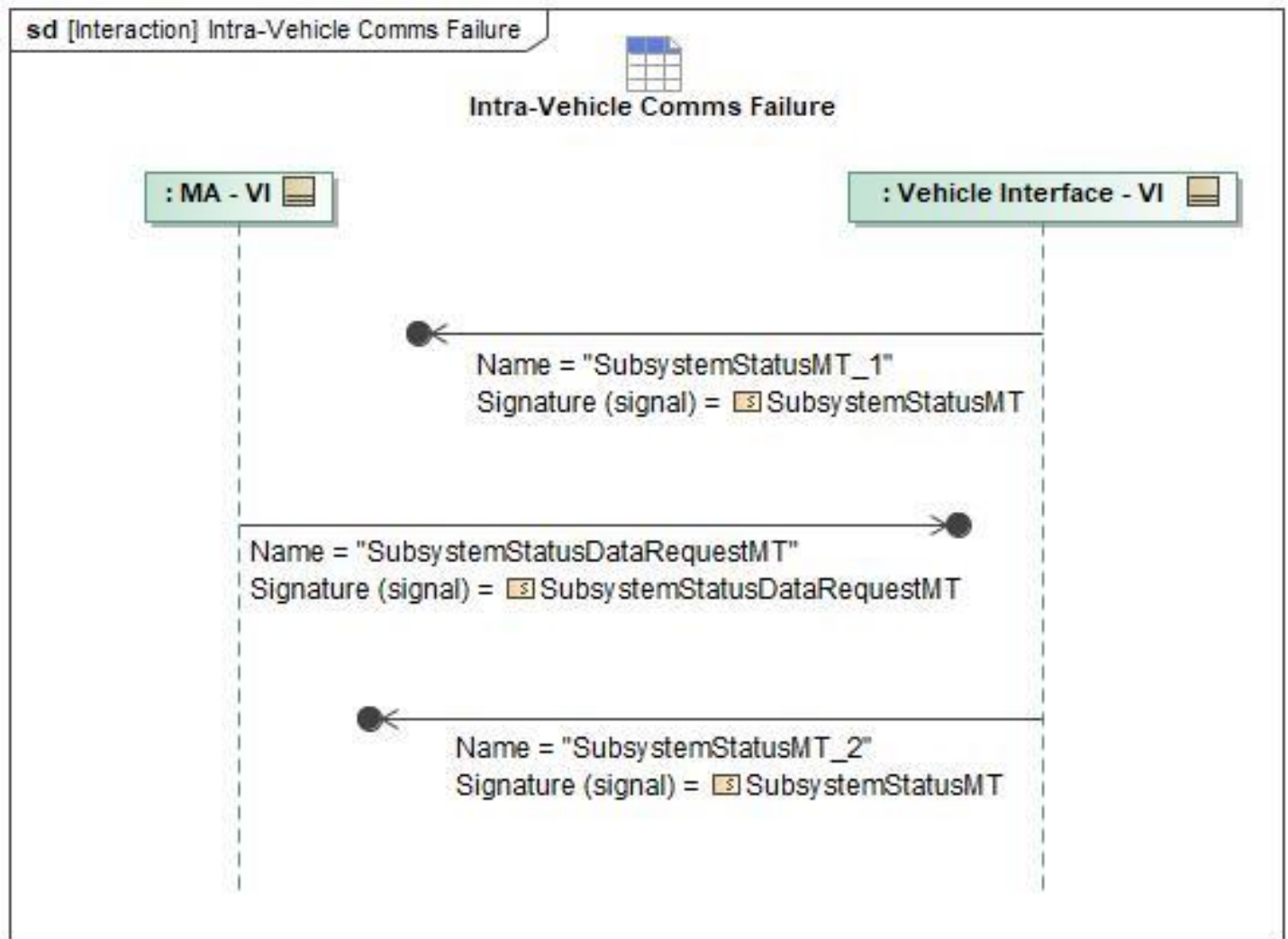


Figure 1-3: Intra-Vehicle Comms Failure

Table 1-2: Intra-Vehicle Comms Failure

Message Name (Name:Type)	Required Fields (Name:Type)
SubsystemStatusDataRequestMT: SubsystemStatusDataRequestMT	None
SubsystemStatusMT_1: SubsystemStatusMT	None
SubsystemStatusMT_2: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.1.3 MA Failsafe

This behavior enables MA to pick a "failsafe plan", a plan for how FA should attempt to recover the ACP in case of MA failure.

MA will send an *MA_ResponseMT* to FA (VI) to describe the failsafe plan. If FA already has an MA failsafe plan in place, this *MA_ResponseMT* can be used to update the plan by using the same

ResponseID as the current plan and by incrementing the *ResponseID.Version*. The *Option.Trigger* set to *System* and the *Option.Trigger.System.SystemID* populated with the UUID for Mission Autonomy will be used to convey that this response is the failsafe plan. *ResponseType* will be set to *PLAN_ACTIVATION*, and as such the *Option.Response* will be set to *ActivatePlan*, with *Option.Response.ActivatePlan.ActivationDetails* set to *BySubPlan* and populated with *RoutePlans*. The *RoutePlans* must at minimum include one of the recovery route plans planned and stored on FA pre-mission as the terminal point of the failsafe plan. Additional transit route plans can also be included in the *RoutePlans* to get the ACP from its current position to the beginning of the pre-planned recovery route plan if MA has created said transit route plans as a *MA_RoutePlanMT* and sent them to FA prior to sending this *MA_ResponseMT*.

To acknowledge receipt of the *MA_ResponseMT* FA (VI) will respond by sending MA the *MA_SystemNotificationMT* message, referencing the *SystemSubjectID*, *SystemPerspective*, and *AssociatedMessage* fields.

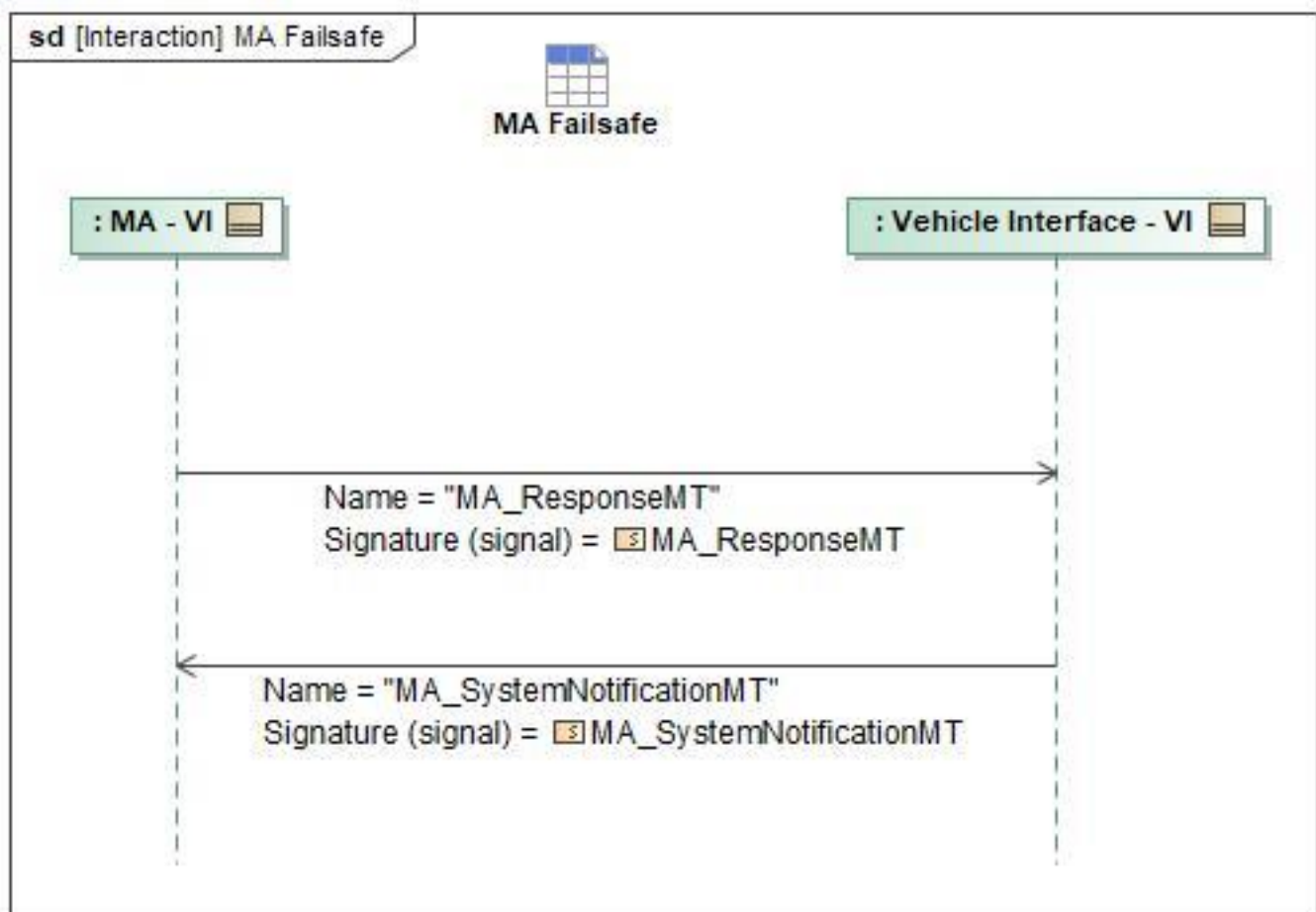


Figure 1-4: MA Failsafe

Table 1-3: MA Failsafe

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ResponseMT: MA_ResponseMT	System: SystemFilterType SystemID: SystemID_Type ActivatePlan: MA_MissionPlanActivationCommandType

	BySubPlan: MA_MissionPlanSubplanActivationType RoutePlan: RoutePlanActivationType
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedMessage: AssociatedMessageType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.1.4 Mechanical Damage Reporting

Vehicle Interface (VI) will perform periodic BITs. If any mechanical damage is identified, VI will report fault data to Mission Autonomy via the MA_FaultMT including data about what the fault was, affected services, severity, and what the subsystems capability is currently. Examples of mechanical damage could include actuator limitations, airframe damage detected by inputs required to fly, etc.

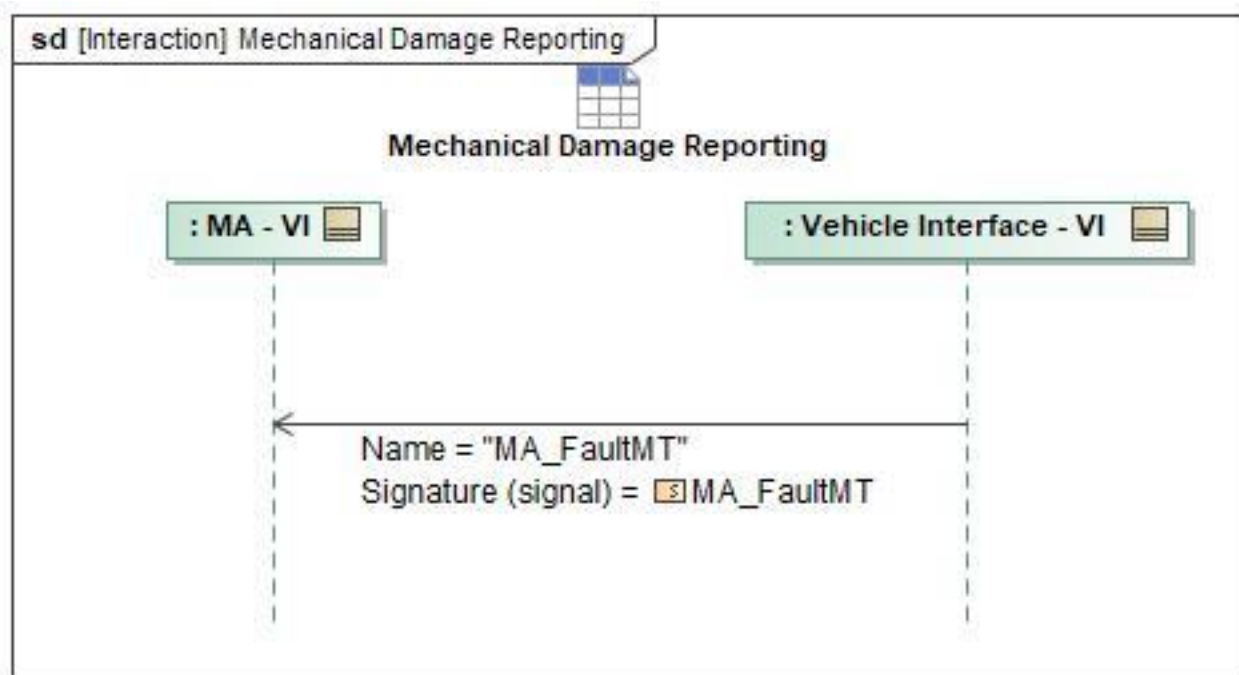


Figure 1-5: Mechanical Damage Reporting

Table 1-4: Mechanical Damage Reporting

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FaultMT: MA_FaultMT	FaultDescription: VisibleString256Type Severity: MA_FaultSeverityEnum FaultCodeCount: int FaultData: FaultDataType AmbiguityGroup: SubsystemFaultAmbiguityGroupType ServiceID: ServiceID_Type DetectionTime: DateTimeType FaultState: FaultStateEnum

SubsystemID: SubsystemID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.1.5 Sensor Failure

Vehicle Interface (VI) will report subsystem status via the *SubsystemStatusMT* UCI message. If a fault is detected, VI will report the fault and necessary fault detail to Mission Autonomy (MA) via the *MA_FaultMT*.

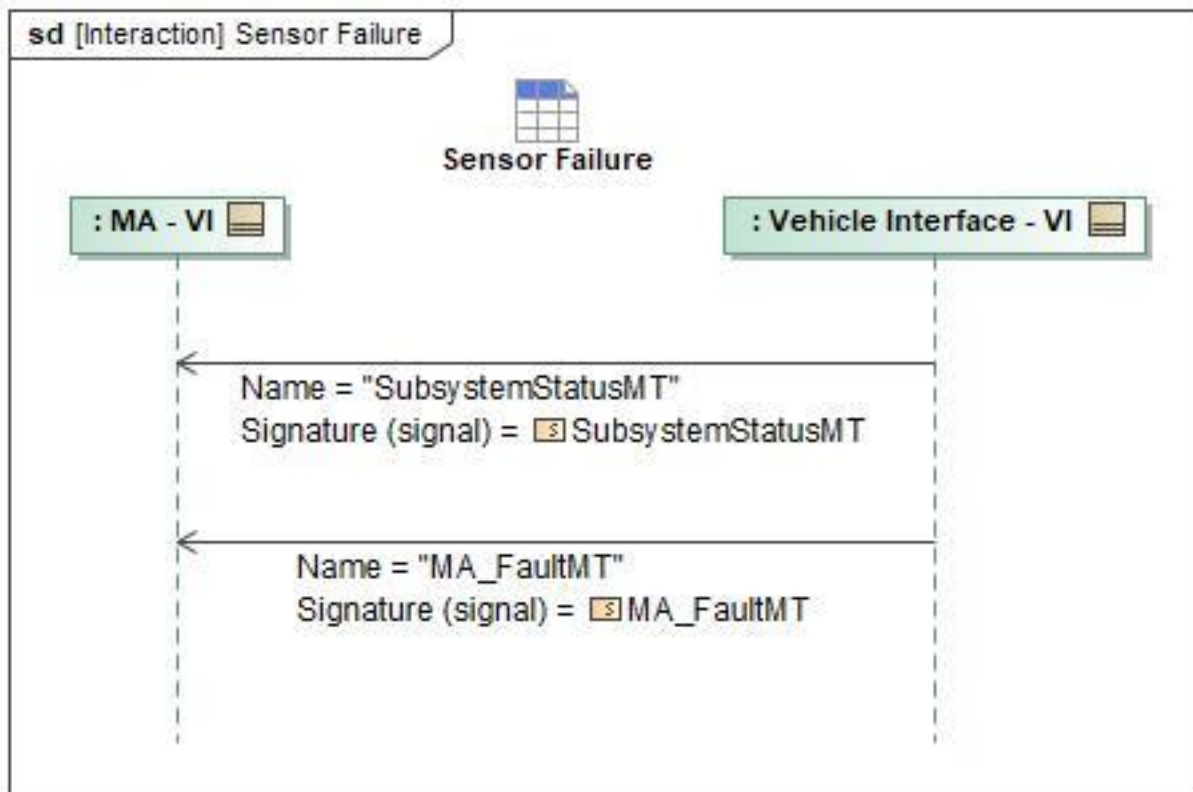


Figure 1-6: Sensor Failure

Table 1-5: Sensor Failure

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FaultMT: MA_FaultMT	FaultCodeCount: int FaultState: FaultStateEnum AmbiguityGroup: SubsystemFaultAmbiguityGroupType Severity: MA_FaultSeverityEnum ServiceID: ServiceID_Type DetectionTime: DateTimeType FaultData: FaultDataType SubsystemID: SubsystemID_Type FaultDescription: VisibleString256Type

SubsystemStatusMT: SubsystemStatusMT	None
--------------------------------------	------

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2 Control and Tasking Package

1.2.2.1 Control by Curve Following

Curve Following control mode enables MA to command more dynamic maneuvers than are possible through either *Waypoint Following* or *HSA/CSA* control. The command paths are specified as curves relative to the *ownship's* local NED coordinates. Each commanded path is composed of 5th-order *Bézier curves*, described by six control points per segment. For *Bézier curves*, the curve passes through the first and last control points, but the curve does not generally pass through the four interior control points. The first control point of a segment matches the last control point of the previous segment to produce a piecewise continuous curve (i.e. C0).

The *MA_FlightCommandMT* message contains the data required to execute the *Curve Following* control mode via the *MA_CurveControlType* field. For a given curve following command, *MA_CurveControlType* may specify a maximum of ten curve segments. Each curve segment defines the six control points using the *MA_NURBS_PointType*, which defines the *CenterReference*, the list of six *ControlPoints*, and a list of *KnotVector* values.

- *CenterReference* indicates the reference from which the control points are defined, representing the *ownship*.
- *ControlPoints* define six control points relative to *CenterReference*, with a *Weight* field set to 1.0 to represent *Bézier curves*.
- *KnotVector* is defined as [0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1] to represent a 5th degree *Bézier curve* using six control points. This clamped knot vector ensures the curve begins at the first control point and ends at the last, with no internal knots, resulting in a single-span *Bézier curve*.

MA_CurveControlType may optionally include the *AppendCurve* field to specify the commanded curve is to be appended to the currently active curve following command. This optional field is therefore only valid when the vehicle is currently executing a curve following command. The appended curve following command will use the *ownship CenterReference* of the preceding curve following command. If the *AppendCurve* field is not specified, the vehicle will issue a new curve following command, terminating any active commands. *MA_CurveControlType* may also optionally specify *EndOfCurveBehavior* to specify the vehicle's behavior at the completion of the curve following command. These behaviors include:

- Continue at the previous Course-Speed-Altitude (CSA) state.
- Begin a circular loiter pattern centered around the curve's endpoint.

In addition, *MA_CurveTraversingType* allows for specification of how long an ACP has to traverse the curve or what the minimum and maximum speed it should use to traverse the curve.

If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* can optionally provide the *CannotComplyDetails* field to include further details on why the command was rejected. The *ValidationResult* field provides the reason or multiple reasons for rejection: *INVALID_CURVE*, *INVALID_WAYPOINT*,

PERFORMANCE_LIMIT_EXCEEDED, *VIOLATION_ENDURANCE*, *VIOLATION_GEOFENCE*, *VIOLATION_AIR_TRAFFIC*, *VIOLATION_TERRAIN*, and *CAPABILITY_NOT_SUPPORTED*. For each rejection reason enumeration, additional context can be optionally provided by FA with the *MA_ValidationResultReasonType*. Each rejection reason enumeration has an analogous field that provides more information on the rejection. The *MA_ValidationResultReasonType* fields include details such as the specific expected invalid curve section of a commanded curve, the expected invalid segment of a commanded path, the performance constraints that are expected to be violated by a flight command, the expected insufficient endurance details including the remaining and required endurance, the specific geofence IDs, entity IDs, or terrain points that FA expects the flight command to violate, as well as a plain-text description for any additional context required. Specifically for curve following rejections, the *MA_CurveSectionType* details the *StartCurveParameterValue*, *EndCurveParameterValue* and the *SegmentNumber* to communicate the specific section of the curve expected to be invalid.

VI may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and *Flight* field. *MA_TaskMT* is used to encapsulate a suggested flight task from VI and gives VI the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the VI's "best-effort" suggestion or choose an entirely different flight command. In the case of a *FAILED ActivityState*, if a best effort *MA_TaskMT* is provided, VI may also optionally reference the UUID of the suggested task in the *ActivityReason.AssociatedID* field of the *FAILED MA_FlightActivityMT*.

If the command is *ACCEPTED*, the VI will then begin publishing the *MA_FlightActivityMT* message, which includes data on the commanded state of the *ownership*, the execution state of the command described initially in *MA_CurveControlType*, and the status of the curve following, including segment number and percentage complete. The command cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until the sequence is terminated. The sequence terminates if a command is *REJECTED*, if the activity state is *COMPLETED* or *FAILED*, if control is taken away from MA, or if MA relinquishes control to the VI.

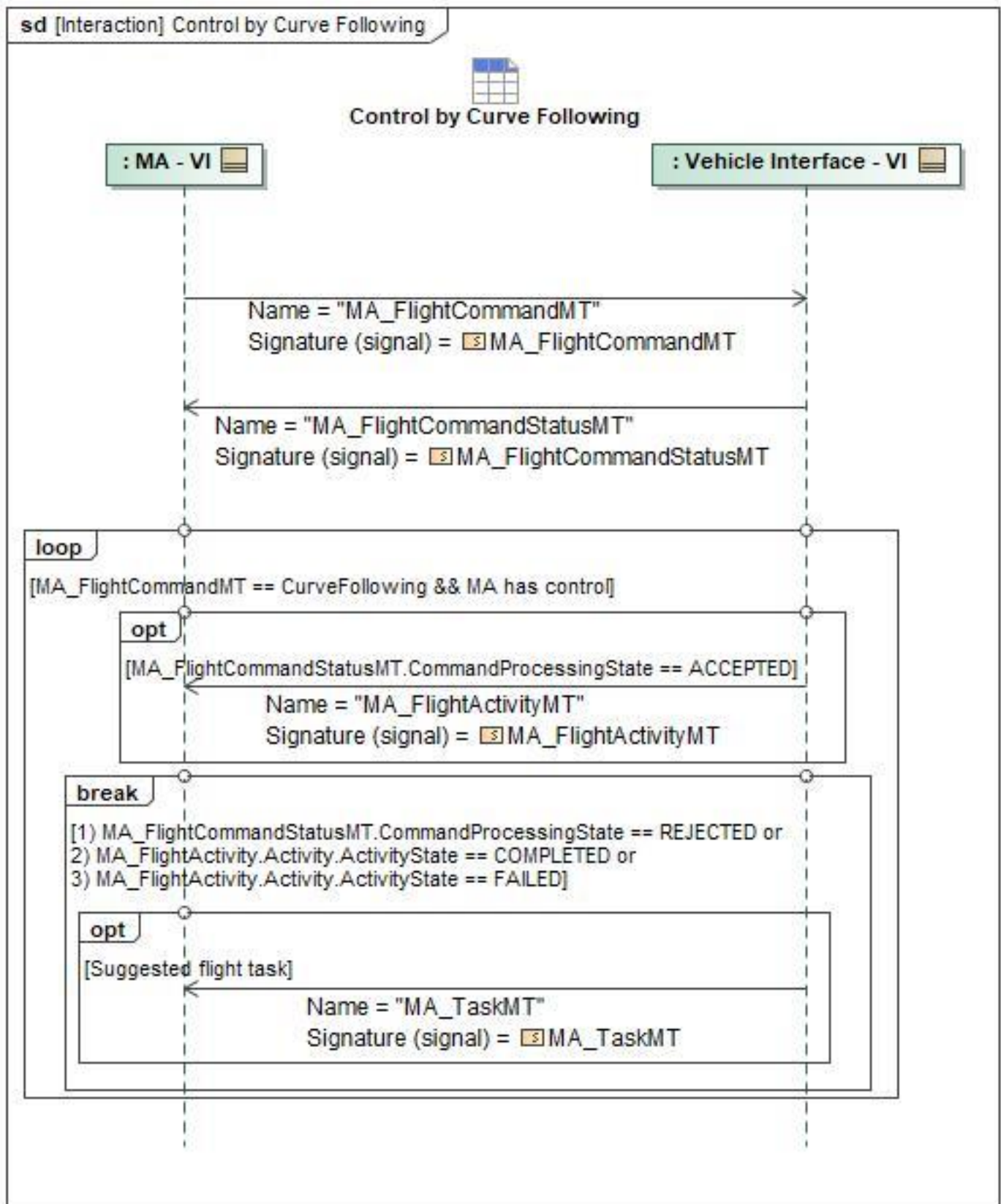


Figure 1-7: Control by Curve Following

Table 1-6: Control by Curve Following

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	CurveFollowing: MA_CurveControlType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	Flight: MA_FlightTaskType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2.2 Control by HSA/CSA Command

Heading-Speed-Altitude/Course-Speed-Altitude (HSA/CSA) control mode commands the vehicle to achieve supplied heading or course, speed, and altitude values. Heading or course can be specified as the desired direction vector. The vehicle tracks the supplied command data subject to its flight envelope and environmental conditions. No rate is specified or required for this control mode. The HSA/CSA control mode is persistent and does not have a defined completion state; the vehicle will continue using the current HSA/CSA command until it is terminated or interrupted by another command. Vehicle motion (turn rate, acceleration, and altitude change behavior) to achieve a given HSA command is left unspecified by Mission Autonomy

Partial HSA/CSA commands are allowed to update the most recent commanded heading or course, speed, and altitude values. For instance, only an altitude can be specified if it is the only change desired. Furthermore, the speed can be specified in terms of Mach number or some other reference, or can be left up to the VI to be optimized according to desired supported enumerations

HSA/CSA Altitude is specified using the standard UCI *AltitudeReferenceType*, along with an altitude value.

HSA/CSA direction is specified using an extended UCI type to specify a *MA_DirectionReferenceType* (true and magnetic north), along with a desired heading or course value.

HSA Speed can be specified using a value, along with the standard UCI *SpeedReferenceEnum* to indicate if the value is based on the ground speed or true air speed, or alternatively *MachType* which indicates speed as a unitless Mach number. Alternatively, the commanded airspeed can be passed in via *SpeedOptimizationEnum*, which supports max cruise or max endurance.

The Vehicle will perform validity checking on the *MA_FlightCommandMT* and accept or reject the command using the *MA_FlightCommandStatusMT* message based on the results of validity checks. If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* can optionally provide the *CannotComplyDetails* field to include further details on why the command was rejected. The *ValidationResult* field provides the reason or multiple reasons for rejection: *INVALID_CURVE*, *INVALID_WAYPOINT*, *PERFORMANCE_LIMIT_EXCEEDED*, *VIOLATION_ENDURANCE*, *VIOLATION_GEOFENCE*, *VIOLATION_AIR_TRAFFIC*, *VIOLATION_TERRAIN*, and *CAPABILITY_NOT_SUPPORTED*. For each rejection reason enumeration, additional context can be optionally provided by FA with the *MA_ValidationResultReasonType*. Each rejection reason enumeration has an analogous field that provides more information on the rejection. The *MA_ValidationResultReasonType* fields include details such as the specific expected invalid curve section of a commanded curve, the expected

invalid segment of a commanded path, the performance constraints that are expected to be violated by a flight command, the expected insufficient endurance details including the remaining and required endurance, the specific geofence IDs, entity IDs, or terrain points that FA expects the flight command to violate, as well as a plain-text description for any additional context required.

VI may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and *Flight* field. *MA_TaskMT* is used to encapsulate a suggested flight task from VI and gives VI the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the VI's "best-effort" suggestion or choose an entirely different flight command. In the case of a *FAILED ActivityState*, if a best effort *MA_TaskMT* is provided, VI may also optionally reference the UUID of the suggested task in the *ActivityReason.AssociatedID* field of the *FAILED MA_FlightActivityMT*.

If the command is *ACCEPTED*, the VI will then begin publishing the *MA_FlightActivityMT* report, which includes data on the commanded state of the ownship, and the execution state of the command described initially in *MA_FlightCommandMT*. The cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until the sequence is terminated. The sequence terminates if a command is *REJECTED*, if the activity state is *COMPLETED* or *FAILED*, if control is taken away from MA, or if MA relinquishes control to the VI.

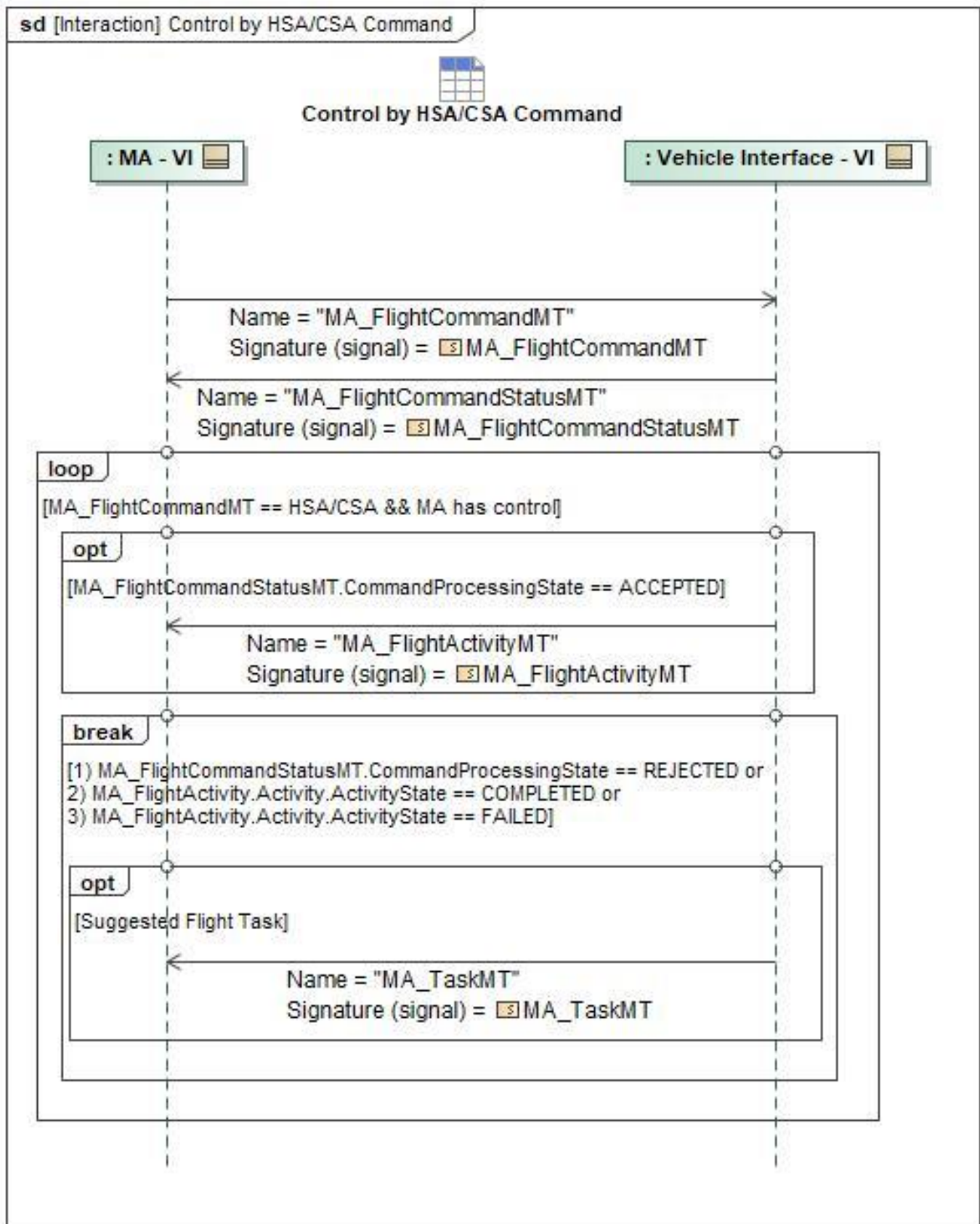


Figure 1-8: Control by HSA/CSA Command

Table 1-7: Control by HSA/CSA Command

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	HSA_CSA: MA_HSA_CSA_Type
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	Flight: MA_FlightTaskType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2.3 Control by Waypoint Following

To ensure safe operation, the Flight Autonomy (FA) has final authority over the vehicle. Before Mission Autonomy (MA) can issue commands, the FA must publish its set of available control modes and signal when it is ready to accept them.

This is a two-step process: First, the FA broadcasts the *FA_FlightCapabilityMT* message. This message describes the types of control available (e.g., waypoint following, curve following) and includes the specific performance parameters (i.e., *FlightCapabilityPerformanceProfile*) MA needs to use them. Second, the FA sends the *FA_FlightCapabilityStatusMT* message. This acts as a ready signal, formally telling MA when it can begin issuing commands.

Finally, once MA receives this ready signal, it can take control by sending commands to the FA.

Waypoint following, alternatively referred to as route following, is a control mode where MA provides the VI a series of waypoints describing a desired flight path. MA may define and send multiple routes to the VI, but a specific route is activated for execution via a command. The vehicle validates the route(s) based on flight envelope and environmental conditions and provides updates regarding route completion. MA may also terminate the route following activity by issuing a new command that supersedes it. Waypoint behavior may include simple commands such as fly-by or fly-through; complex commands including payload action and loiter/hold; or bounding constraints such as vehicle attitude or timing.

For taxi routes, the *PathType* will be designated as *TAXI*, and with *PathID* referencing a validated taxi path. The vehicle validates the taxi route(s) based on the vehicle performance conditions (e.g. taxi speed, turn rate, etc.) and provides updates regarding route completion. This allows MA to send commands to FA to taxi based upon conditions safely achievable by the vehicle along a route. It is understood that the associated taxi commands and constraints are only relevant when the aircraft is on the ground.

For loiter types, *Circle*, *Racetrack*, *Figure-eight*, or *ATC hold* may be selected, along with duration, direction, and other parameters to define the pattern. A *Circle* defines the radius, center point, and relative offset if applicable. For the *Figure-eight*, two circles define the loops with the appropriate relative offsets. For the *Racetrack*, two circle types define the semi-circle ends of the *Racetrack* connected by legs. For the *ATC hold*, two circles also define the semi-circle ends of the *Racetrack*, but with the appropriate relative offsets to accommodate the holding fix. To command a specific loiter pattern outside of *Circles*, *Racetrack*, *Figure-Eight*, or *ATC Hold*, use the *Point InboundDirection*, *LegLength*, *TurnGeometry*, and *TurnDirection* fields within the *MA_FixOrbitType* portion of the

MA_FlightCommand message. This allows MA to command specific holds based upon a relative point with defined leg lengths, turns, directions, and geometries to provide a more refined pattern when needed. If no inbound course is specified with the *ATC hold*, MA will use an inbound course equal to the arrival course. If no turn direction is specified, MA will use right turns by default. If no leg length is specified, the inbound leg length will be based on timing: 60 seconds, when at or below 14,000 ft MSL; or 90 seconds, when above.

Communication of new waypoints begins with the transmission from MA to the VI of *MA_FlightCommandMT* to begin receiving a *Waypoint Following* command. A well-conditioned *MA_WaypointFollowingType* message will simply include a *RouteType* message for the VI to execute. Detailed information on the contents of *RouteType* and *RoutePathType* are available within the OMS UCI 2.3 schema. The Vehicle will perform validity checking on the *MA_FlightCommandMT* and accept or reject the command using the *MA_FlightCommandStatusMT* message based on the results of validity checks.

If the command is *REJECTED*, the loop in the sequence will break and the *MA_FlightCommandStatusMT* can optionally provide the *CannotComplyDetails* field to include further details on why the command was rejected. The *ValidationResult* field provides the reason or multiple reasons for rejection: *INVALID_CURVE*, *INVALID_WAYPOINT*, *PERFORMANCE_LIMIT_EXCEEDED*, *VIOLATION_ENDURANCE*, *VIOLATION_GEOFENCE*, *VIOLATION_AIR_TRAFFIC*, *VIOLATION_TERRAIN*, and *CAPABILITY_NOT_SUPPORTED*. For each rejection reason enumeration, additional context can be optionally provided by FA with the *MA_ValidationResultReasonType*. Each rejection reason enumeration has an analogous field that provides more information on the rejection. The *MA_ValidationResultReasonType* fields include details such as the specific expected invalid curve section of a commanded curve, the expected invalid segment of a commanded path, the performance constraints that are expected to be violated by a flight command, the expected insufficient endurance details including the remaining and required endurance, the specific geofence IDs, entity IDs, or terrain points that FA expects the flight command to violate, as well as a plain-text description for any additional context required. Specifically for waypoint following rejections, the *RouteValidationInvalidPathType* details the *PathID* and *PathSegmentID* to communicate the specific segment of the path expected to be invalid.

VI may optionally specify a suggested flight task with adjusted parameters that constitute a "best-effort" using the *MA_TaskMT* message and *Flight* field. *MA_TaskMT* is used to encapsulate a suggested flight task from VI and gives VI the ability to define the parameters of the command that could be accepted. MA can restart the sequence with the VI's "best-effort" suggestion or choose an entirely different flight command.

If the command is *ACCEPTED*, the VI will then begin publishing the *MA_FlightActivityMT* report, which includes data on the commanded state of the ownship, and the execution state of the command described initially in *MA_FlightCommandMT*. The command cycle of *MA_FlightCommandMT* > *MA_FlightCommandStatusMT* > *MA_FlightActivityMT* repeats until the sequence is terminated. The sequence terminates if a command is *REJECTED*, if the activity state is *COMPLETED* or *FAILED*, if control is taken away from MA, or if MA relinquishes control to the VI.

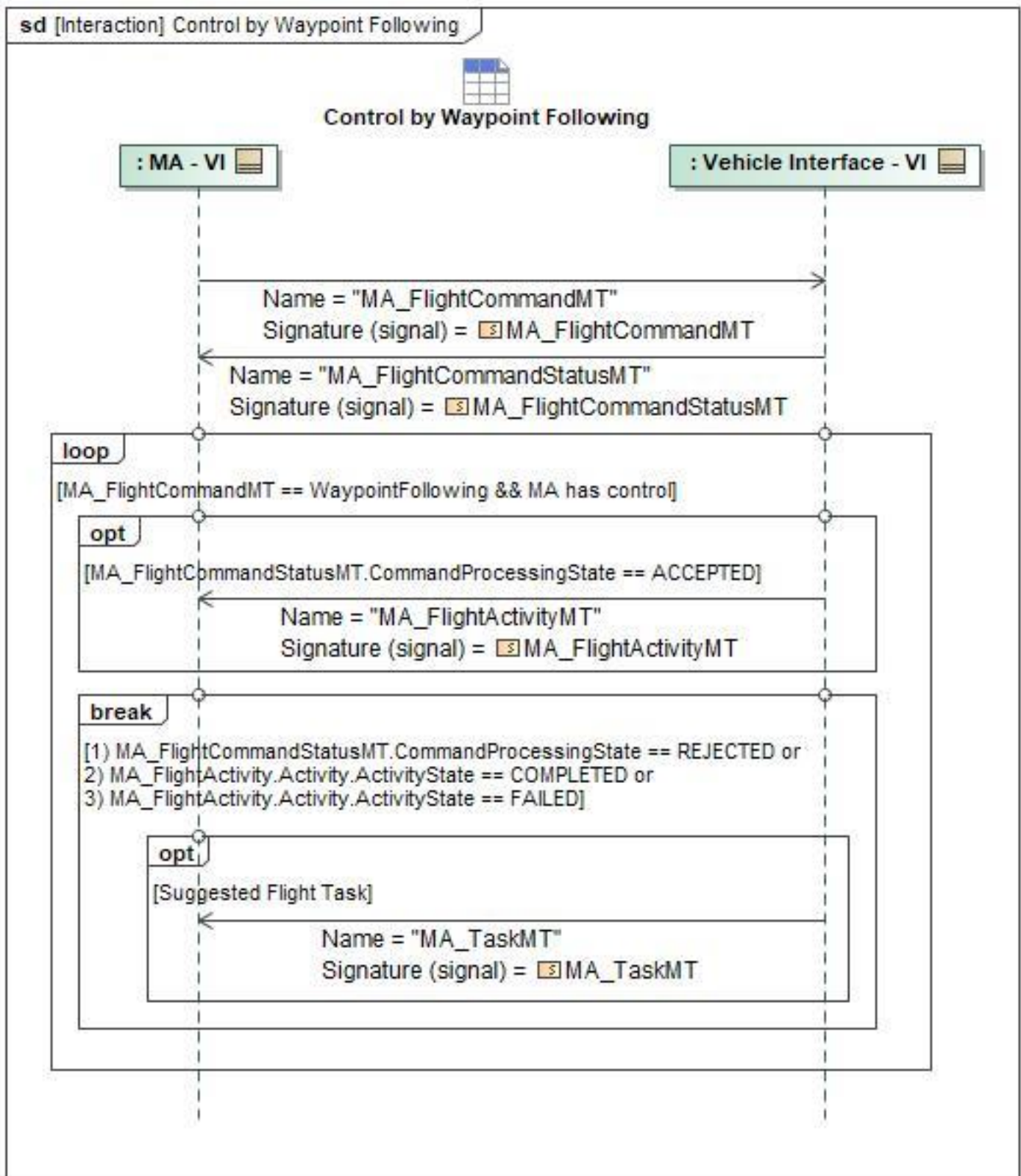


Figure 1-9: Control by Waypoint Following

Table 1-8: Control by Waypoint Following

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCommandMT: MA_FlightCommandMT	WaypointFollowing: MA_WaypointFollowingType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_TaskMT: MA_TaskMT	Flight: MA_FlightTaskType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2.4 Control Mode Authorization

The VI is the ultimate arbitrator of control authority over the ownship. Therefore, for MA to command the VI, the VI must first offer control once it is available. The *MA_FlightCapabilityMT* message published by the VI outlines which control modes are available to MA. The *MA_FlightCapabilityMT* message requires the *FlightCapabilityPerformanceProfile* field be populated with the appropriate subtypes of the capabilities advertised and any parameters needed by MA for commanding each type present. Additionally, *MA_FlightCapabilityStatusMT* signals availability of control from the VI to MA. Once ready, MA can issue commands to the VI via the Control by HSA/CSA Command use case, Control by Waypoint Following use case, or Control by Curve Following use case.

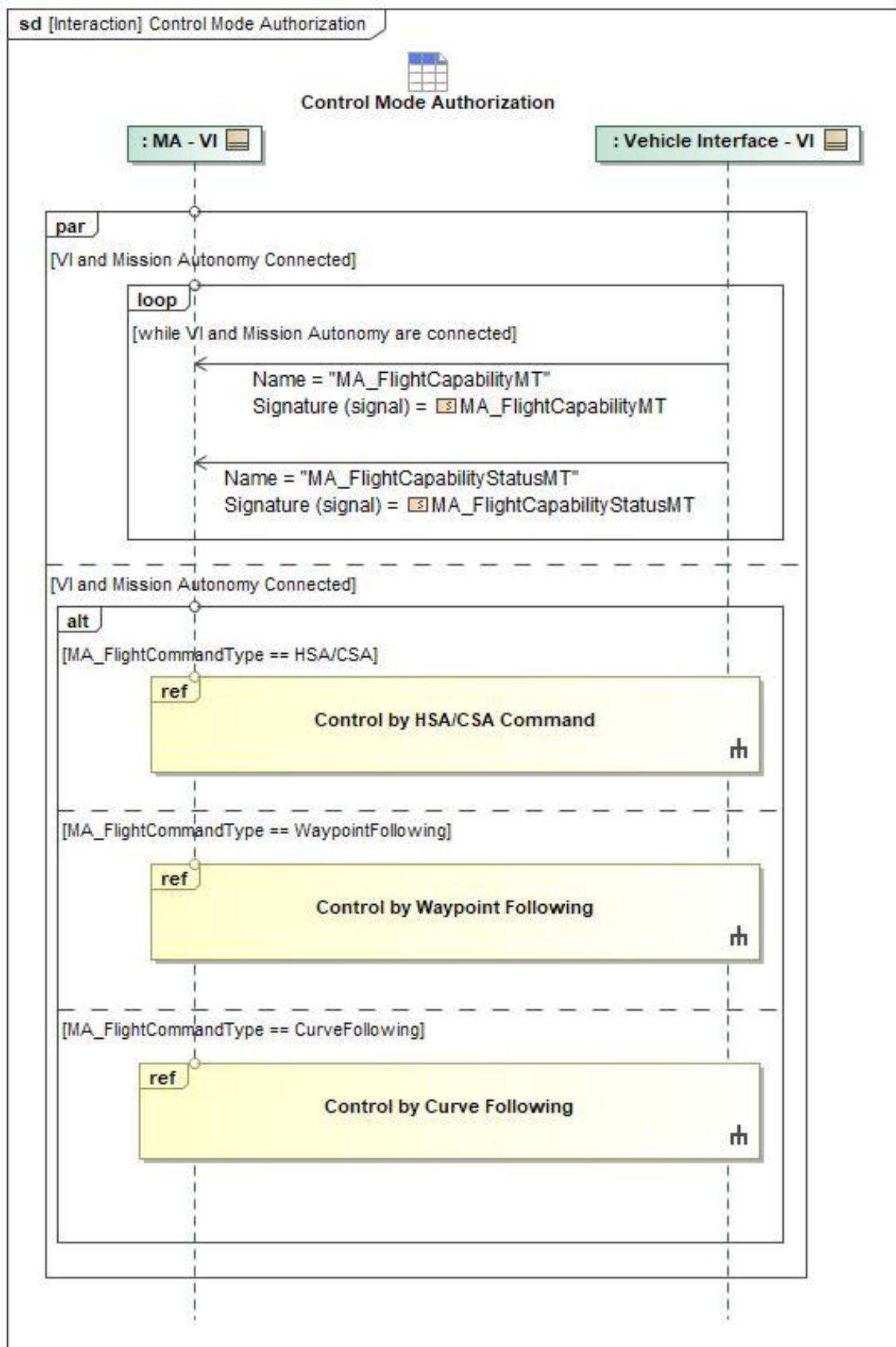


Figure 1-10: Control Mode Authorization

Table 1-9: Control Mode Authorization

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2.5 MA-VI Command Task

MA can command VI using the UCI *MA_Task* construct. When MA has the intent to command VI via *MA_Task*, MA first needs to make sure that VI has the *MA_Task* of *TaskID* that MA intends to command. In the event that MA has not already sent VI the *MA_TaskMT*, MA sends *MA_TaskMT* to VI, and VI responds with *MA_SystemNotificationMT*. Once VI has the *MA_Task*, MA sends an *MA_TaskCommandMT* to command VI to execute the *MA_Task*. VI responds with *MA_TaskCommandStatus* VI responds with *MA_TaskCommandStatus* messages and additionally, since VI will also be executing an *MA_Task*, VI will publish a *TaskStatus* message. If VI rejects the command, VI indicates this by sending MA an *MA_TaskCommandStatusMT* with *CommandProcessingState* = *REJECTED*.

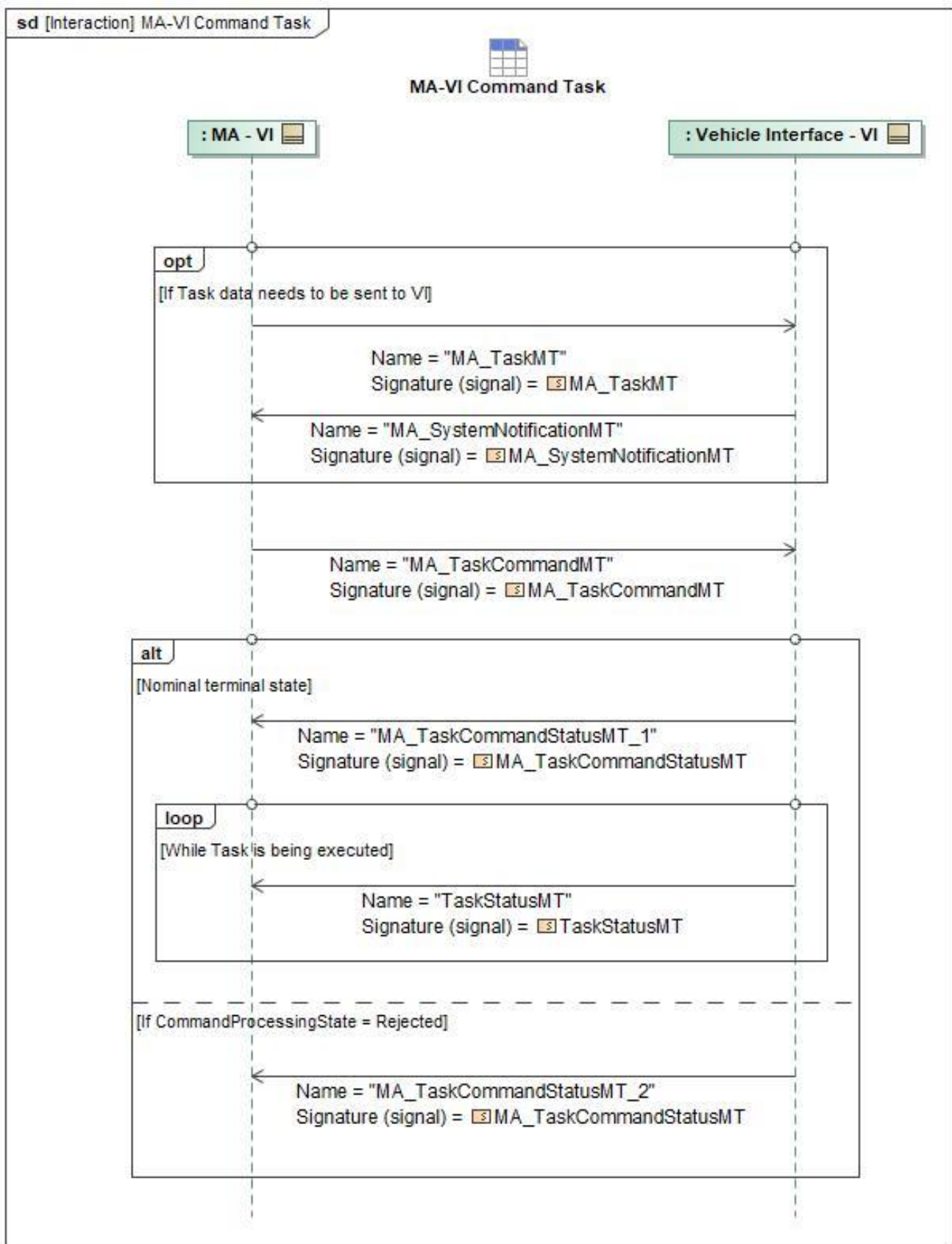


Figure 1-11: MA-VI Command Task

Table 1-10: MA-VI Command Task

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemNotificationMT: MA_SystemNotificationMT	AssociatedID: ID_Type AssociatedMessage: AssociatedMessageType NotificationDetail: CannotComplyType
MA_TaskCommandMT: MA_TaskCommandMT	None
MA_TaskCommandStatusMT_1: MA_TaskCommandStatusMT	None
MA_TaskCommandStatusMT_2: MA_TaskCommandStatusMT	CommandProcessingStateReason: CannotComplyType
MA_TaskMT: MA_TaskMT	None
TaskStatusMT: TaskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2.6 Modify Capabilities

This sequence is intended for when FA has a condition that requires modification of MA capability control but does not constitute a complete removal of secondary control for MA. FA, through the VI, will send a command to reduce MA's ACP control functionalities. When FA must reduce MA control, the *MA_FlightCapabilityStatusMT* message, containing the *CapabilityStatus.AvailabilityInfo.Availability* field will be populated with "TEMPORARILY_UNAVAILABLE" to provide the status that capabilities are unavailable for MA at the given time. FA should also provide reasoning for reducing MA's control using the *MA_FlightCapabilityStatusMT* message containing the *CapabilityStatus.AvailabilityInfo.Availability.AvailabilityReason.Reason* field. The *MA_FlightCapabilityMT* will also be sent to capture any capability changes outside of *MA_FlightCapabilityStatusMT*. It should be noted, a different interaction will be used in the event of a full unpairing scenario, where FA must remove *SECONDARY_CONTROLLER* from MA. In the condition that Capabilities have a state change to available (i.e. "TEMPORARILY_UNAVAILABLE" to "AVAILABLE"), the reasoning for this capability change that would be conveyed within *CapabilityStatus.AvailabilityInfo.Availability.AvailabilityReason.Reason* is considered optional. It's expected that capabilities upon entry into a negative state (*UNAVAILABLE*, *TEMPORARILY_UNAVAILABLE*, etc.) will have listed a reason and thus coming out of the state should not require a repeat of that reason.

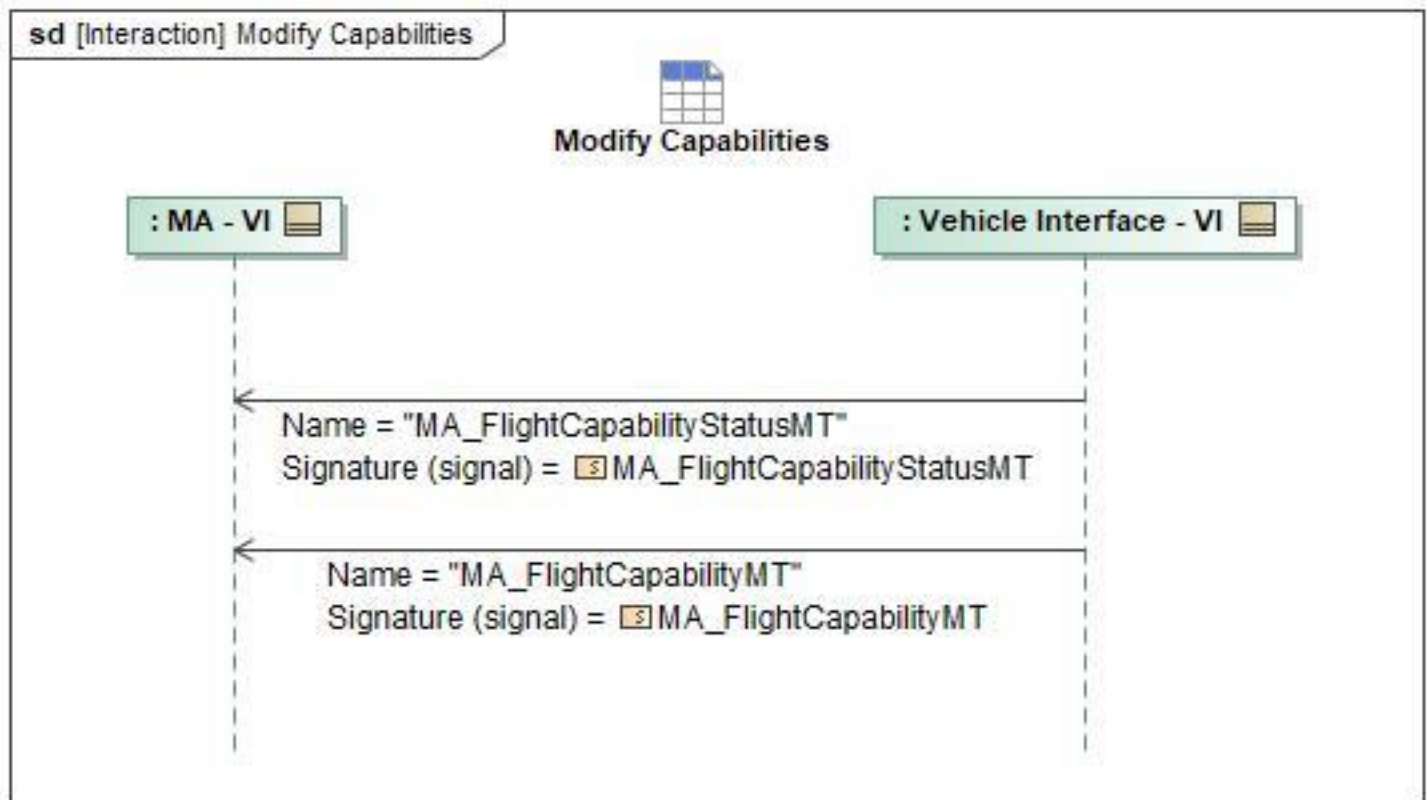


Figure 1-12: Modify Capabilities

Table 1-11: Modify Capabilities

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2.7 Receive Control Request

MA can attain capability control over one or more capabilities advertised by VI. Advertised capabilities can be discovered by subscribing to *MA_FlightCapabilityMT* messages in the exchange described by the "Control Mode Authorization" behavior and corresponding sequence diagram.

MA can initiate the request to attain capability control by sending *MA_ControlRequestMT* to VI. MA will specify *ControlAssignment/PermittedControlType* of "CAPABILITY_CONTROL". MA will specify that the sibling Controller be set with the *SystemID* and *ServiceID* corresponding to the particular MA service that would be requesting to send subsequent messages to VI. MA will specify the *Controller* with the *SystemID* corresponding to the VI and the *CapabilityID* matching the *desired* capability advertised by VI in a previous *MA_FlightCapabilityMT* message. The *RequestType* should be specified as "ACQUIRE".

VI can optionally provide a *MA_ControlRequestStatusMT* to MA indicating the initial request was received and is being processed by specifying the *ApprovalRequestProcessingState* as "PENDING".

If VI accepts the request and is ready to start listening to the MA service that sent the original request, VI can reply with a *MA_ControlRequestStatusMT* with the *ApprovalRequestProcessingState* set to "APPROVED". Before sending *MA_FlightCommandMT* or *MA_Task* messages, the requesting MA service should consume the next published *ControlStatusMT* published by VI to determine the granted *CapabilityIDs*. See the referenced "Publish Control Status" behavior for details.

If VI rejects the request and will subsequently not forward managed capability control to the requesting MA service, VI will reply with a *MA_ControlRequestStatusMT* message with the *ApprovalRequestProcessingState* set to "REJECTED". Furthermore, the *ApprovalRequestProcessingStateReason* will be set to an enumeration value appropriate for the situation. While MA has secondary control the "Publish Control Status" message exchange, in particular the *ControlStatusMDT ControlType/CapabilityControl/SecondaryController/ServiceID* indicates which MA service has secondary flight capability control. If Flight Autonomy (FA) via the Vehicle Interface (VI) needs to revoke Mission Autonomy (MA) secondary control it can publish *MA_ControlRequestStatusMT* (indicated by *MA_ControlRequestStatusMT_4*) and *MA_ControlAssignmentMT*. VI will inform secondary controllers, per capability, with their remaining capabilities, if any, with *MA_ControlAssignmentMT*. The *MA_ControlRequestStatusMT* message's *ApprovalRequestProcessingState* should indicate "CANCELED". The *ApprovalRequestProcessingStateReason* will be set to an enumeration value appropriate for the situation. This is a similar pattern to "Control by..." VI behaviors where VI can end the exchange when it determines MA secondary control must be revoked or modified. The *MA_ControlAssignmentMT* message will indicate *ControlType/CapabilityControl/PrimaryController/SystemID* and *ControlType/CapabilityControl/PrimaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* corresponding to an appropriate OMS service on the VI. It is an aperiodic version of the same information sent in the *ControlStatusMT* message sent in "Publish Control Status". For each *CapabilityID*, VI will indicate *ControlType/CapabilityControl/SecondaryController/SystemID* and *ControlType/CapabilityControl/SecondaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* for any MA service that still retains control. If no MA services have secondary controller status for a given capability, none will be reported. The *AcceptedInterface* will be populated with the appropriate command interfaces for the given capability.

Table 1-12: Receive Control Request

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ControlAssignmentMT: MA_ControlAssignmentMT	SecondaryController: SecondaryControllerType AcceptedInterface: SubsystemBIT_ControlInterfacesEnum CapabilityControl: ControlStatusCapabilityControlType PrimaryController: PrimaryControllerType
MA_ControlRequestMT: MA_ControlRequestMT	ServiceID: ServiceID_Type CapabilityID: CapabilityID_Type
MA_ControlRequestStatusMT_1: MA_ControlRequestStatusMT	None
MA_ControlRequestStatusMT_2: MA_ControlRequestStatusMT	None
MA_ControlRequestStatusMT_3: MA_ControlRequestStatusMT	ApprovalRequestProcessingStateReason: CannotComplyType
MA_ControlRequestStatusMT_4: MA_ControlRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2.8 Unpair Control Assignment

If FA via the VI needs to revoke MA secondary control it can publish *MA_ControlRequestStatusMT* and *MA_ControlAssignmentMT*. Using *MA_ControlAssignmentMT*, VI will inform secondary controllers of their remaining capabilities, if any. This message will be sent on a per capability basis for each secondary controller. The *MA_ControlRequestStatusMT* message's *ApprovalRequestProcessingState* should indicate "CANCELED". The *ApprovalRequestProcessingStateReason* will be set to an enumeration value appropriate for the situation. The *MA_ControlAssignmentMT* message will also indicate *ControlType/CapabilityControl/PrimaryController/SystemID* and *ControlType/CapabilityControl/PrimaryController/ServiceID* with a *SystemID* and *ServiceID* corresponding to an appropriate OMS service on the VI. For each *CapabilityID*, VI will indicate *ControlType/CapabilityControl/SecondaryController/SystemID* and *ControlType/CapabilityControl/SecondaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* for any MA service that still retains control. If no MA services have secondary controller status for a given capability, none will be reported. The *AcceptedInterface* will be populated with the appropriate command interfaces for the given capability.

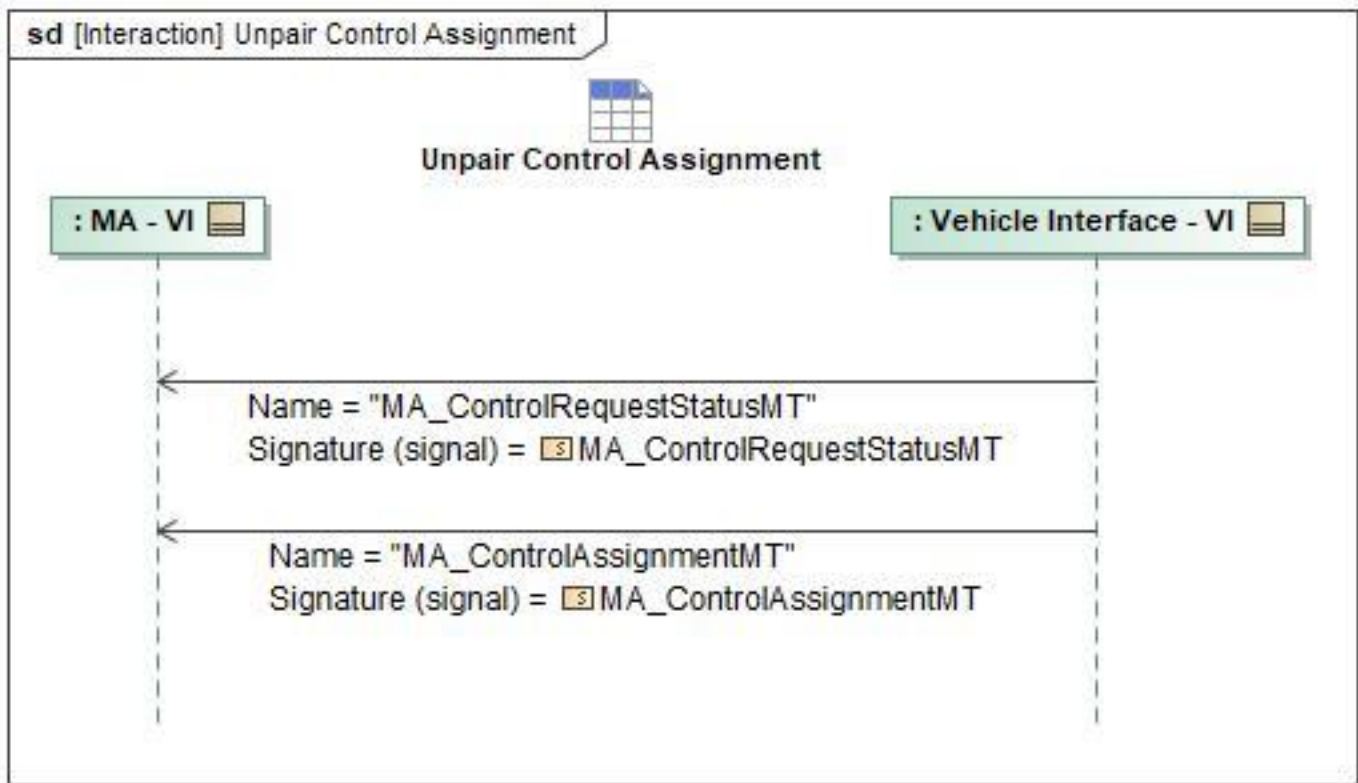


Figure 1-14: Unpair Control Assignment

Table 1-13: Unpair Control Assignment

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ControlAssignmentMT: MA_ControlAssignmentMT	SecondaryController: SecondaryControllerType AcceptedInterface: SubsystemBIT_ControlInterfacesEnum CapabilityControl: ControlStatusCapabilityControlType PrimaryController: PrimaryControllerType
MA_ControlRequestStatusMT: MA_ControlRequestStatusMT	ApprovalRequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.2.9 Update C2 Control Designations

C2 may command the aircraft control paradigm including revoking or authorizing MA to command the flight of the aircraft. C2 will revoke or authorize MA to command specific flight capabilities of the aircraft as outlined by the "Update MA-FA Control Designations" sequence in the C2 interface. Upon receiving the *MA_FlightCapabilityMT* from C2, MA will forward it to FA to communicate the authorized flight capabilities. The *MA_FlightCapabilityMT* will have the *CapabilityType.MA_FlightCapabilityEnum* field populated with the allowable control modes that MA can command. This enumeration list will be redacted to revoke specific flight capabilities or amended to authorize additional flight capabilities as desired by C2.

MA will also forward the C2 provided *MA_ControlRequestMT* with the *Controller* populated with the MA System ID and the *Controllee* populated with the FA system ID as MA will be controlling FA through flight commands. The *MA_ControlRequestMT* will also include the *GrantedControlType* for MA as *CAPABILITY_SECONDARY*, as the *CAPABILITY_PRIMARY* control type is reserved for FA as the safety critical system on the aircraft. The *MA_ControlRequestMT* will also include the *PermittedControlType*, set to *CAPABILITY_CONTROL* to correlate with the Flight Capabilities that C2 is granting/revoking. Lastly, the *CapabilityID* will be set as the *CapabilityID* of the *MA_FlightCapabilityMT* that proceeded the *MA_ControlRequestMT*.

FA will respond through the VI with the *MA_ControlRequestStatusMT* to acknowledge the commanded control paradigm. Then FA will broadcast its update flight capabilities using the *MA_FlightCapabilityMT* that will have the *CapabilityType.MA_FlightCapabilityEnum* populated to align with the C2 commanded flight capabilities. FA will then send the *MA_FlightCapabilityStatusMT* to communicate the status of the broadcasted flight capabilities.

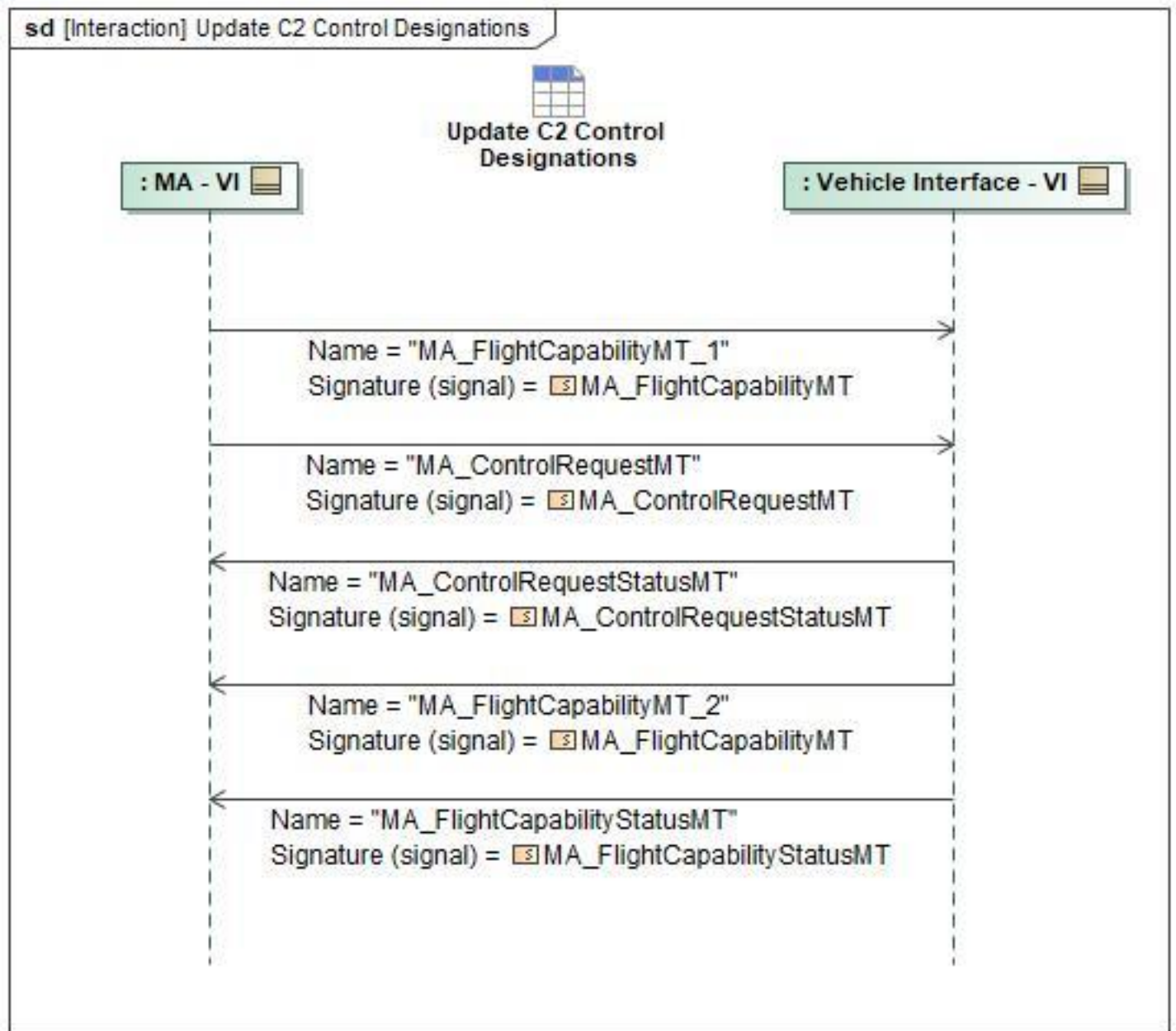


Figure 1-15: Update C2 Control Designations

Table 1-14: Update C2 Control Designations

Message Name (Name:Type)	Required Fields (Name:Type)
MA_ControlRequestMT: MA_ControlRequestMT	CapabilityID: CapabilityID_Type
MA_ControlRequestStatusMT: MA_ControlRequestStatusMT	None
MA_FlightCapabilityMT_1: MA_FlightCapabilityMT	None
MA_FlightCapabilityMT_2: MA_FlightCapabilityMT	None

MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None
---	------

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.3 COP Package

1.2.3.1 VI Updates to COP

The VI Updates to COP sequence diagram specifies the interaction of VI sending updates of the vehicle state to MA.

At a predefined periodic rate, the following messages are sent: *MA_PositionReportDetailedMT*, *NavigationReportMT*, *ComponentStatusMT*, *MA_FlightCapabilityMT*, *MA_FlightCapabilityStatusMT*, and *MA_FlightActivityMT*.

The *MA_PositionReportDetailedMT* must have *Source*, *Position* with *Timestamp*, *Velocity*, and *Orientation* set. The *NavigationReportMT* must have the fuel mass set in *Fuel*, the duration set in *Duration*, the fuel percent set in *Percent*, and the *ContingencyLevel* set to the contingency level. The *ComponentStatusMT* must have the *ComponentID* set to the *System ID* and the *ComponentState* set to the *Vehicle Autonomy's state*. The *MA_FlightCapabilityMT* must contain the applicable performance limits. The *MA_FlightCapabilityStatusMT* must contain the *CapabilityID* of the flight capability and the *Availability* set the availability of Vehicle Autonomy to accept flight commands. The *MA_FlightActivityMT* must contain the last flight command (if any).

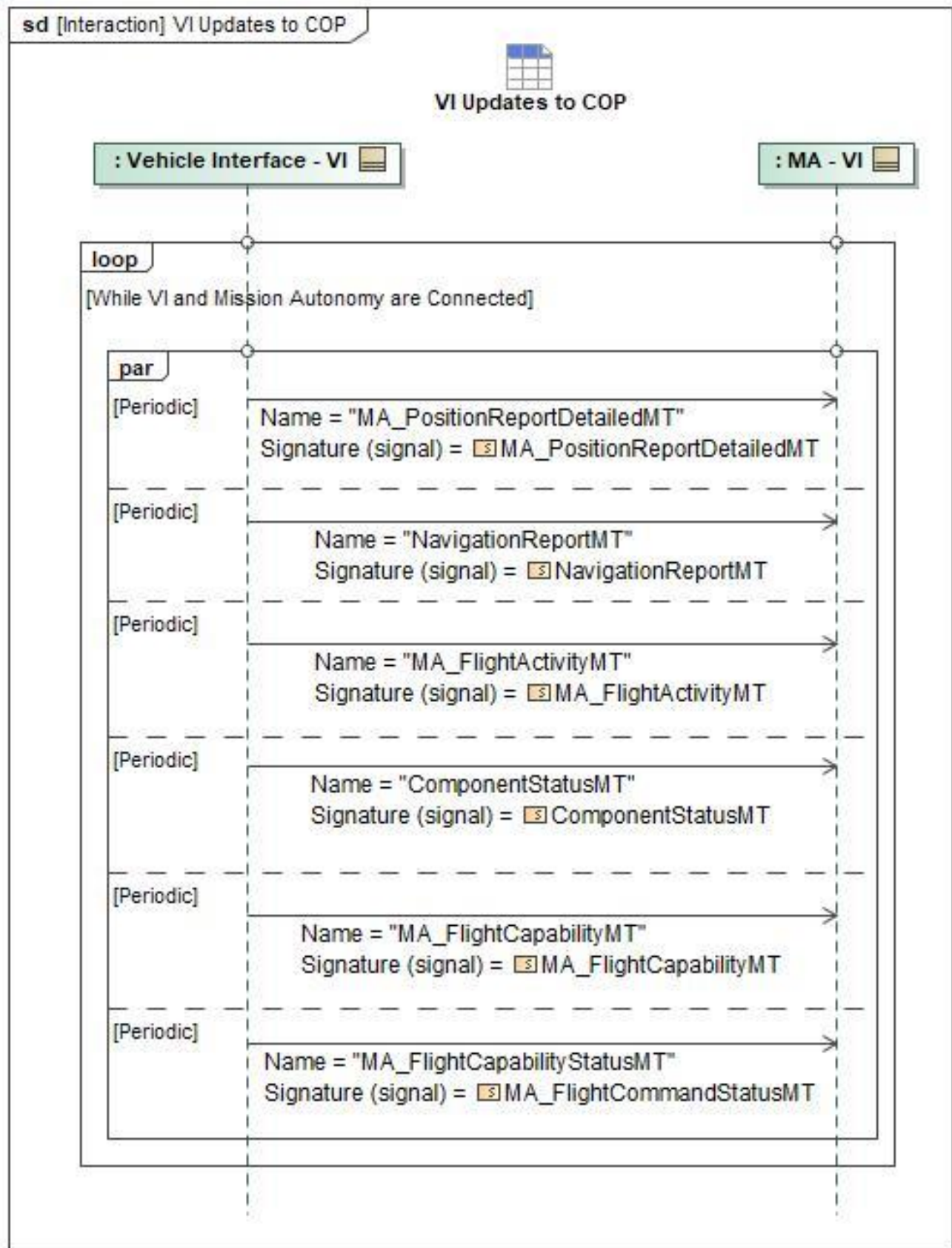


Figure 1-16: VI Updates to COP

Table 1-15: VI Updates to COP

Message Name (Name:Type)	Required Fields (Name:Type)
ComponentStatusMT: ComponentStatusMT	None
MA_FlightActivityMT: MA_FlightActivityMT	None
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCommandStatusMT	None
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	Kinematics: DetailedKinematicsType
NavigationReportMT: NavigationReportMT	Fuel: MassType Percent: PercentType Duration: DurationType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.4 Data Validation Package

1.2.4.1 Checksum Validation

MA can check that the data stored on MA and FA is consistent using a checksum approach. MA can compare the *SHA_2_Hash* in the *FileMetadataMT* for the associated object (airfield, route plan, etc.) to identify if there are discrepancies between the files. Because MA and FA may store files in different native formats, this checksum should compare the *SHA_2_Hash* on the UCI version of the file (after conversion from native format for MA, before conversion to native file format for FA).

If MA is missing the data records to be able to validate data stored on MA and FA via checksum, it can send a *QueryDataRequestMT* to FA with a query including the associated ID of the applicable data record. Once requested, FA will send the *FileLocationMT* and *FileMetadataMT* containing the details of the relevant data record, including the *SHA_2_Hash* field in *FileMetadataMT*. If FA is unable to complete the query, *QueryDataRequestStatusMT* will be returned with *RequestProcessingState* set to *FAILED*, and including *RequestProcessingStateReason* with the reason for the failure.

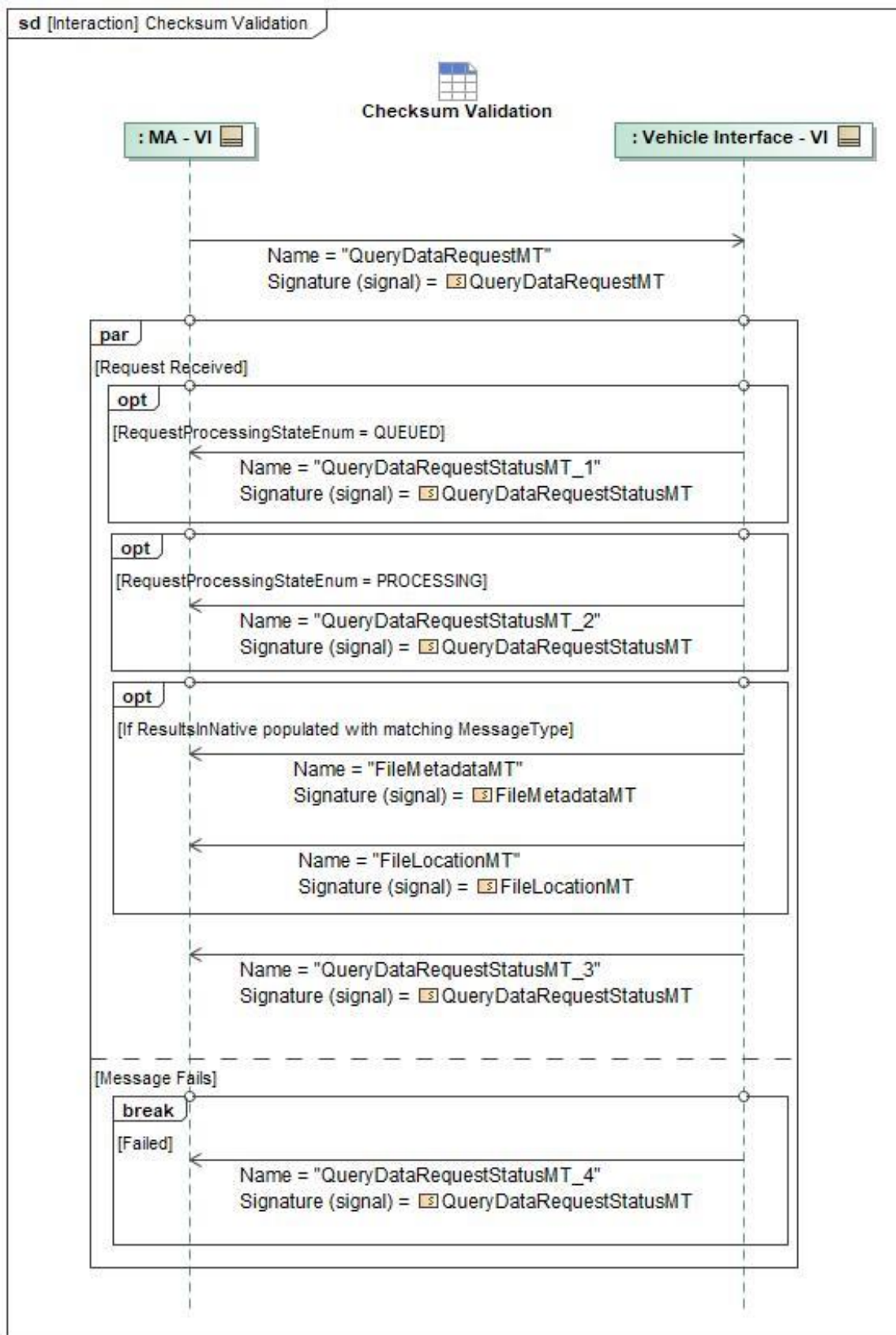


Figure 1-17: Checksum Validation

Table 1-16: Checksum Validation

Message Name (Name:Type)	Required Fields (Name:Type)
FileLocationMT: FileLocationMT	None
FileMetadataMT: FileMetadataMT	SHA_2_Hash: SHA_2_256_HashType
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.4.2 Query for Missing Data

MA can query FA for a list of all object IDs of a given data type (ex: Route Plans and Airfields for route plan data validation) stored on FA by sending *QueryDataRequestMT* to FA (via VI), with *QueryIdentifiersOnly* to indicate only IDs should be returned, and *QueryMessage.Query* specifying the types of data for which MA is querying. FA will respond with *QueryDataRequestStatusMT* with the list of requested IDs in *Results*.

Once the list of IDs is returned, MA evaluates if there are any it was not aware of or any that FA is missing. For any data that MA is not aware of (e.g. due to lost awareness after a reboot), it will then call 'Query Airfield Update' and/or 'Query Route Plan' behaviors to request the missing data, along with the associated metadata required for checksum validation, from FA.

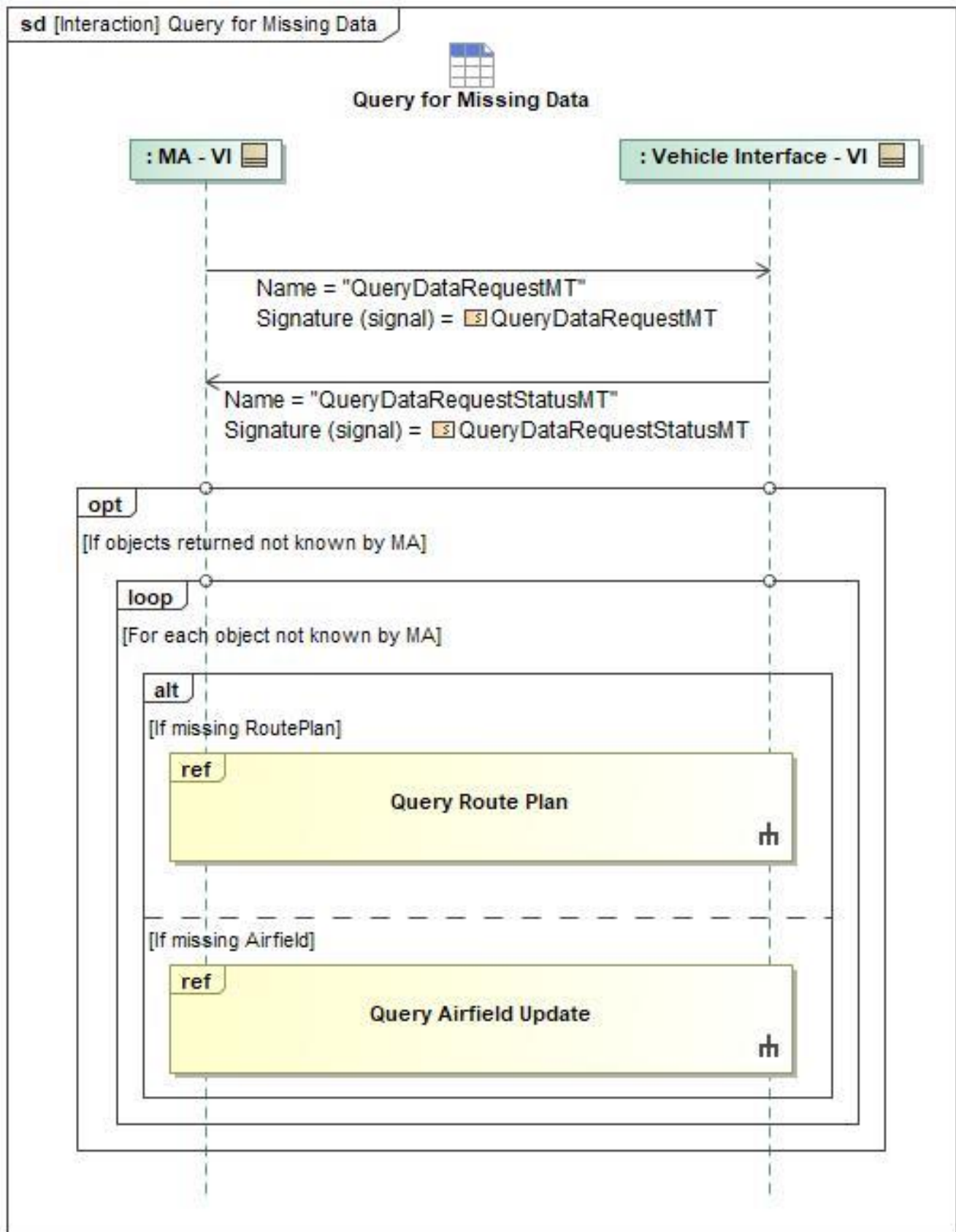


Figure 1-18: Query for Missing Data

Table 1-17: Query for Missing Data

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	Query: QueryPET
QueryDataRequestStatusMT: QueryDataRequestStatusMT	Result: QueryResultType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.4.3 Route Plan Data Validation

When MA needs to validate that it has the same route plan-related data as FA, there are three steps that can be used as needed for this data validation. Route plan-related data stored on FA and thus covered here include *MA_RoutePlanMTs* and Airfields (contained in *AirfieldReportMTs*).

First, MA can use the 'Query for Missing Data' behavior to query for a list of all object IDs of a given data type (RoutePlans or Airfields) stored on FA. This step must be performed on startup and if MA ever reboots. It can subsequently be re-run if there is ever a concern that MA has lost awareness of data stored on FA, or as needed. One example of wanting to double check for missing data is if MA determines this platform needs to land safely as soon as possible, and it can query to make sure there are no airfields stored on FA that it does not have awareness of.

Once the list of IDs is returned, MA checks the list for inconsistencies. For any data that MA is not aware of, it will then query FA for the missing data, as well as the file location and metadata (including Hash) of the missing object(s).

Second, MA can check that the data stored on MA and FA is consistent using a checksum approach. MA can compare the *SHA_2_Hash* in the *FileMetadataMT* for the associated object (airfield, route plan, etc.) to identify if there are discrepancies between the files. Because MA and FA may store files in different native formats, this checksum should compare the *SHA_2_Hash* on the UCI version of the file before conversion to/from native file format. If MA is missing the file metadata of the data it wants to validate, or has reason to request a fresh hash, it can query by ID for the *FileMetadataMT* and *FileLocationMT* of the file in question using the 'Checksum Validation' behavior.

Finally, if a checksum is performed and finds that there is a discrepancy between data saved on MA and FA, it will be corrected by having the 'authority system' send the correct data to the other system. FA is the authority for airfields and for route plans associated with takeoff, departure, approach, and landing, while MA is the authority for other, non-safety critical route plans. If there is inconsistency with data for which MA is the authority, it will send the correct data to FA. As MA is the one performing the checksum, if there is inconsistency with data for which FA is the authority, MA will query for the correct versions from FA.

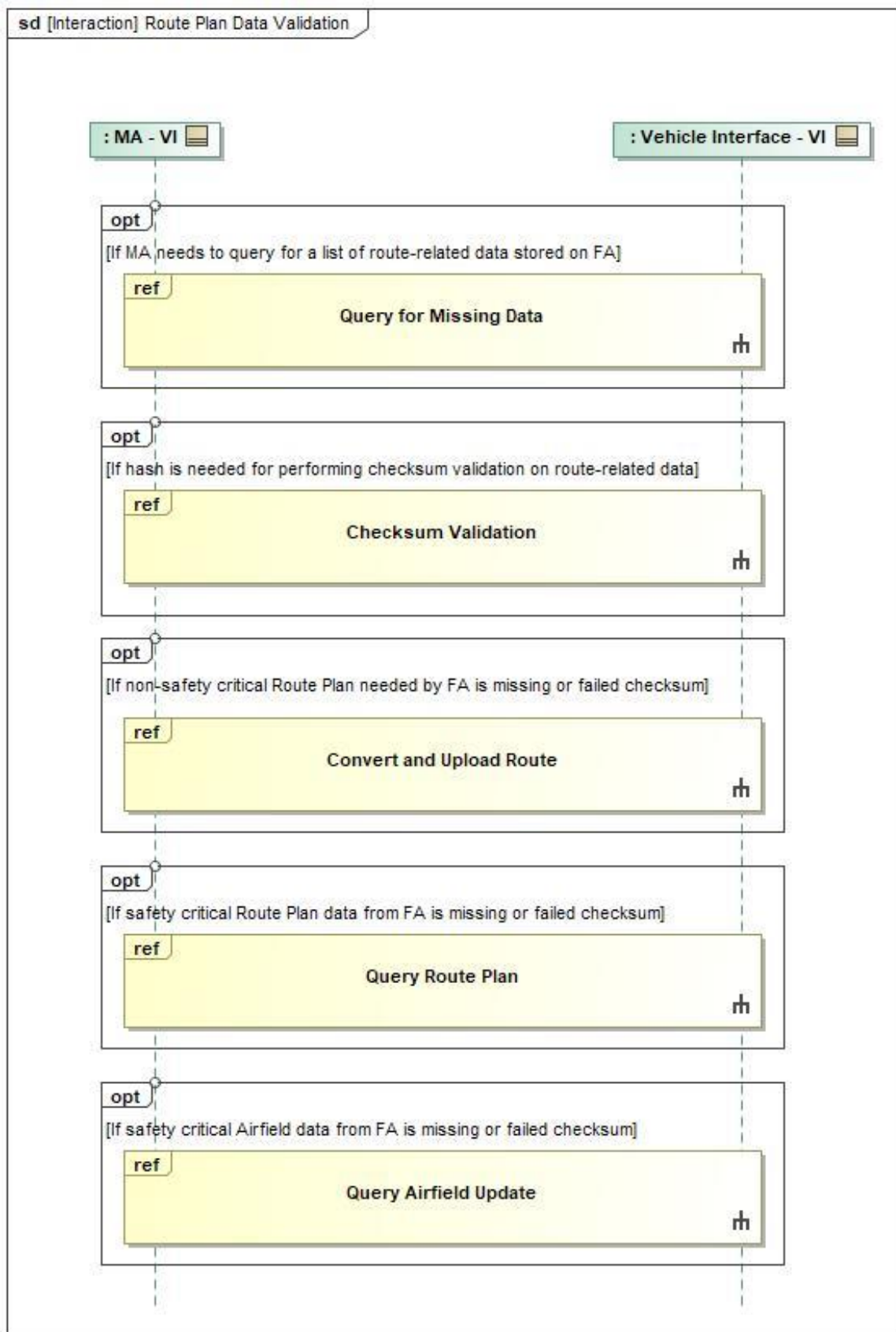


Figure 1-19: Route Plan Data Validation

Table 1-18: Route Plan Data Validation

Message Name (Name:Type)	Required Fields (Name:Type)
--------------------------	-----------------------------

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.5 Route Plan Behaviors Package

1.2.5.1 Activate Route

Activate Route is used to signal that FA should begin to follow (i.e. execute) a stored *MA_RoutePlan* that has been uploaded (if not already stored on FA pre-mission) and prepared for activation by previous interactions.

The precondition for this interaction is that the *MA_RoutePlan* is already in the *READY_FOR_ACTIVATION* state. The activation state of the route plan is determined by querying data storage internal to MA for the latest *MissionPlanActivationStatus* that corresponds to the *MA_RoutePlan* in question.

Mission Autonomy (MA) can coordinate activation of routes with the VI isolator. This can be used for takeoff, landing, or any other phase of flight. In order to activate takeoff, MA will send *MA_MissionPlanActivationCommand* with the *BySubPlan* field populated, referring to the takeoff *MA_RouteActivityPlan* stored on FA. Before routes can be activated, the *MA_RoutePlanMT* must be taken through the plan activation state machine provided by the semantics of the OMS/UCI schema. The diagrams: "Convert and Upload Route" and "Prepare for Route Activation" show the interactions required to achieve this between MA and FA.

MA coordinates the activation of the route plan by sending an *MA_MissionPlanActivationCommandMT* with *CommandType* set to *ACTIVATE*. Once VI activates the route and begins guiding, it replies with *MA_MissionPlanActivationCommandStatusMT* with *ActivationCommandByState/PlanActivationCommandState* set to *ACTIVATE* and *ActivationStatus/CommandStatus* set to *COMPLETED*.

Each *MA_MissionPlanActivationCommandMT* should use choice type *Command/ActivationDetails/BySubPlan*. Furthermore, the *SubPlan* should be a *RoutePlanActivationType*. *MA_MissionPlanActivationCommandStatusMT* should specify *ActivationCommandByState/Plans* with *RoutePlanID*.

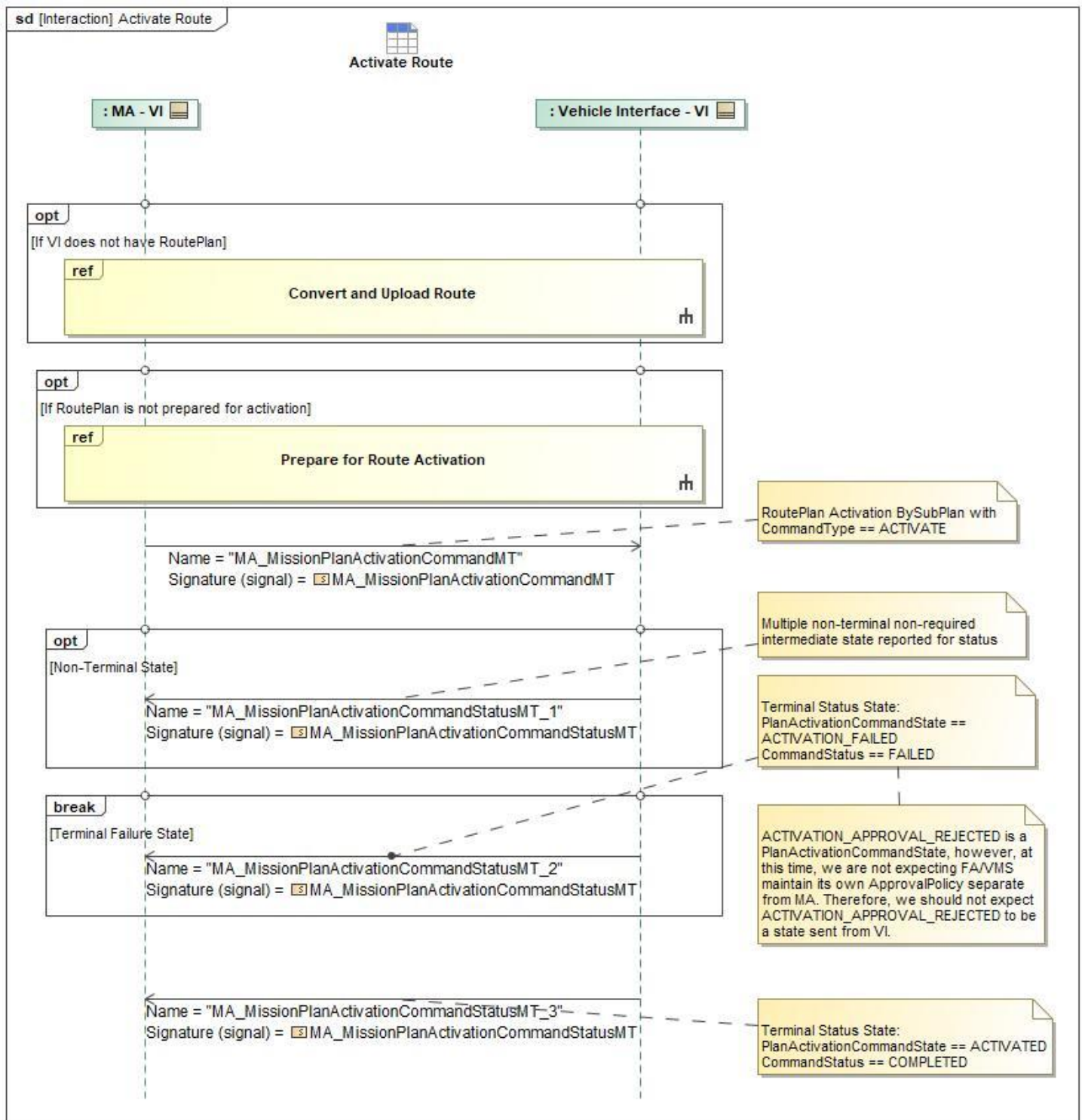


Figure 1-20: Activate Route

Table 1-19: Activate Route

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	RoutePlan: RoutePlanActivationType

MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type ActivationCommandByState: MA_PlanActivationStatusType
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type ActivationCommandByState: MA_PlanActivationStatusType
MA_MissionPlanActivationCommandStatusMT_3: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type ActivationCommandByState: MA_PlanActivationStatusType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.5.2 Convert and Upload Route

Mission Autonomy (MA) can coordinate conversion, upload, and preparation for route activation with the VI isolator. Note, MA may not send new *MA_RoutePlanMTs* to FA related to takeoff, departure, approach, or landing, as those are considered safety critical. For those phases of flight, MA must activate one of the pre-planned, safety-validated takeoff, departure, approach, and/or landing Route Plans stored pre-flight on FA.

The *PREPARE_FOR_UPLOAD* exchange serves two purposes: data conversion and upload preparation. *MA_RoutePlanMTs* typically require translation to native vehicle management system (VMS) formats. Conversion can occur at this time if required. Upload is critical to ensuring routes are properly loaded to FA/VMS via VI. At any given time, many *MA_RoutePlans* may be transmitted on MA message busses that are not of concern to FA. Since the VMS isolator may be running in a real time and/or compute constrained environment it is important to signal that routes are being prepared for it to consume. In this way, the VI isolator does not need to consume each and every *MA_RoutePlanMT* published. VI isolator only needs to receive *MA_RoutePlanMTs* during the upload state and only for the UUIDs associated with the corresponding *MA_MissionPlanActivationCommandMT*.

PREPARE_FOR_UPLOAD is accomplished by sending an *MA_MissionPlanActivationCommandMT* with *Command/ActivationDetails/BySubPlan/RoutePlan*. The corresponding *CommandType* is set to *PREPARE_FOR_UPLOAD*. In previous UCI major versions *PREPARE_FOR_UPLOAD* was *CONVERT*. When conversion is complete VI will reply with a *MA_MissionPlanActivationCommandStatusMT* with *ActivationCommandByState/PlanActivationCommandState* *PREPARE_FOR_UPLOAD* and *ActivationStatus/CommandStatus* set to *COMPLETED*.

In response to FA receiving *MA_RoutePlanMT*, a *MA_SystemNotificationMT* is sent back to MA with the *SubsystemID* and *ServiceID* set to the corresponding message receiver. This could be the VI isolator or wrapper. The *AssociatedID* is set to the *RoutePlanID* corresponding to the original message. *NotificationState* should be set to *CONFIRMED* if the message was received properly. When FA returns a queried *MA_RoutePlanMT*, FA also sends *FileLocationMT* and *FileMetadataMT* (including the *SHA_2_Hash* field) to MA, both containing details of FA's stored UCI version (as opposed to the version in FA's native file format) of the *AirfieldReportMT*. These will enable the performance of MA-FA route plan data validation behaviors. A single *MA_MissionPlanActivationCommandMT* can be used to coordinate multiple route or route/airfield updates. Loop over each artifact as required to pass all data to the VI.

Once the VI isolator is prepared for upload, *MA_RoutePlanMTs* are published by MA. MA signals VI that the routes have been published by sending a *MA_MissionPlanActivationCommandMT* with *CommandType* set to *UPLOAD*. The intent is for the converted *MA_RoutePlanMT* to be saved within the VMS's memory for future reference. Once this is complete VI replies to MA with *MA_MissionPlanActivationCommandStatusMT* with *ActivationCommandByState/PlanActivationCommandState* set to *UPLOAD* and *ActivationStatus/CommandStatus* set to *COMPLETED*. This gives the VI isolator the opportunity to stop listening.

Each *MA_MissionPlanActivationCommandMT* uses choice type *Command/ActivationDetails/BySubPlan*. Furthermore, the *SubPlan* should be a *RoutePlanActivationType*. *MA_MissionPlanActivationCommandStatusMT* should specify *ActivationCommandByState/Plans* with *RoutePlanID*.

If the VI is not able to upload or process data as commanded, the VI can publish *MA_MissionPlanActivationCommandStatusMT* with *CommandStatus* set to *FAILED*.

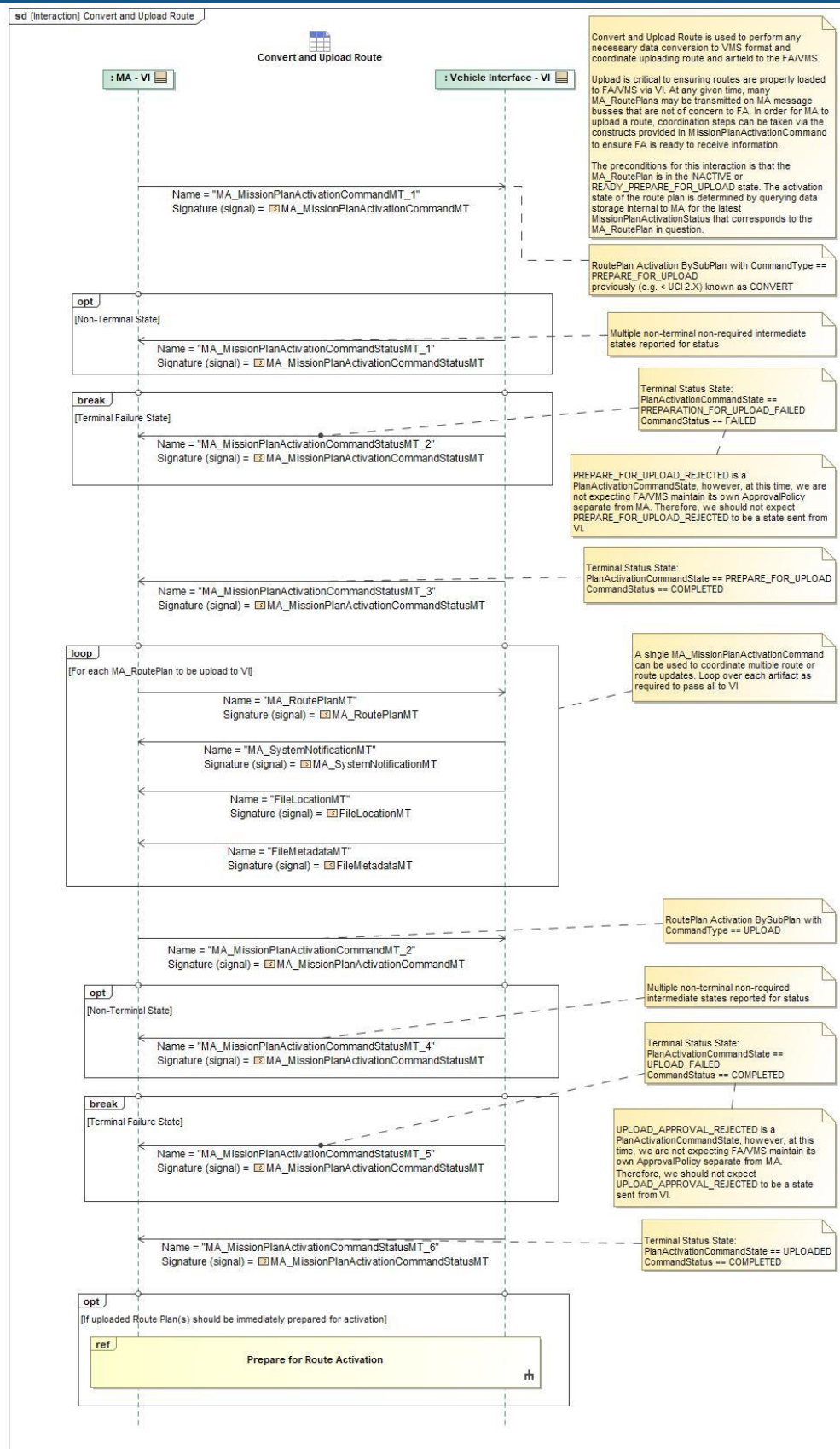


Figure 1-21: Convert and Upload Route

Table 1-20: Convert and Upload Route

Message Name (Name:Type)	Required Fields (Name:Type)
FileLocationMT: FileLocationMT	None
FileMetadataMT: FileMetadataMT	SHA_2_Hash: SHA_2_256_HashType
MA_MissionPlanActivationCommandMT_1: MA_MissionPlanActivationCommandMT	RoutePlan: RoutePlanActivationType
MA_MissionPlanActivationCommandMT_2: MA_MissionPlanActivationCommandMT	RoutePlan: RoutePlanActivationType
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_3: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_4: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_5: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_6: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_RoutePlanMT: MA_RoutePlanMT	None
MA_SystemNotificationMT: MA_SystemNotificationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.5.3 Prepare for Route Activation

Prepare for Route Activation is used to ensure that an *MA_RoutePlan* saved within FA/VMS is ready to activate for execution without delay upon a subsequent Activate Route exchange. This includes any final checks, preparations, memory allocations, etc.

A precondition for this interaction is that the *MA_RoutePlan* is already in the *UPLOAD* state. The activation state of the plan is determined by querying data storage internal to MA for the latest *MissionPlanActivationStatusMT* that corresponds to the *MA_RoutePlan* in question.

If VI fails to prepare the *MA_RoutePlan* for activation, for any reason, it is possible for VI to report a *PlanActivationCommandState* of *PREPARATION_FOR_ACTIVATION_FAILED* with corresponding *CommandStatus* set to *FAILED*.

If VI succeeds in preparing the *MA_RoutePlan* for subsequent activation this sequence can terminate with a *PlanActivationCommandState* of *READY_FOR_ACTIVATION* and a corresponding

CommandStatus set to **COMPLETED**.

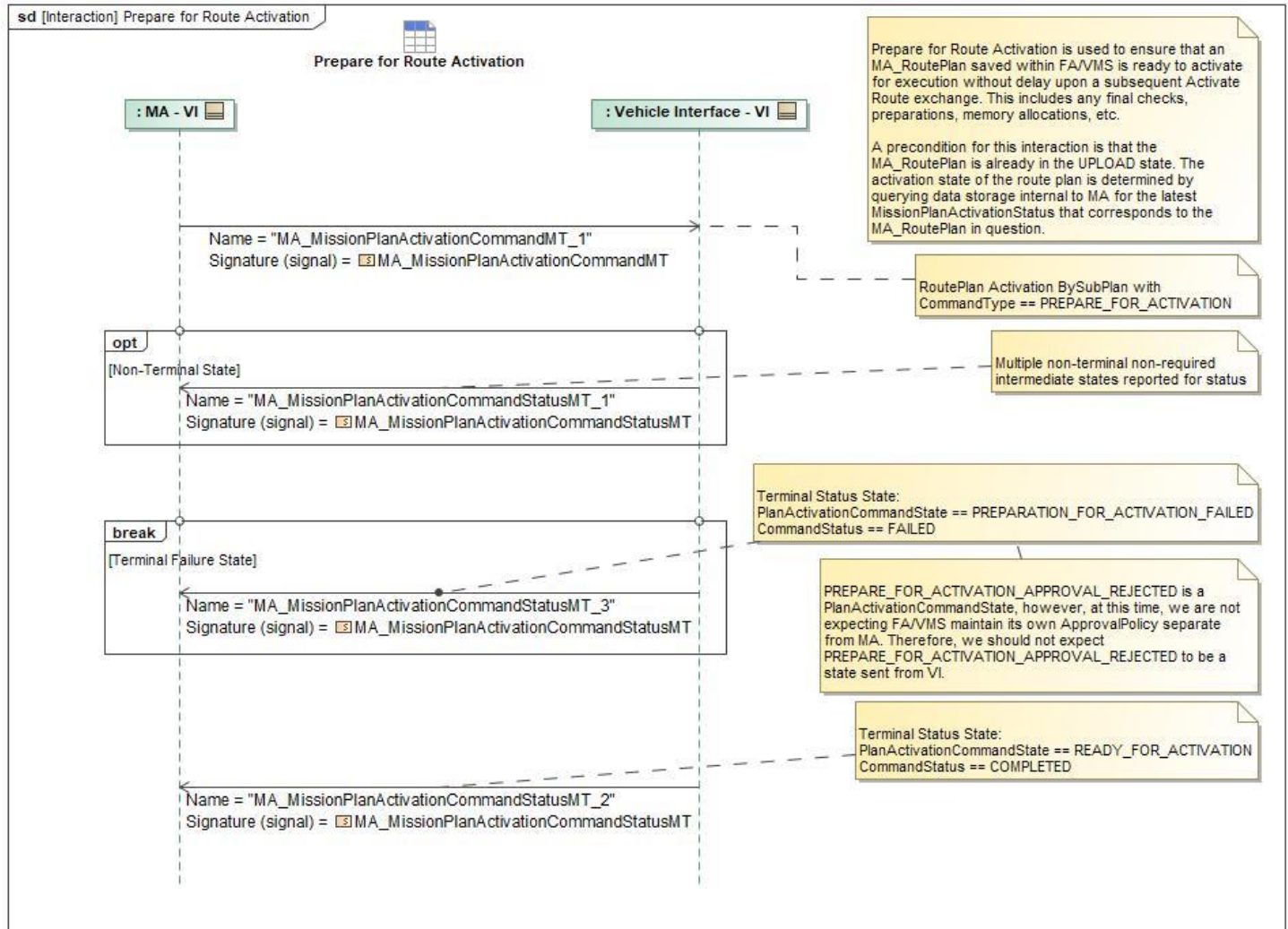


Figure 1-22: Prepare for Route Activation

Table 1-21: Prepare for Route Activation

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT_1: MA_MissionPlanActivationCommandMT	RoutePlan: RoutePlanActivationType
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type
MA_MissionPlanActivationCommandStatusMT_3: MA_MissionPlanActivationCommandStatusMT	RoutePlanID: RoutePlanID_Type

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

1.2.5.4 Receive Deactivate Route

This interaction will forward the command to deactivate a route from MA to VI and return the status.

The precondition for this interaction is:

MA_MissionPlanMT and *MA_RoutePlanMT* is loaded and either ready for activation or that the *MA_RoutePlanMT* is already in the *ACTIVATED* state.

The activation state of the route plan is determined by querying data storage internal to MA for the latest *MissionPlanActivationStatusMT* that corresponds to the *MA_RoutePlanMT* in question.

This interaction will use *MA_MissionPlanActivationCommandMT* to indicate the *MA_RoutePlanMT* to *DEACTIVATE*. It will set the *BySubPlan:RoutePlanID* to the UUID of the route to deactivate.

If the route plan was not yet *EXECUTING* will get back a *MA_MissionPlanActivationCommandStatusMT* with an *ActivationStatus* of *COMPLETED*.

Else the route plan is *EXECUTING* will get back a *MA_MissionPlanActivationCommandStatusMT* with an *ActivationStatus* of *FAILED*.

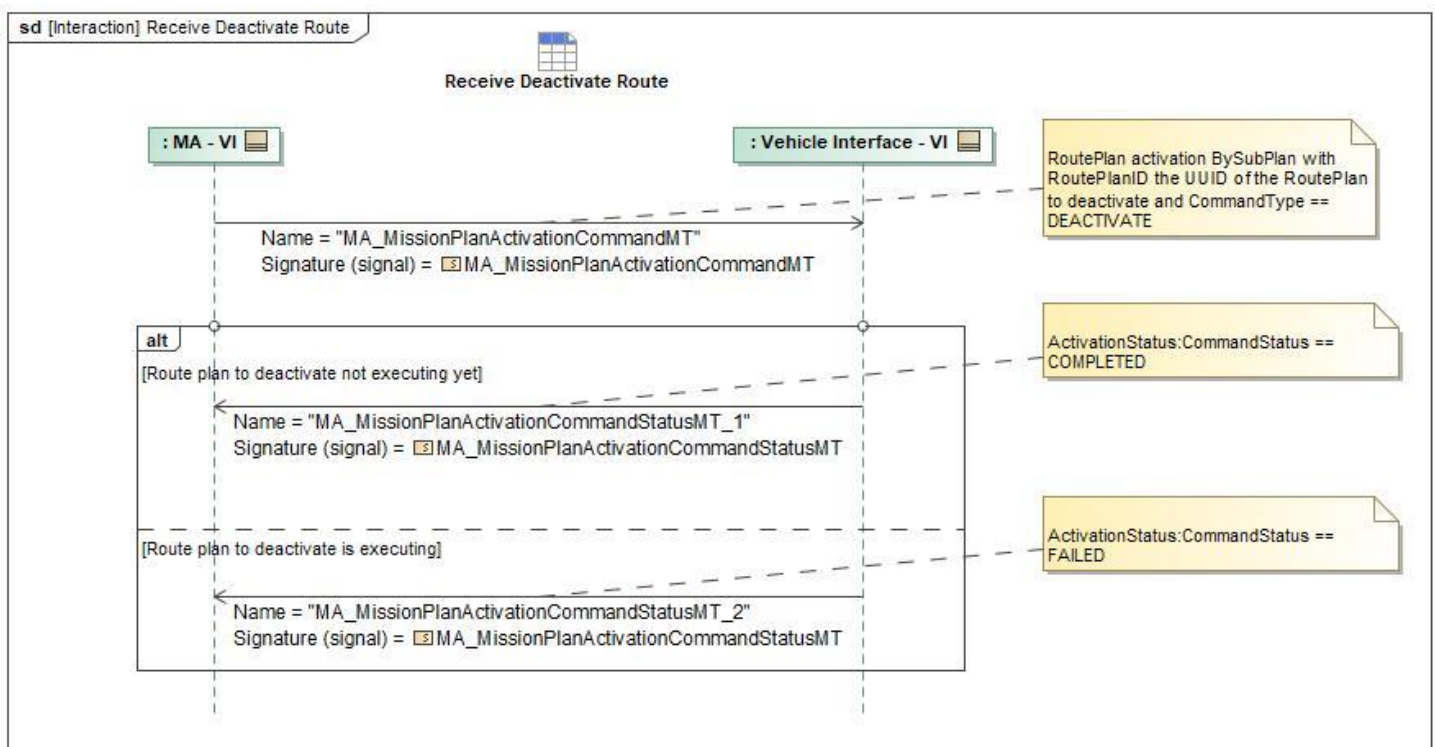


Figure 1-23: Receive Deactivate Route

Table 1-22: Receive Deactivate Route

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationCommandMT: MA_MissionPlanActivationCommandMT	CommandType: PlanActivationCommandEnum BySubPlan: MissionPlanSubplanActivationType

	DEACTIVATE:
MA_MissionPlanActivationCommandStatusMT_1: MA_MissionPlanActivationCommandStatusMT	CommandStatus: ProcessingStatusEnum COMPLETED:
MA_MissionPlanActivationCommandStatusMT_2: MA_MissionPlanActivationCommandStatusMT	FAILED: CommandStatus: ProcessingStatusEnum

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.5.5 Validate Route Plan

This sequence allows MA to have VI validate a RoutePlan. First the *RoutePlanValidationCommandMT* is sent from MA to VI. VI acknowledges the command by replying with *RoutePlanValidationCommandStatusMT*, and then follows up with the validation results in *RoutePlanValidationMT*.

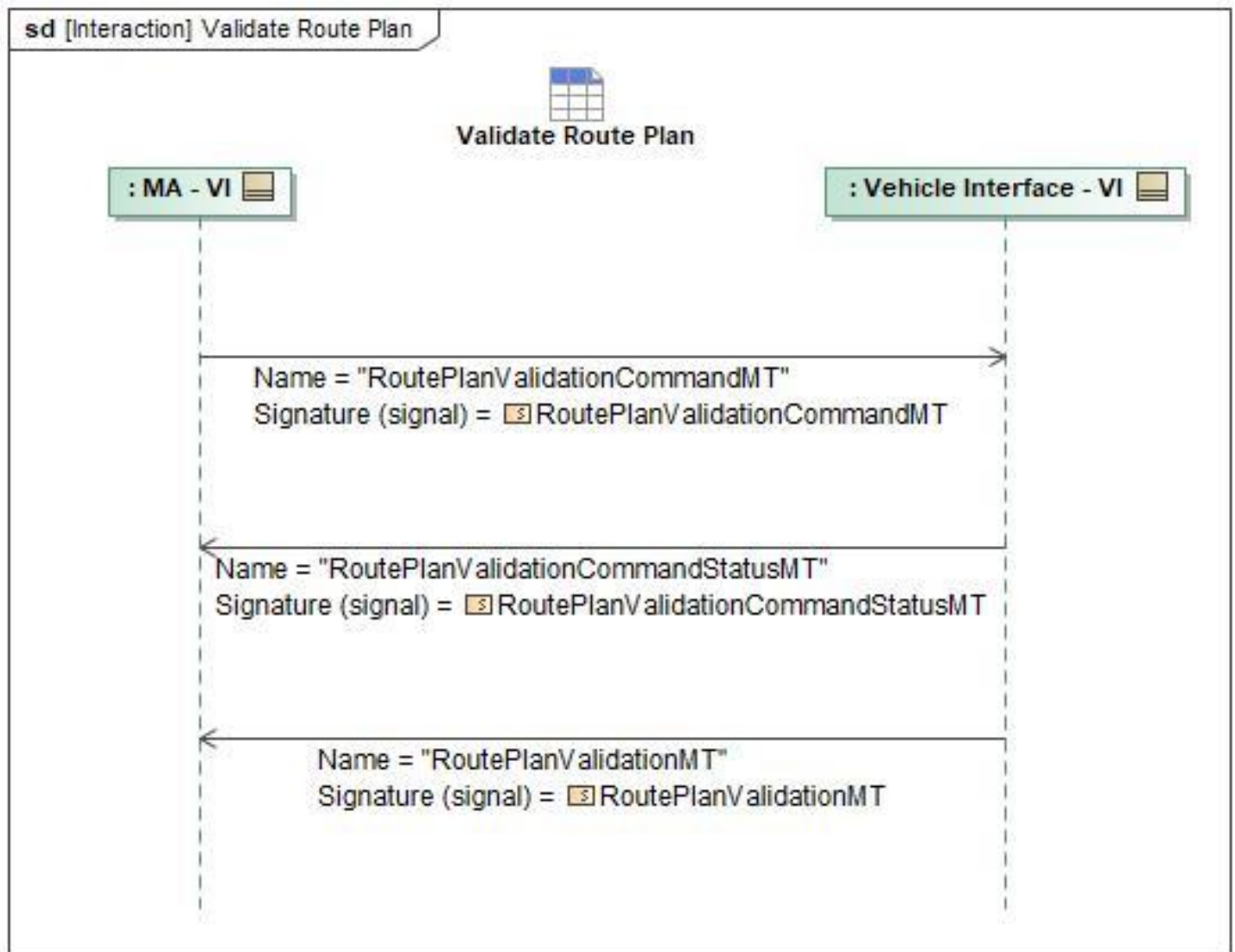


Figure 1-24: Validate Route Plan

Table 1-23: Validate Route Plan

Message Name (Name:Type)	Required Fields (Name:Type)
RoutePlanValidationCommandMT: RoutePlanValidationCommandMT	WeatherAreaData: WeatherAreaDataType
RoutePlanValidationCommandStatusMT: RoutePlanValidationCommandStatusMT	None
RoutePlanValidationMT: RoutePlanValidationMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.5.6 VI Deactivate Route

VI Deactivate Route Activation is used to signal that FA determined it should abort a route either

before or after it starts executing based on previous interactions (Activate Route)

The precondition for this interaction is:

EITHER *MissionPlan* and *RoutePlan* are loaded and ready for activation

OR that the *MA_RoutePlan* is already in the *ACTIVATED* state. The activation state of the route plan is determined by querying data storage internal to MA for the latest *MissionPlanActivationStatus* that corresponds to the *MA_RoutePlan* in question.

At some point FA/VI will abort a route. This can be either before or after the *RoutePlan* starts executing.

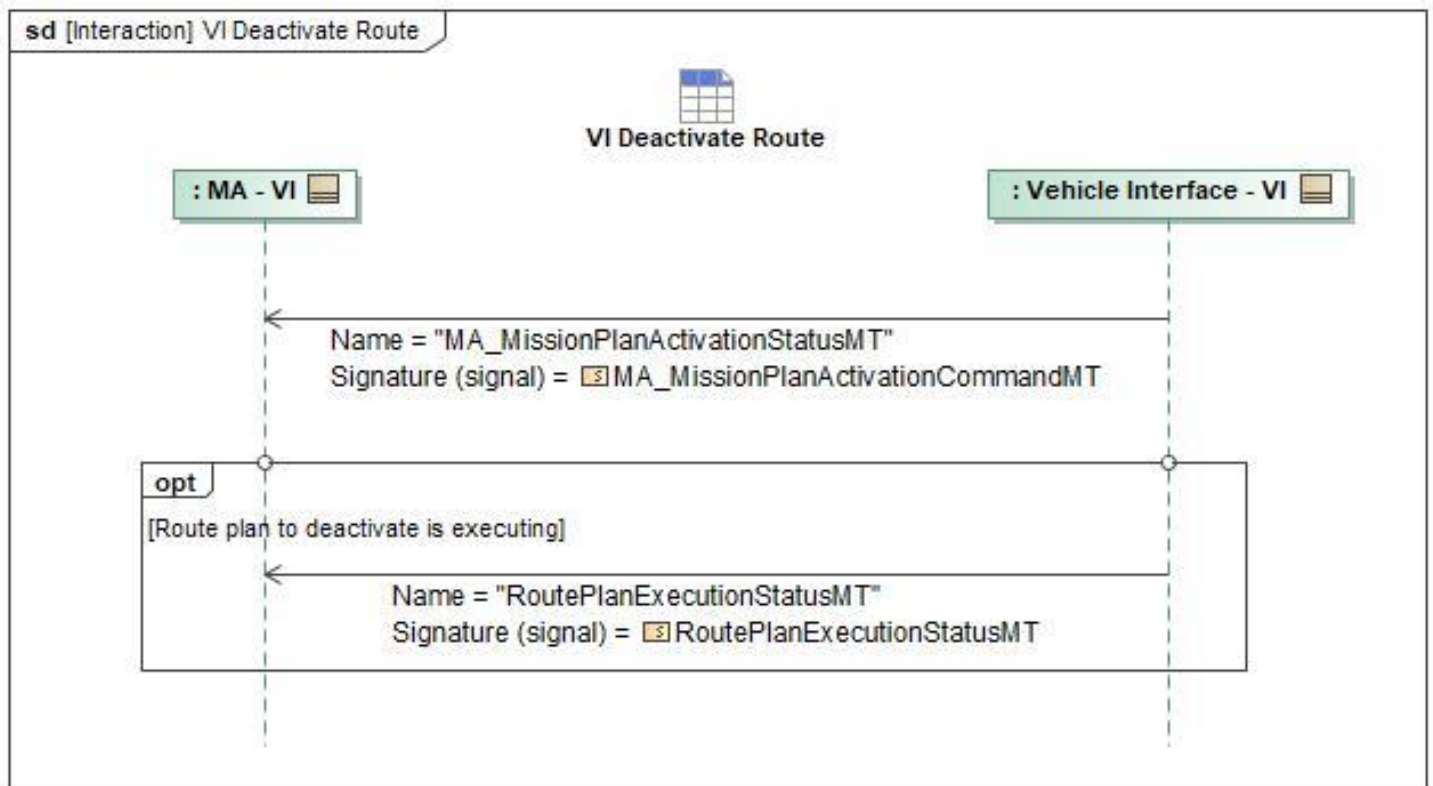


Figure 1-25: VI Deactivate Route

Table 1-24: VI Deactivate Route

Message Name (Name:Type)	Required Fields (Name:Type)
MA_MissionPlanActivationStatusMT: MA_MissionPlanActivationCommandMT	RoutePlanID: RoutePlanID_Type DEACTIVATED: SubPlanActivationState: PlanActivationStateType
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	CANCELED: PlanExecutionStatus: RoutePlanExecutionStateType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set

Tables section)

1.2.6 Status Package

1.2.6.1 Exchange Heartbeat - Subsystem Status Reports

The OMS UCI construct allows MA to broadcast MA health by using periodic UCI messages, all of which use a common *timestamp* in the message header. It is expected that the vehicle will employ an OMS Isolator to interact with the OMS Mission Package and MA. At a minimum, the OMS Isolator will publish *ServiceStatus* (for the Isolator software itself) and *FlightCapabilityStatus* to expose availability status of its flight capabilities. MA will subscribe to all of these UCI messages to confirm VI connectivity and health/status. MA will also publish *ServiceStatus* to confirm its presence and connectivity to the VI.

The *ServiceStatus* rate should be set in a configuration file for the Vehicle Interface with a default value of 1 second. While it is expected that all messages published and subscribed to by the OMS Isolator can and will be used to monitor overall health of the VI, the minimum message set for heartbeat/status exchange are *ServiceStatus* and *FlightCapabilityStatus* from VI to MA, and *ServiceStatus* from MA to VI. The component heartbeat message from Flight Autonomy to Mission Autonomy is achieved with the *SubsystemStatus* UCI message. The heartbeat message from the OMS Isolator is achieved with the *ServiceStatus* UCI message. The *ServiceStatus* is a periodic message with a configured time interval. MA can send *ServiceStatusDataRequestStatus* to provide the status of the OMS Isolator.

SubsystemStatus is used to report vehicle subsystem status, and doubles as a heartbeat message from Flight Autonomy (FA) to the MA. This is a periodic message with a configurable specific time interval. The *SubsystemStatusDataRequest* is originated from the MA to the FA to query the status of the FA. A response for the *SubsystemStatusDataRequest* is send out with *SubsystemStatusDataRequestStatus*. The *SubsystemStatus* schema identifies individual components of the FA in the schema to provide multiple components in the *SpecificStatus* of the UCI schema. All component aggregations are reported to Heath & Status Aggregator. This status gives MA the overall health of the FA and serves as the Heartbeat message. Note that Component identification and description are left to the interface implementers to define. *MA_Fault* contains a *Severity* field that supports the following enumerations: *NOMINAL*, *ADVISORY*, *CAUTION*, *WARNING*, and *FAILED*. This message should be used in conjunction with *SubsystemStatus* to report on both the component state and fault status.

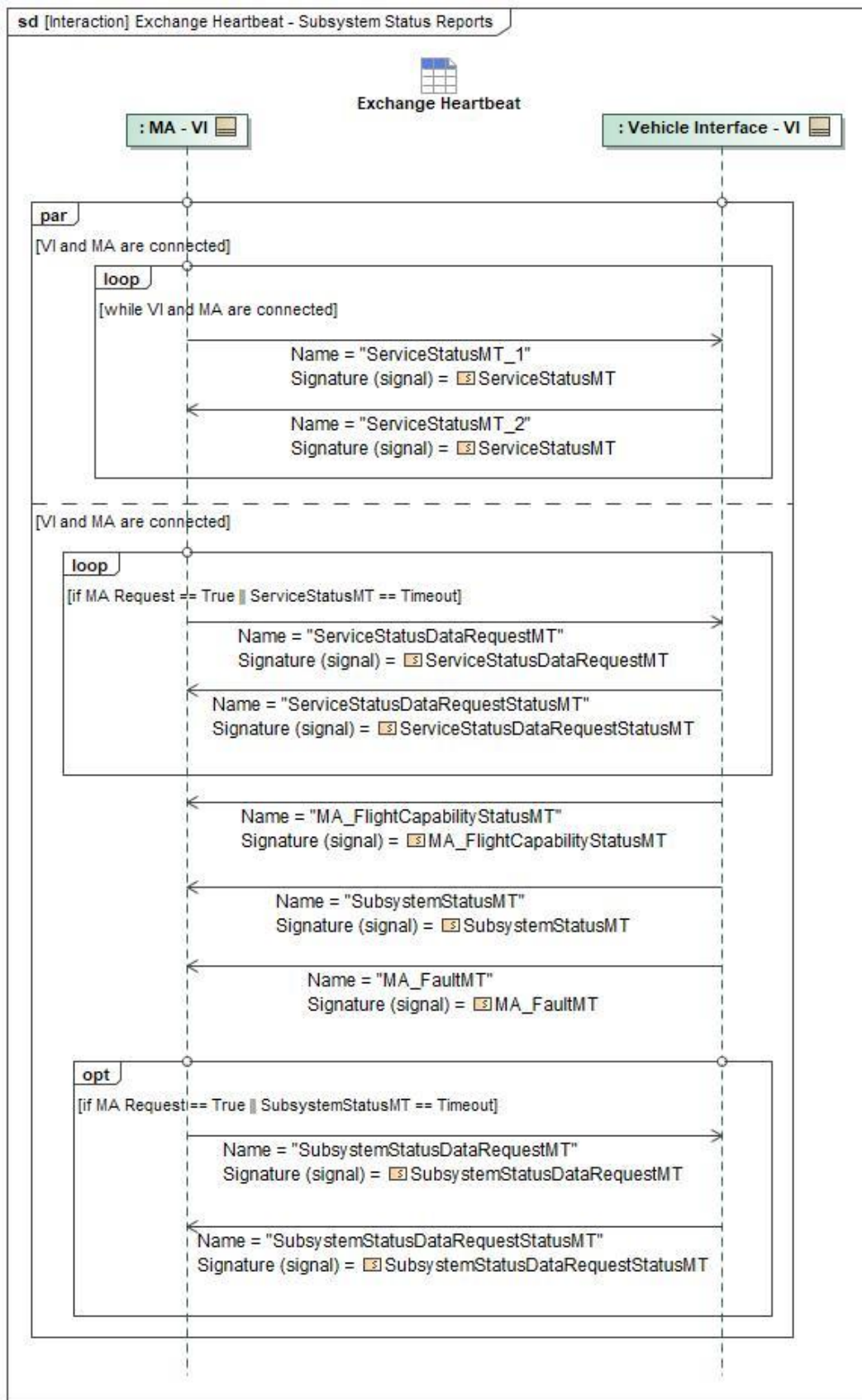


Figure 1-26: Exchange Heartbeat - Subsystem Status Reports

Table 1-25: Exchange Heartbeat - Subsystem Status Reports

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FaultMT: MA_FaultMT	Severity: FaultSeverityEnum
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None
ServiceStatusDataRequestMT: ServiceStatusDataRequestMT	None
ServiceStatusDataRequestStatusMT: ServiceStatusDataRequestStatusMT	None
ServiceStatusMT_1: ServiceStatusMT	None
ServiceStatusMT_2: ServiceStatusMT	None
SubsystemStatusDataRequestMT: SubsystemStatusDataRequestMT	None
SubsystemStatusDataRequestStatusMT: SubsystemStatusDataRequestStatusMT	None
SubsystemStatusMT: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.2 Publish Control Status

Flight Autonomy (FA), via Vehicle Interface (VI), publishes the *ControlStatusMT* message to Mission Autonomy (MA) to indicate the current controlling system and service. This status is indicated per *ControlType/CapabilityControl/CapabilityID* where *CapabilityIDs* can be identified by referencing the *MA_FlightCapabilityMT* messages from the "Control Mode Authorization" diagram. FA is responsible for safe operation of the vehicle and will always publish itself as the *PrimaryController* for all flight-related capabilities. This is indicated by specifying *ControlType/CapabilityControl/PrimaryController/SystemID* and *ControlType/CapabilityControl/PrimaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* corresponding to an appropriate OMS service on the VI. If VI is accepting inputs for a given capability from MA it will indicate a *ControlType/CapabilityControl/SecondaryController/SystemID* and *ControlType/CapabilityControl/SecondaryController/ServiceID* with the corresponding *SystemID* and *ServiceID* for a particular MA service and *CapabilityID*. Alternatively if VI is not accepting inputs from MA it will not populate any *ControlType/CapabilityControl/SecondaryController* entries.

VI will also indicate which level of control is appropriate for a given combination of *CapabilityID*. This is accomplished by enumerating the types of commands accepted by VI for that *CapabilityID* in the *ControlType/CapabilityControl/AcceptedInterface* list.

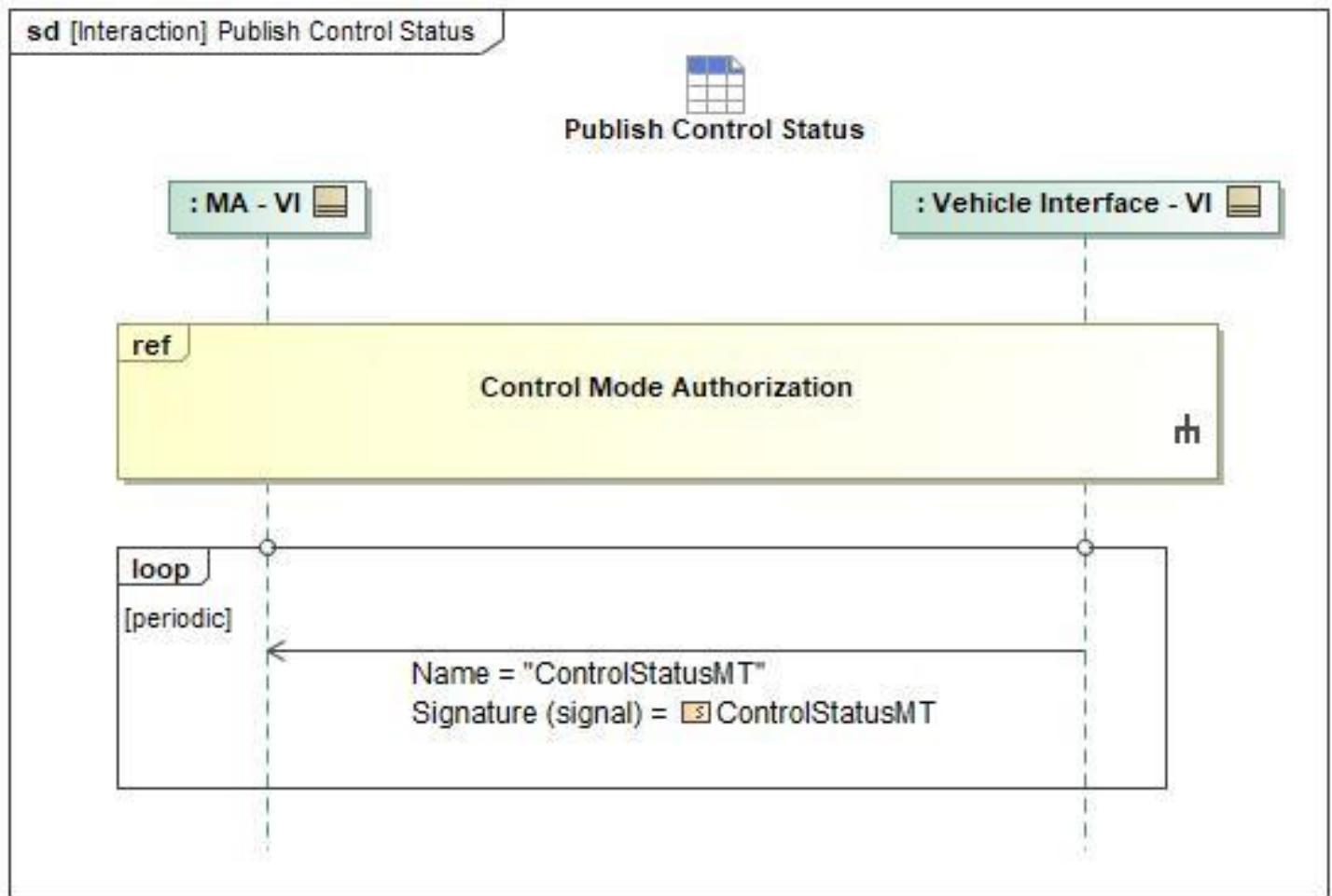


Figure 1-27: Publish Control Status

Table 1-26: Publish Control Status

Message Name (Name:Type)	Required Fields (Name:Type)
ControlStatusMT: ControlStatusMT	SecondaryController: SecondaryControllerType CapabilityControl: ControlStatusCapabilityControlType PrimaryController: PrimaryControllerType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.3 Query Airfield Update

Mission Autonomy (MA) can query Flight Autonomy (FA) for pre-loaded and pre-validated Airfield and Take off and Landing (TO/L) route information stored within FA. This data includes alternate airfields for emergency landings as well as optional alternate TO/L routes for a given airfield.

In terms of sequence, once FA has received MA's query via *QueryDataRequest*, it will respond with *QueryDataRequestStatus* indicating *QUEUED* and *PROCESSING* states. If the query is successful and *ResultsInNative* is populated with matching *MessageType*, FA will respond with *AirfieldReportMT* message, that shall contain *AirfieldRunwayType* data structures that describe the geometry of takeoff

and landing runways. To respond to the query for TO/L routes, FA shall respond with *MA_RoutePlanMT* data structures, indicating TO/L routes.

If describing a takeoff airfield, runway, and route combination, the corresponding reference fields in the two messages—*AirfieldReportMT* and *MA_RoutePlanMT*—must be populated appropriately. For example, *MA_RoutePlanMT* must have a take-off path whose *AirfieldID* and *RunwayID* match the data in *AirfieldReportMT*. The *AirfieldReportMT* Take-off coordinates and limits must also be specified. For takeoff, the 'Path' element within *MA_RouteType* will capture the takeoff route data, with its 'PathType' subelement designating the particular path as a takeoff route path, and 'PathSegment' elements representing the takeoff route data by use of waypoints. Wherever in the takeoff route that specific climb rate requirements apply (as denoted in the mission SIDS), the *Climb* subfield for the respective *PathSegment* in the takeoff procedure can specify the required Climb Rate. For aborted Take-off taxi routes, a separate 'Path' element, with a *PathType* enum of *AbortedTakeoffRoute* can be defined. The *PathSegment* element within this path can define the taxi route to exit the runway after a takeoff abort. This additional *Path* can be linked to the takeoff route through conditional chaining.

Note: To achieve conditional chaining, the *ConditionalPathSegment* subelement within the main takeoff *Path* element can be used. The *PathID* field within this *ConditionalPathSegment* subelement can point to the ID of the Aborted Takeoff Route taxi path. The *Condition* element in the *ConditionalPathSegment* can define when to take the abort path.

If describing a landing airfield, runway, and route combination, the corresponding fields must be populated appropriately. For example, the *MA_RoutePlanMT* must have a landing path whose *AirfieldID* and *RunwayID* match the data in *AirfieldReportMT*. The *AirfieldReportMT* landing coordinates and limit must also be specified. Similar to take-off procedure's implementation, the *Path* element in *MA_RouteType* will be used to define the landing approach route. If applicable, the *Climb* subfield in the *PathSegment* element will be used to define any required autonomous descent rate for the landing approach route. Lastly, similar to the aborted take-off taxi route for the take-off case, a combination of additional defined *Path* for missed approach hold procedure, along with conditional chaining to tie it to the main approach route can be used to define the missed approach procedure. If the *QueryDataRequestMT* fails, FA will respond with a *QueryDataRequestStatusMT* with the *RequestProcessingState* indicating *FAILED*.

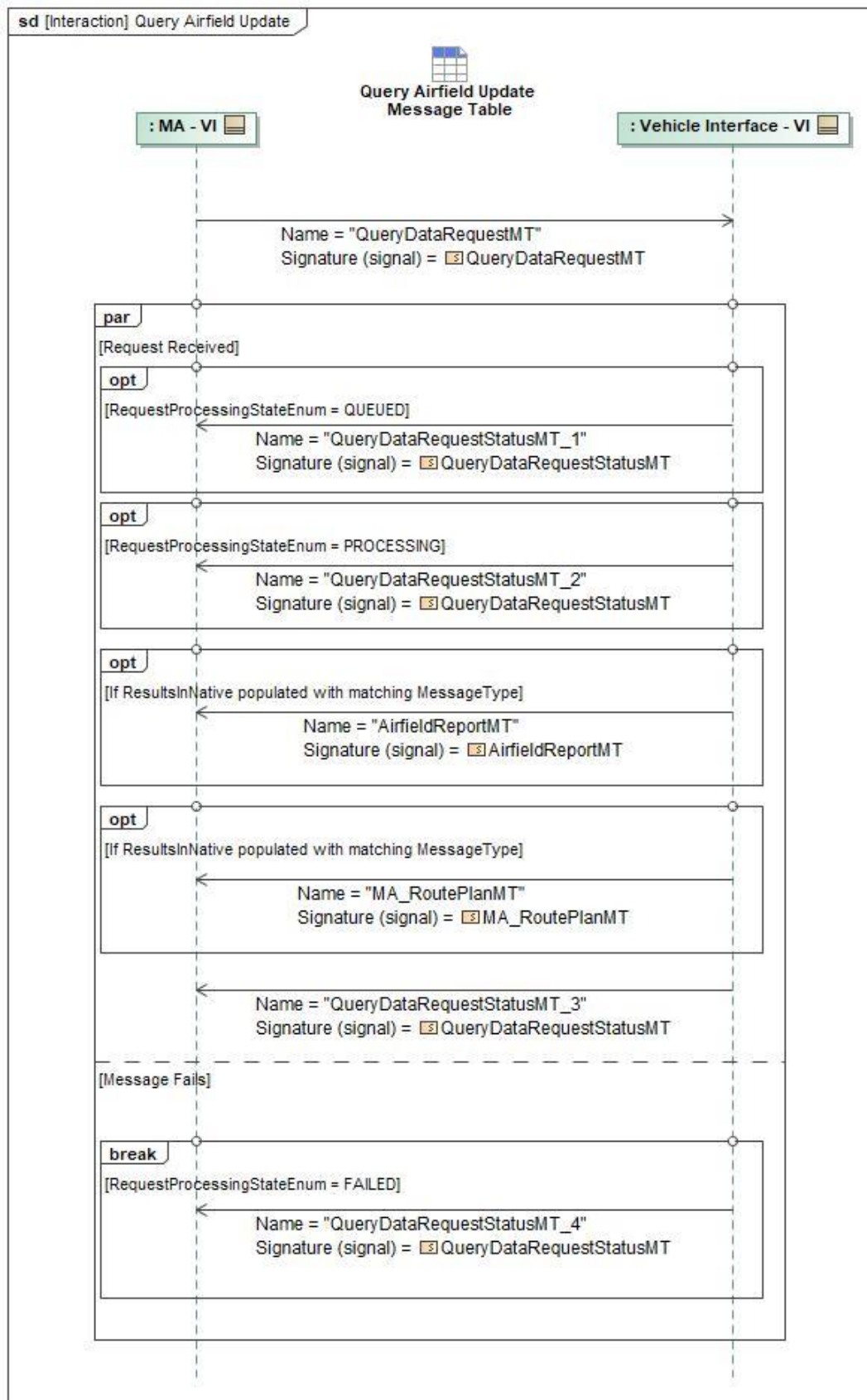


Figure 1-28: Query Airfield Update

Table 1-27: Query Airfield Update

Message Name (Name:Type)	Required Fields (Name:Type)
AirfieldReportMT: AirfieldReportMT	TakeoffCoordinates: RunwayCoordinatesType LandingCoordinates: RunwayCoordinatesType Runway: AirfieldRunwayType Limit: Point3D_Type
MA_RoutePlanMT: MA_RoutePlanMT	AirfieldID: AirfieldID_Type RunwayID: RunwayID_Type
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.4 Query Route Plan

Mission Autonomy (MA) can use *QueryDataRequestMT* to query for route plan information stored in Flight Autonomy (FA) via the Vehicle Interface (VI) including preload options for takeoff, landing, and alternate airfields for emergency. Once FA has received the query, it will respond with *QueryDataRequestStatusMT* messages providing MA with status updates for the query until it reaches a terminal state, along with the queried *MA_RoutePlanMT* (assuming the query does not fail). When FA returns the queried *MA_RoutePlanMT*, FA also sends *FileLocationMT* and *FileMetadataMT* (including the *SHA_2_Hash* field) to MA, both containing details of FA's stored UCI version (as opposed to the version converted to FA's native file format) of the *MA_RoutePlanMT*. These will enable the performance of MA-FA route plan data validation behaviors.

The *MA_RoutePlanMT* data structures can indicate takeoff and landing paths. Critically, the *AirfieldReportMT* contains *AirfieldRunwayType* data structures that describe the geometry of the associated runways for takeoff and landing.

If describing a takeoff airfield, runway, and route combination, the corresponding fields must be populated appropriately. For example, the *MA_RoutePlanMT* must have a takeoff path whose *AirfieldID* and *RunwayID* match the data in *AirfieldReportMT*. The *AirfieldReportMT TakeoffCoordinates/Limit* must also be specified. In this case, it is important that the *Route/Path/PathType* be set appropriately to indicate *TAKEOFF*.

If describing a landing airfield, runway, and route combination, the corresponding fields must be populated appropriately. For example, the *MA_RoutePlanMT* must have a landing path whose *AirfieldID* and *RunwayID* match the data in *AirfieldReportMT*. The *AirfieldReportMT LandingCoordinates/Limit* must also be specified. In this case, it is important that the

Route/Path/PathType be set appropriately to indicate *LANDING*.

Ideally, takeoff and landing information can be specified simultaneously. Furthermore, multiple approach departure, and missed approach routes can be specified for an airport and per runway(s). This can be used during normal takeoff, landing, emergency diverts, and contingency management scenarios when MA is not in control of the vehicle. When specifying a takeoff or landing path within the *MA_RoutePlanMT* the *AirfieldID* and *RunwayID* must be specified and match associated fields in the corresponding *AirfieldReportMT*

This sequence should occur early in the mission before taxi to ensure MA is aware of all available route plans and airfields saved on FA to safely takeoff and land. The sequence can occur subsequently as required if MA needs to confirm the routes stored on FA.

This airfield information and the route plans associated with takeoff, departure, approach, and landing are loaded on FA pre-mission. They are considered Safety Critical and are thus "read only". New airfields (*AirfieldReportMTs*), as well as route plans pertaining to safety critical operations takeoff, departure, approach, and are not to be sent to FA.

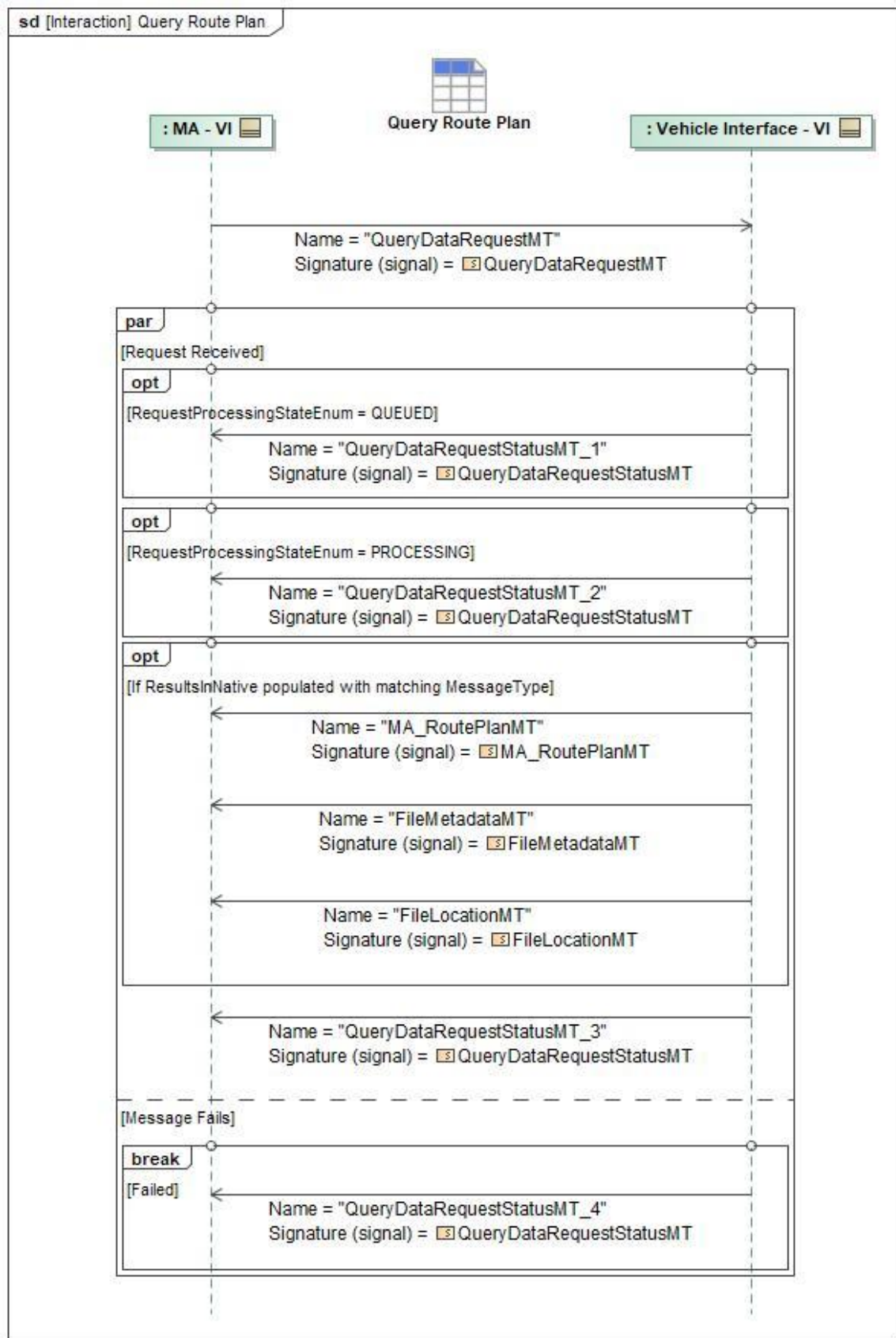


Figure 1-29: Query Route Plan

Table 1-28: Query Route Plan

Message Name (Name:Type)	Required Fields (Name:Type)
FileLocationMT: FileLocationMT	None
FileMetadataMT: FileMetadataMT	SHA_2_Hash: SHA_2_256_HashType
MA_RoutePlanMT: MA_RoutePlanMT	None
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.5 Receive Barometric Pressure

Barometric pressure reference is used for determining the vehicle's altitude above mean sea level. This value is the QNH pressure and this use case allows the value to be changed. MA provides the VI the QNH setting using the *MA_SystemManagementRequestMT* command. The *RequestType* will be *VehicleSettings*, with *VehicleSettings.QNH_Setting* set to the barometric pressure in kilopascals (kPa). The VI accepts or rejects the command using the *MA_SystemManagementRequestStatusMT* message.

The *MA_SystemManagementRequestStatusMT* will have the *RequestProcessingState* field set to "COMPLETED". If the *MA_SystemManagementRequestMT* was not able to be applied, then the *RequestProcessingState* will be set to "FAILED" and the *RequestProcessingStateReason* set to the reason the *MA_SystemManagementRequestMT* couldn't be applied.

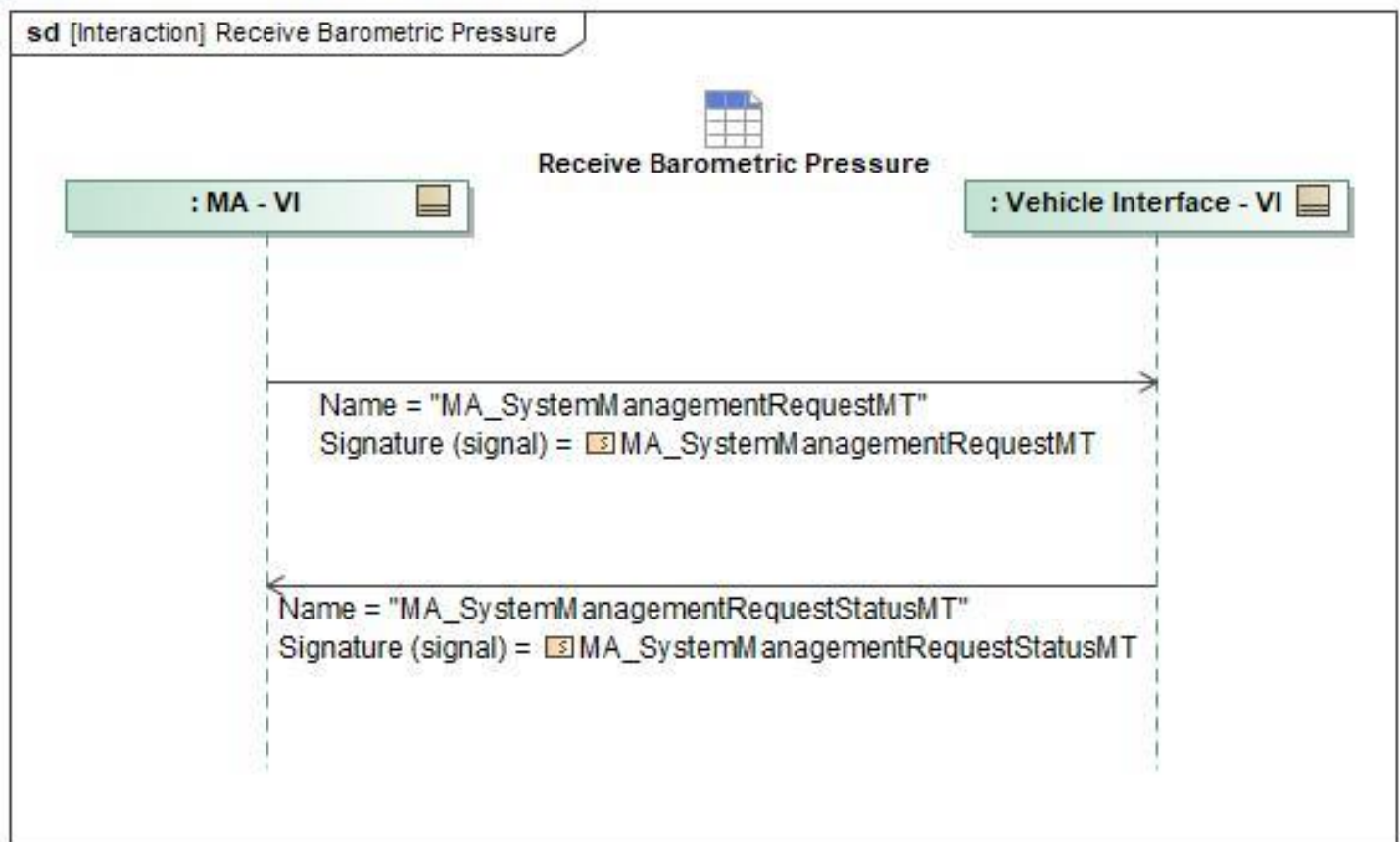


Figure 1-30: Receive Barometric Pressure

Table 1-29: Receive Barometric Pressure

Message Name (Name:Type)	Required Fields (Name:Type)
MA_SystemManagementRequestMT: MA_SystemManagementRequestMT	None
MA_SystemManagementRequestStatusMT: MA_SystemManagementRequestStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.6 Receive Execution Status

VI will periodically provide updates to MA on the status of actively executing or queued actions, tasks, plans, routes, and missions.

MA_ActionStatusMT and *TaskStatusMT* convey the status of the ongoing Action or Task. Both *ActionStatusMT* and *TaskStatusMT* contain the required *ExecutionState* field denoting the executing state.

If VI is executing a route, it will send *RoutePlanExecutionStatusMT* and *RouteActivityPlanExecutionStatusMT* containing the required field *PlanExecutionStatus.PlanExecutionState* and *PlanExecutionStatus.RouteActivityPlanExecutionStateType*, respectively, denoting the executing

state.

If VI is executing a mission, it will send *MissionPlanExecutionStatusMT* and *MissionPlanActivationStatusMT* containing the required field *PlanExecutionStatus.MissionPlanExecutionStateType* and *PlanActivationState.PlanActivationStateEnum*, respectively, denoting the executing state.

If VI is executing an activity plan or task plan, it will send *ActivityPlanExecutionStatusMT* and *TaskPlanExecutionStatusMT* containing the required field *PlanExecutionStatus.ActivityPlanExecutionStateType* and *PlanExecutionStatus.TaskPlanExecutionStateType*, respectively, denoting the executing state.

If VI is executing a response, it will send *ResponsePlanExecutionStatusMT* containing the required field *PlanExecutionStatus.ResponsePlanExecutionStateType* denoting the executing state of the response plan.

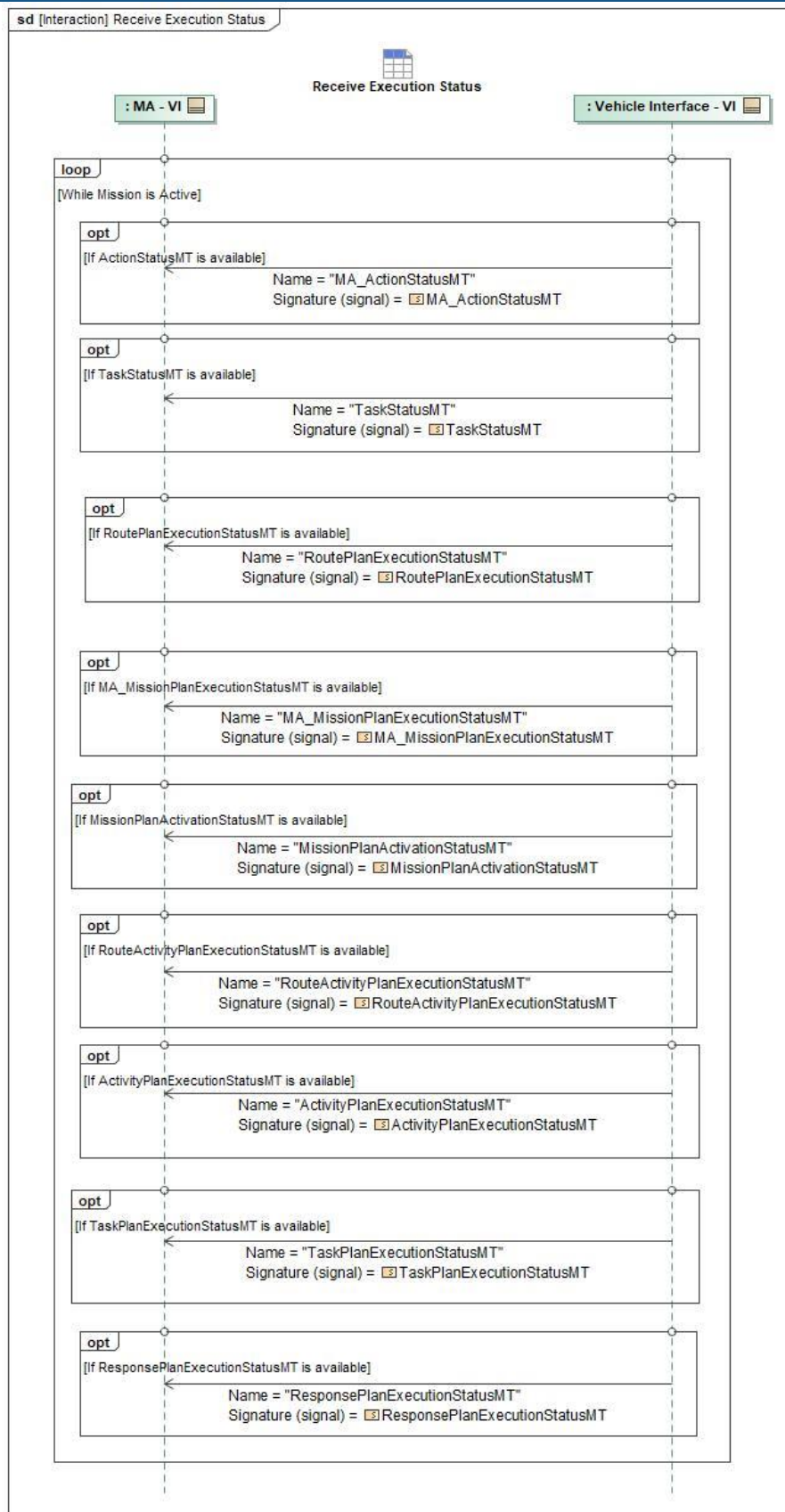


Figure 1-31: Receive Execution Status

Table 1-30: Receive Execution Status

Message Name (Name:Type)	Required Fields (Name:Type)
ActivityPlanExecutionStatusMT: ActivityPlanExecutionStatusMT	None
MA_ActionStatusMT: MA_ActionStatusMT	None
MA_MissionPlanExecutionStatusMT: MA_MissionPlanExecutionStatusMT	None
MissionPlanActivationStatusMT: MissionPlanActivationStatusMT	None
ResponsePlanExecutionStatusMT: ResponsePlanExecutionStatusMT	None
RouteActivityPlanExecutionStatusMT: RouteActivityPlanExecutionStatusMT	None
RoutePlanExecutionStatusMT: RoutePlanExecutionStatusMT	None
TaskPlanExecutionStatusMT: TaskPlanExecutionStatusMT	None
TaskStatusMT: TaskStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.7 Receive Vehicle Performance Values

The MA subscribes to *MA_FlightCapability* and *MA_FlightCapabilityStatus* as published across the Vehicle Interface (VI) to discover the currently available control modes. Also included in the *MA_FlightCapability* message are command limits that are applicable to the current command mode. These limits can be used by MA to inform trajectory shaping and commands to ensure MA commands to the vehicle are executable. As the vehicle should be continuously publishing *MA_FlightCapability*, these limits can be updated on a regular basis such that MA can monitor any changes to the performance profile as a function of vehicle configuration or flight condition. These limits do not have to specifically be limits associated with the vehicle AOL (aircraft operational limits), but should be provided as guard rails for MA commands, and based on the control mode limitations, could correlate to AOLs. The profile limits will be found under *MA_FlightCapability* as *FlightCapabilityPerformance*. The *FlightCapabilityPerformanceProfile* will be filled by parameters relevant to the selected control mode (currently HSA/CSA, Waypoint Following, or Curve Following) using the fields *HSA_CSA_PerformanceProfile*, *WaypointFollowingPerformanceProfile*, or *CurveFollowingPerformanceProfile*. The data package's elements may be optionally filled according to the needs of each control mode, and will be defined with *MA_FlightControlModesPerformanceProfile*. Maximum and minimum airspeed limits are referenced to *TRUE_AIRSPEED* in *SpeedReferenceEnum* and will be defined as a set from 0 to n to allow for a progression of velocities with an altitude pair to aid in the definition of a flight envelope. *MinAltitude* and *MaxAltitude* will also be defined to aid in checking trajectory validity. These values will be updated to reflect the current altitude limits at the current point in a vehicle's flight envelope. The *MinAccelerationLimits* and the *MaxAccelerationLimits* are defined using the same data structure. The acceleration limits are indexed from 0 to n to create a flight envelope boundary. The limits in all three

axes can be reported in both inertial and body axes. The body axis limits are scheduled as a function of *MachValue* in order to define a boundary that resembles a load's flight envelope. The *MaxOrientationLimits* also known as the attitude of the vehicle can also be limited in all three axes and reported at the current position in the flight envelope. The *MaxOrientationRateLimits* are the reported orientation rate limits paired with airspeed to define a boundary to the flight envelope. The rate limits are defined by roll rate, pitch rate, and yaw rate in order to account for both symmetric and asymmetric aircraft loading. There are also limiting values that are not part of the basic aircraft operational limits but are defined by the control system. These limits are reported at the current part of the flight envelope. The limits include maximum turn rate, maximum climb rate, and maximum descent rate.

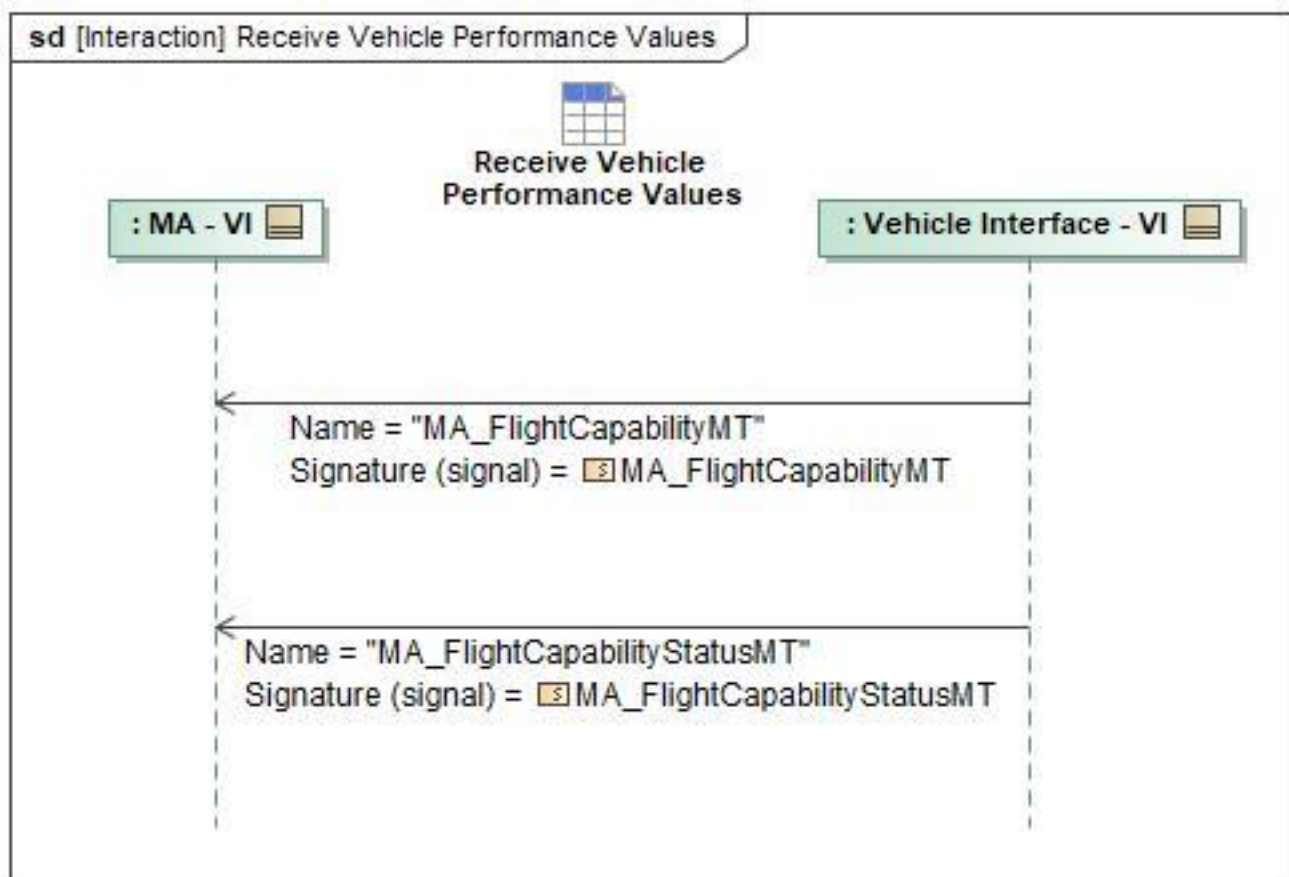


Figure 1-32: Receive Vehicle Performance Values

Table 1-31: Receive Vehicle Performance Values

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCapabilityMT: MA_FlightCapabilityMT	None
MA_FlightCapabilityStatusMT: MA_FlightCapabilityStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.8 Receive Vehicle State Data

MA treats the VI as a time, space, and position information (TSPI) truth source. A selection of parameters is provided in this ICD to ensure MA receives appropriate state information from the VI to operate in a safe and efficient manner. Within the *Kinematics* field in the *MA_PositionReportDetailedMT* message, *Position*, *Velocity*, *AirData*, *Acceleration*, *Orientation*, *WanderAngle*, *MagneticHeading*, *OrientationRate*, and *OrientationAcceleration* must be provided. The *AirData* field defines *IndicatedBaroAltitude*, *Kollsman*, *TrueAirspeed*, *CalibratedAirspeed*, and *Mach*. These messages should be published from the vehicle to MA at the highest supportable rates. In addition to providing the *MA_PositionReportDetailedMT* message, "Winds" are conveyed to MA via the *WeatherObservationMT* message with the *WindDataType* field. Within the *WindDataType* field, the *Source* enumeration will be set to *Other* to indicate the measurements are from the vehicle platform and only contain wind data. *ObservationPoint* will also be provided to indicate time and location of the wind data measurements.

FlightActivity has been extended to include Vehicle Command States. These are the commands FA is using to fly the aircraft, and can be used by MA to better estimate the vehicle's energy state. Finally, the base schema messages of *ComponentStatus* and *NavigationReport* are used to report component status information, such as engine data, and the overall fuel state of the vehicle, respectively.

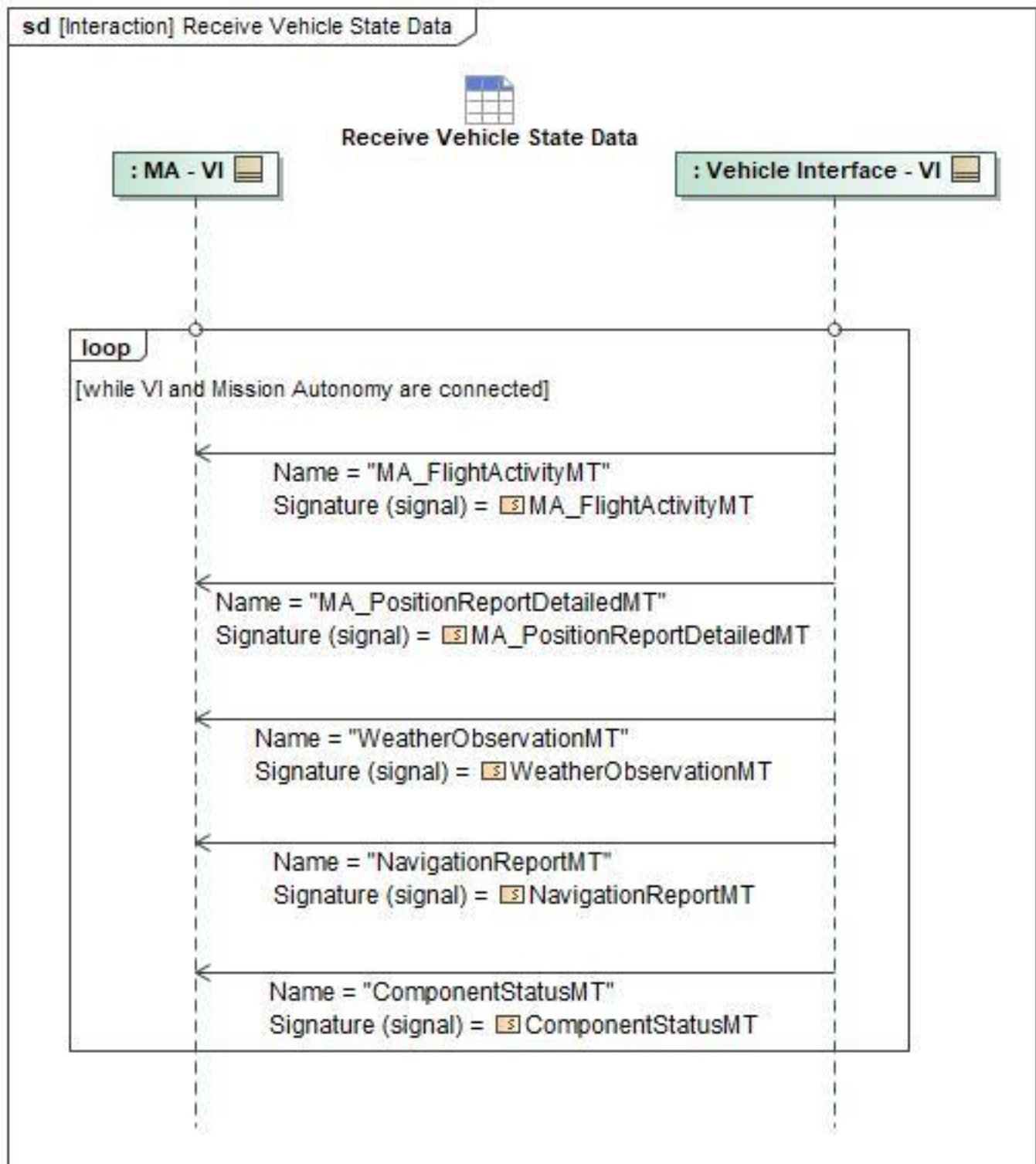


Figure 1-33: Receive Vehicle State Data

Table 1-32: Receive Vehicle State Data

Message Name (Name:Type)	Required Fields (Name:Type)
ComponentStatusMT: ComponentStatusMT	None

MA_FlightActivityMT: MA_FlightActivityMT	None
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	Orientation: OrientationType MagneticHeading: AngleType WanderAngle: AngleType OrientationRate: OrientationRateType OrientationAcceleration: OrientationAccelerationType Acceleration: Acceleration3D_Type AirData: MA_AirDataType
NavigationReportMT: NavigationReportMT	None
WeatherObservationMT: WeatherObservationMT	WindChoice: WindDataChoiceType WindData: WindDataType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.9 Request Terrain Data

Mission Use Case: MA Terrain Data

MA must have a representation of terrain data to accomplish the validation of flight commands and routes prior to sending to FA. This increases the likelihood that MA will produce flight commands that will be accepted by FA's safety critical validator. Digital Terrain Elevation Data (DTED) is a standard of digital datasets which consists of a matrix of terrain elevation values. DTED is a standard National Geospatial-Intelligence Agency (NGA) product that provides medium resolution, quantitative data in a digital format for military system applications that require terrain elevation. Although DTED is the most widely used there could be other elevation sources like the Global Terrain Elevation MAP (GDEM). DTED is specifically referenced here for contextual reasons only, not as a requirement for A-GRA compliance.

The DTED format for level 0, 1 and 2 is described in U.S. Military Specification Digital Terrain Elevation Data (DTED) MIL-PRF-89020B, and amongst other parameters describes the resolution for each level:

- Level 0 has a post spacing of approximately 900 meters.
- Level 1 has a post spacing of approximately 90 meters.
- Level 2 has a post spacing of approximately 30 meters.

For FA safety critical systems such as a Ground Collision Avoidance System (GCAS), level 2 or higher DTED is required for a higher fidelity representation of the terrain. Because FA will have higher fidelity terrain representation's, MA can query the higher fidelity terrain data for specific areas to perform specific missions that are tactically relevant. To do this, MA will send the *ElevationRequestMT* to FA that will specify multiple *RequestPoint* fields building a specific zone of elevation data. If FA has the specific *RequestPoint(s)* requested by MA, FA will respond with *ElevationRequestStatusMT*, specifying the *ElevationReturned* field, which will include *RequestPoint* fields characterized by level 2 or higher DTED or similar elevation data. If FA does not have the specific *RequestPoint(s)* specified by MA, FA will respond with a *RequestProcessingState* of *FAILED*,

CANCELED, or *REJECTED* and MA will do an internal terrain request for lower fidelity, non-safety critical terrain data. In this case, FA is still the safety critical system responsible for ground collision avoidance and geofence violation avoidance, which may include a defined floor geofence.

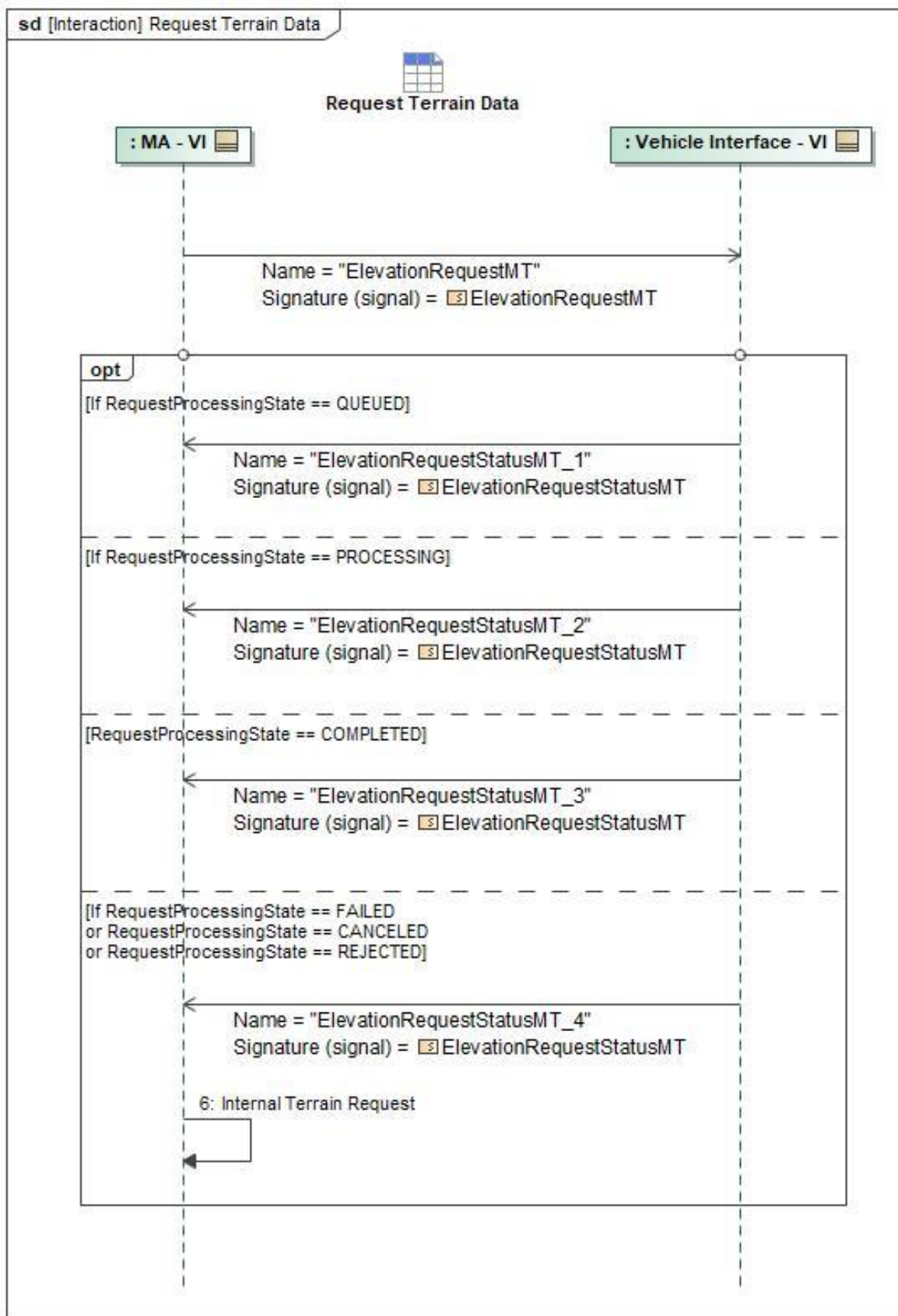


Figure 1-34: Request Terrain Data

Table 1-33: Request Terrain Data

Message Name (Name:Type)	Required Fields (Name:Type)
ElevationRequestMT: ElevationRequestMT	None
ElevationRequestStatusMT_1: ElevationRequestStatusMT	QUEUED:
ElevationRequestStatusMT_2: ElevationRequestStatusMT	PROCESSING:
ElevationRequestStatusMT_3: ElevationRequestStatusMT	COMPLETED:
ElevationRequestStatusMT_4: ElevationRequestStatusMT	FAILED: REJECTED: CANCELED:
Internal Terrain Request:	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.10 Vehicle Status Reporting

SubsystemStatusMT shall be used to report status of specific FA components, as well as doubles as a heartbeat message between MA and FA. MA and FA exchange *SubsystemStatusMT_1* and *SubsystemStatusMT_2* asynchronously, not in response to one another. All component aggregations are reported to MA. The *SubsystemStatusMT* schema can identify individual components of FA generically using the *SubsystemComponent.SpecificStatus* field.

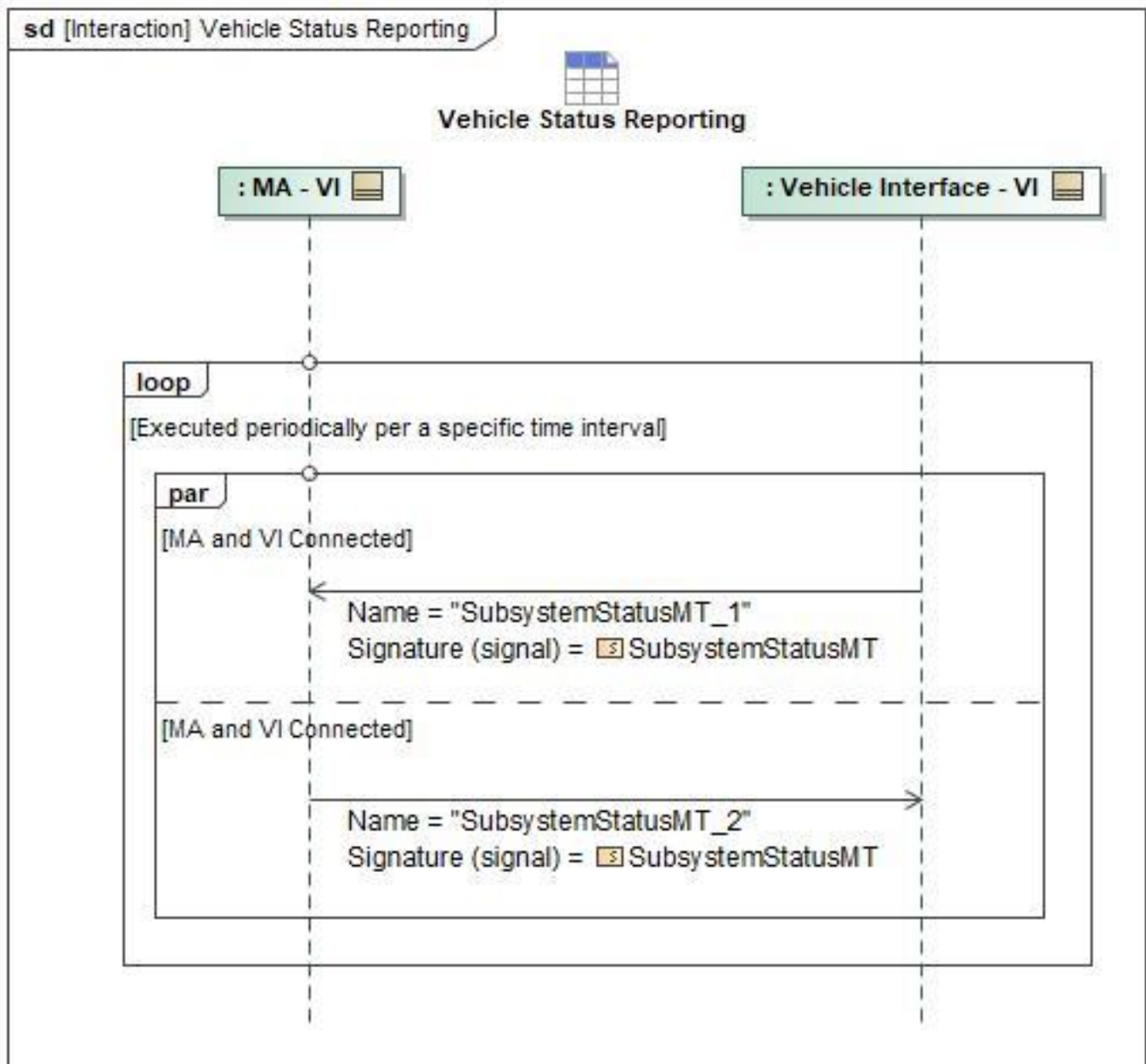


Figure 1-35: Vehicle Status Reporting

Table 1-34: Vehicle Status Reporting

Message Name (Name:Type)	Required Fields (Name:Type)
SubsystemStatusMT_1: SubsystemStatusMT	None
SubsystemStatusMT_2: SubsystemStatusMT	None

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.6.11 VI Responds to Query for Flight Capabilities

MA_FlightCapabilityMT and *MA_FlightCapabilityStatusMT* are delivered from VI to MA in the *Receive*

Vehicle Performance Values reference. This data is queried via *QueryDataRequestMT* and responded to with one or more *QueryDataRequestStatusMT* messages. Data management services may also respond with intermediate *QueryDataRequestStatusMT* messages with *RequestProcessingStateEnum* set to indicate additional in-process states, such as "QUEUED" and "PROCESSING".

When complete, the *QueryDataRequestStatusMT* message's *RequestProcessingStateEnum* is set to "COMPLETED" and contains the requested data.

When the original *QueryDataRequestMT*'s optional field *ResultsInNative* is populated with the matching message type, the underlying data management services instead publishes the requested data in its original or raw format as a separate message and indicates *QueryDataRequestStatusMT*.

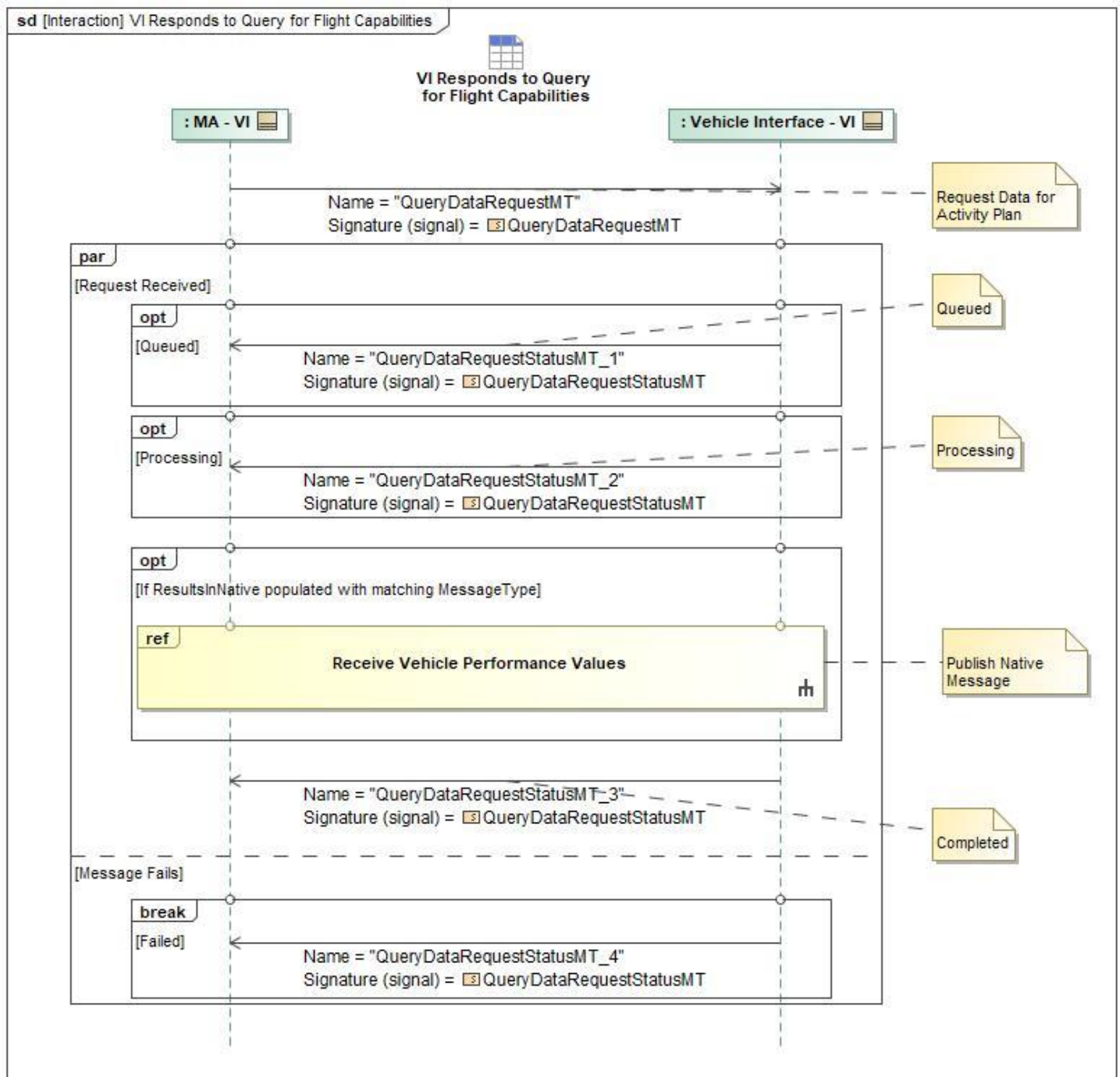


Figure 1-36: VI Responds to Query for Flight Capabilities

Table 1-35: VI Responds to Query for Flight Capabilities

Message Name (Name:Type)	Required Fields (Name:Type)
QueryDataRequestMT: QueryDataRequestMT	None
QueryDataRequestStatusMT_1: QueryDataRequestStatusMT	None
QueryDataRequestStatusMT_2: QueryDataRequestStatusMT	None

QueryDataRequestStatusMT_3: QueryDataRequestStatusMT	Result: QueryResultType
QueryDataRequestStatusMT_4: QueryDataRequestStatusMT	RequestProcessingStateReason: CannotComplyType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

1.2.7 Weapon Employment Package

1.2.7.1 Validate Release Envelope

Prior to this interaction, MA has sent MS a *MA_Task* of type *StrikeTask* for the intended target and calculated its preferred maneuver for striking a target. MA has also validated the target's ID and that the strike complies with any enabled ROE. To begin, MA sends its maneuver through a *MA_FlightCommandMT* to VI. This command is associated with the existing Strike Task through its identifier (*TaskID*), set in the *MA_FlightCommandMT* message's *Capability.Traceability.Requirement* field. Upon receiving this message, VI will determine via vehicle and store position if the criteria to release the weapons store is met. If the release envelope criteria are met, VI will return the *MA_FlightCommandStatusMT* to MA with status *ACCEPTED*, indicating acceptance of the *MA_FlightCommandMT*, and begin the maneuver. If the criteria are not met for release, VI will send a *MA_FlightCommandStatusMT* with status of *REJECTED* to MA. As maneuvering continues, MA will use position information in *MA_PositionReportDetailedMT* to determine if the ownship has maneuvered to the expected position for release.

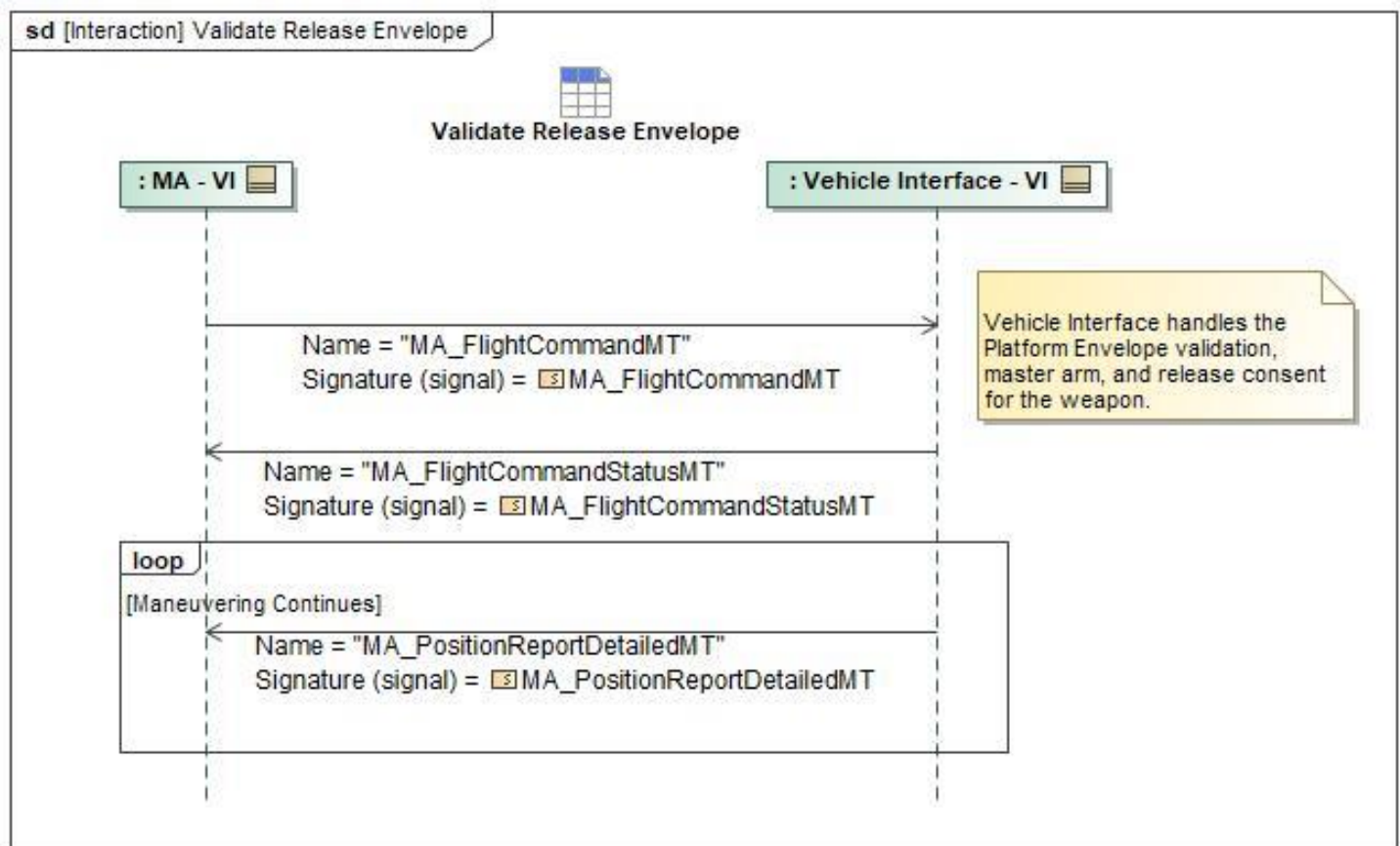


Figure 1-37: Validate Release Envelope

Table 1-36: Validate Release Envelope

Message Name (Name:Type)	Required Fields (Name:Type)
MA_FlightCommandMT: MA_FlightCommandMT	Traceability: TraceabilityType Requirement: RequirementInstanceID_ChoiceType
MA_FlightCommandStatusMT: MA_FlightCommandStatusMT	None
MA_PositionReportDetailedMT: MA_PositionReportDetailedMT	Orientation: OrientationType MagneticHeading: AngleType WanderAngle: AngleType OrientationRate: OrientationRateType OrientationAcceleration: OrientationAccelerationType Acceleration: Acceleration3D_Type AirData: MA_AirDataType

Note: The fields above are required for this use case in addition to what is specified in Autonomy Schema. The exhaustive list of all fields are in the Autonomy Schema. (See Message Set Tables section)

This diagram captures all the interactions expected between VI and MA. Each interaction contains a sequence diagram describing the interaction.

Content Diagram Interactions [Vehicle Interface Interactions]

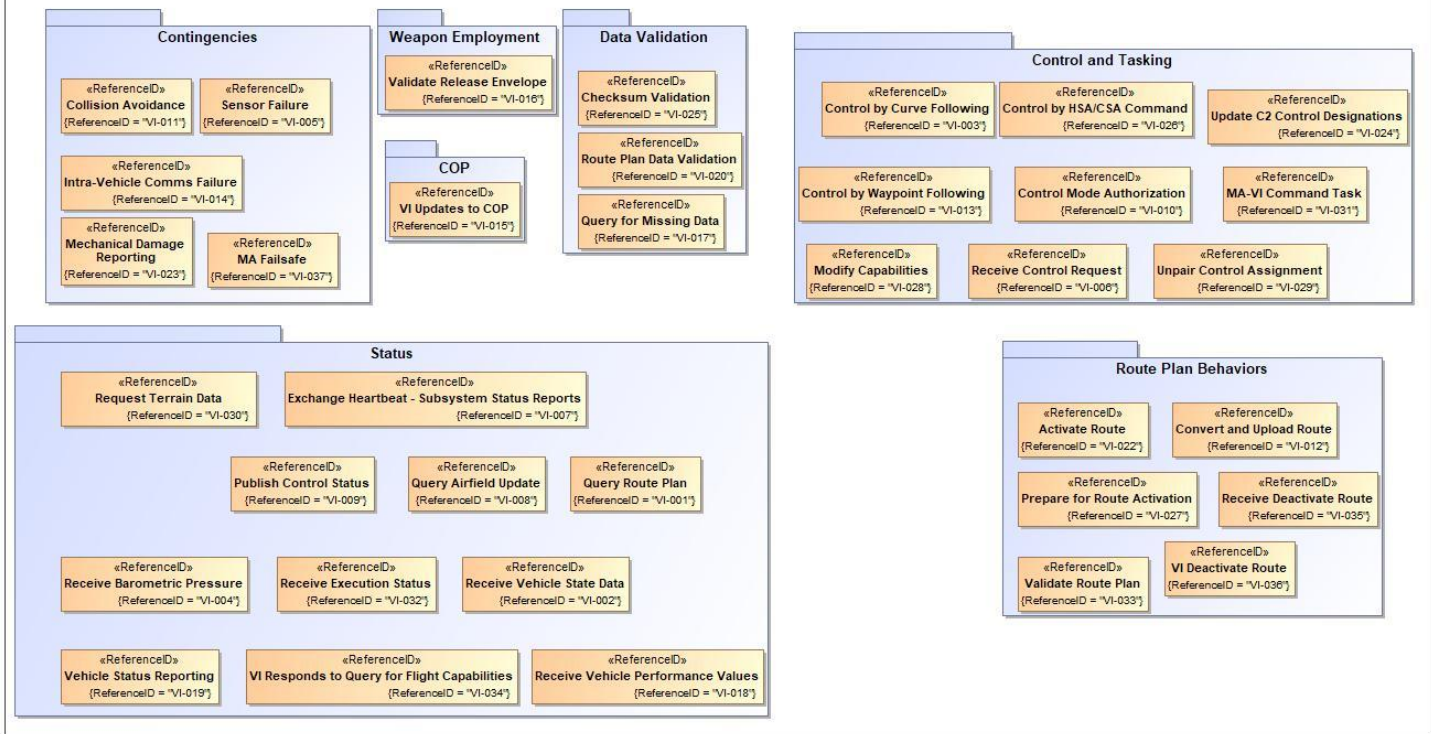


Figure 1-38: Vehicle Interface Interactions

1.3 Interface Compliance

The Interface Compliance table contains the compliance requirements for the L1 C2 interface.

Table 1-37: Vehicle Interface Compliance Requirements

ID	Requirement
MA-L1-015	A Level 1 VI interface shall support the VI Interface Minimum Message Set defined in this architecture for signals that own a "VI = 1" tagged value.
MA-L1-016	A Level 1 VI interface shall implement all required messaging sequences in the VI feature profile, unless that sequence is marked as "optional" (1 Problem Domain > 2 White Box > 3 Functional Analysis > L1 Functional Analysis > L1 Interface Behaviors > VI Interface Behavior).

1.3.1 VI MMS

The Minimum Message Set referenced in the Interface Compliance Requirements is shown in this section. For an implementor to be compliant, all messages shown in the table shall be supported. The "Direction" column specifies the message direction relative to the VI interface. The "MUC" column specifies the message's Mission Use Case. The message and its respective interaction(s) are only required for the particular Mission Use Case listed.

Mission Use Cases (MUCs) identify specific implementation needs for different ACP solutions. These include a Core set of behaviors that all A-GRA-compliant systems must meet, as well as select use

cases such as Carrier-based or Electronic Support Measure-capable. MUCs are considered to be L0 as they describe the use of an ACP, not necessarily a single L1 system. For L1 interface compliance, all implementations must meet the interactions delegated to the Core MUC and all interactions delegated to other MUCs relevant to their implementation. Compliance to interactions not delegated to MUCs do not count towards A-GRA compliance.

Table 1-38: VI MMS

Message Type	Message Name	Interface	Direction	Applicable MUCs
UCI 2.3	ActivityPlanExecutionStatusMT	VI	out	Core
UCI 2.3	AirfieldReportMT	VI	out	Core
UCI 2.3	ComponentStatusMT	VI	out	Core
UCI 2.3	ControlStatusMT	VI	out	Core
UCI 2.3	ElevationRequestMT	VI	in	MA Terrain Data
UCI 2.3	ElevationRequestStatusMT	VI	out	MA Terrain Data
UCI 2.3	FileLocationMT	VI	out	Core
UCI 2.3	FileMetadataMT	VI	out	Core
A-GRA Custom Extension	MA_ActionStatusMT	VI	out	Core
A-GRA Custom Extension	MA_ControlAssignmentMT	VI	out	Core
A-GRA Custom Extension	MA_ControlRequestMT	VI	in	Core
A-GRA Custom Extension	MA_ControlRequestStatusMT	VI	out	Core
A-GRA Custom Extension	MA_FaultMT	VI	out	Core
A-GRA Custom Extension	MA_FlightActivityMT	VI	out	Core
A-GRA Custom Extension	MA_FlightCapabilityMT	VI	inout	Core
A-GRA Custom Extension	MA_FlightCapabilityStatusMT	VI	out	Core
A-GRA Custom Extension	MA_FlightCommandMT	VI	in	Core
A-GRA Custom Extension	MA_FlightCommandStatusMT	VI	out	Core
A-GRA Custom Extension	MA_MissionPlanActivationCommandMT	VI	inout	Core
A-GRA Custom Extension	MA_MissionPlanActivationCommandStatusMT	VI	out	Core
A-GRA Custom Extension	MA_MissionPlanExecutionStatusMT	VI	out	Core
A-GRA Custom Extension	MA_PositionReportDetailedMT	VI	out	Core
A-GRA Custom Extension	MA_ResponseMT	VI	in	Core
A-GRA Custom Extension	MA_RoutePlanMT	VI	inout	Core

A-GRA Custom Extension	MA_SystemManagementRequestMT	VI	in	Core
A-GRA Custom Extension	MA_SystemManagementRequestStatusMT	VI	out	Core
A-GRA Custom Extension	MA_SystemNotificationMT	VI	out	Core
A-GRA Custom Extension	MA_TaskCommandMT	VI	in	Core
A-GRA Custom Extension	MA_TaskCommandStatusMT	VI	out	Core
A-GRA Custom Extension	MA_TaskMT	VI	inout	Core
UCI 2.3	MissionPlanActivationStatusMT	VI	out	Core
UCI 2.3	NavigationReportMT	VI	out	Core
UCI 2.3	QueryDataRequestMT	VI	in	Core
UCI 2.3	QueryDataRequestStatusMT	VI	out	Core
UCI 2.3	ResponsePlanExecutionStatusMT	VI	out	Core
UCI 2.3	RouteActivityPlanExecutionStatusMT	VI	out	Core
UCI 2.3	RoutePlanExecutionStatusMT	VI	out	Core
UCI 2.3	RoutePlanValidationCommandMT	VI	in	Core
UCI 2.3	RoutePlanValidationCommandStatusMT	VI	out	Core
UCI 2.3	RoutePlanValidationMT	VI	out	Core
UCI 2.3	ServiceStatusDataRequestMT	VI	in	Core
UCI 2.3	ServiceStatusDataRequestStatusMT	VI	out	Core
UCI 2.3	ServiceStatusMT	VI	inout	Core
UCI 2.3	SubsystemStatusDataRequestMT	VI	in	Core
UCI 2.3	SubsystemStatusDataRequestStatusMT	VI	out	Core
UCI 2.3	SubsystemStatusMT	VI	inout	Core
UCI 2.3	TaskPlanExecutionStatusMT	VI	out	Core
UCI 2.3	TaskStatusMT	VI	out	Core
UCI 2.3	WeatherObservationMT	VI	out	Core

1.3.2 VI UCI Extensions

1.3.2.1 MA_FlightCapabilityMT UCI Extension

MA_FlightCapabilityMT	
Description	Extended to indicate the type of flight capability via the FlightCapabilityEnum and provide per-control mode performance parameters. See the XSD File for more info
Ext. Fields	HSA_CSA_PerformanceProfile : MA_FlightControlModesPerformanceProfileType WaypointFollowingPerformanceProfile : MA_FlightControlModesPerformanceProfileType CurveFollowingPerformanceProfile : MA_FlightControlModesPerformanceProfileType

AirspeedLimit : PathSegmentSpeedValueType
 AccelerationLimit : MA_BodyReferenceAccelerationType
 AccelerationLimitPair : MA_AccelerationLimitPairType
 BodyReferenceOrientationRate : MA_BodyReferenceOrientationRateType
 OrientationRateLimits : MA_BodyReferenceOrientationRateType
 AirspeedPair : PathSegmentSpeedValueType

Table A-1-39: Vehicle Interface MA_FlightCapabilityMT UCI Extension

HSA_CSA_PerformanceProfile		
Description	Indicates performance profile for HSA_CSA flight control mode.	
Base Type	MA_FlightControlModesPerformanceProfileType	
Message Structure		
Field Name	Field Type	Field Usage
MinAirspeed	MA_AirspeedLimitType	Repeated
Description	Minimum allowable airspeed given as a function of altitude.	
MaxAirspeed	MA_AirspeedLimitType	Repeated
Description	Maximum allowable airspeed given as a function of altitude.	
BestEnduranceAirspeed	MA_AirspeedLimitType	Repeated
Description	Airspeed of maximum endurance given as a function of altitude.	
MinAltitude	AltitudeReferenceType	Optional
Description	Minimum allowable altitude.	
MaxAltitude	AltitudeReferenceType	Optional
Description	Maximum allowable altitude.	
MinAccelerationLimits	MA_AccelerationLimitsType	Repeated
Description	The vehicle's acceleration limits in the current operating domain of the system as a function of time. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxAccelerationLimits	MA_AccelerationLimitsType	Repeated
Description	The vehicle's acceleration limits in the current operating domain of the system as a function of time. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxOrientationLimits	OrientationType	Repeated
Description	Euler Angle Sequence describing the maximum orientation of the vehicle in the order yaw, pitch, roll as a function of time. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	

MaxOrientationRateLimits	MA_OrientationRateLimitsType	Repeated
Description	Euler Angle Sequence describing the rate of orientation change of the vehicle in the order yaw, pitch, roll. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxTurnRate	AngleRateType	Optional
Description	Maximum allowable turn rate in radians/sec.	
MaxClimbRate	SpeedType	Optional
Description	Maximum allowable climb rate in meters/sec.	
MaxDescentRate	SpeedType	Optional
Description	Maximum allowable descent rate in meters/sec.	

Table A-1-40: Vehicle Interface MA_FlightCapabilityMT - HSA_CSA_PerformanceProfile UCI Extension Field

WaypointFollowingPerformanceProfile		
Description	Indicates performance profile for Waypoint Following flight control mode.	
Base Type	MA_FlightControlModesPerformanceProfileType	
Message Structure		
Field Name	Field Type	Field Usage
MinAirspeed	MA_AirspeedLimitType	Repeated
Description	Minimum allowable airspeed given as a function of altitude.	
MaxAirspeed	MA_AirspeedLimitType	Repeated
Description	Maximum allowable airspeed given as a function of altitude.	
BestEnduranceAirspeed	MA_AirspeedLimitType	Repeated
Description	Airspeed of maximum endurance given as a function of altitude.	
MinAltitude	AltitudeReferenceType	Optional
Description	Minimum allowable altitude.	
MaxAltitude	AltitudeReferenceType	Optional
Description	Maximum allowable altitude.	
MinAccelerationLimits	MA_AccelerationLimitsType	Repeated
Description	The vehicle's acceleration limits in the current operating domain of the system as a function of time. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxAccelerationLimits	MA_AccelerationLimitsType	Repeated
Description	The vehicle's acceleration limits in the current operating domain of the system as a function of time. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxOrientationLimits	OrientationType	Repeated

Description	Euler Angle Sequence describing the maximum orientation of the vehicle in the order yaw, pitch, roll as a function of time. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxOrientationRateLimits	MA_OrientationRateLimitsType	Repeated
Description	Euler Angle Sequence describing the rate of orientation change of the vehicle in the order yaw, pitch, roll. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxTurnRate	AngleRateType	Optional
Description	Maximum allowable turn rate in radians/sec.	
MaxClimbRate	SpeedType	Optional
Description	Maximum allowable climb rate in meters/sec.	
MaxDescentRate	SpeedType	Optional
Description	Maximum allowable descent rate in meters/sec.	

Table A-1-41: Vehicle Interface MA_FlightCapabilityMT - WaypointFollowingPerformanceProfile UCI Extension Field

CurveFollowingPerformanceProfile		
Description	Indicates performance profile for Curve Following flight control mode.	
Base Type	MA_FlightControlModesPerformanceProfileType	
Message Structure		
Field Name	Field Type	Field Usage
MinAirspeed	MA_AirspeedLimitType	Repeated
Description	Minimum allowable airspeed given as a function of altitude.	
MaxAirspeed	MA_AirspeedLimitType	Repeated
Description	Maximum allowable airspeed given as a function of altitude.	
BestEnduranceAirspeed	MA_AirspeedLimitType	Repeated
Description	Airspeed of maximum endurance given as a function of altitude.	
MinAltitude	AltitudeReferenceType	Optional
Description	Minimum allowable altitude.	
MaxAltitude	AltitudeReferenceType	Optional
Description	Maximum allowable altitude.	
MinAccelerationLimits	MA_AccelerationLimitsType	Repeated
Description	The vehicle's acceleration limits in the current operating domain of the system as a function of time. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	

MaxAccelerationLimits	MA_AccelerationLimitsType	Repeated
Description	The vehicle's acceleration limits in the current operating domain of the system as a function of time. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxOrientationLimits	OrientationType	Repeated
Description	Euler Angle Sequence describing the maximum orientation of the vehicle in the order yaw, pitch, roll as a function of time. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxOrientationRateLimits	MA_OrientationRateLimitsType	Repeated
Description	Euler Angle Sequence describing the rate of orientation change of the vehicle in the order yaw, pitch, roll. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle. Flight control modes such as HSA and Waypoint Following should only use the first time index, while Curve Following may utilize an unbounded number of time indices to form dynamically feasible trajectories.	
MaxTurnRate	AngleRateType	Optional
Description	Maximum allowable turn rate in radians/sec.	
MaxClimbRate	SpeedType	Optional
Description	Maximum allowable climb rate in meters/sec.	
MaxDescentRate	SpeedType	Optional
Description	Maximum allowable descent rate in meters/sec.	

Table A-1-42: Vehicle Interface MA_FlightCapabilityMT - CurveFollowingPerformanceProfile UCI Extension Field

AirspeedLimit		
Description	Specified airspeed limit at corresponding altitude value.	
Base Type	PathSegmentSpeedValueType	
Message Structure		
Field Name	Field Type	Field Usage
Value	SpeedType	Required
Description	Indicates the speed along the PathSegment at the EndPoint.	
Reference	SpeedReferenceEnum	Required
Description	Indicates whether the given vehicle speed is reference to ground speed or true air speed.	

Table A-1-43: Vehicle Interface MA_FlightCapabilityMT - AirspeedLimit UCI Extension Field

AccelerationLimit	
Description	Specified acceleration limits in body reference or NED coordinate frame.

Base Type			MA_BodyReferenceAccelerationType
Message Structure			
Field Name		Field Type	Field Usage
X_Accel		AccelerationType	Required
Description		Body-referenced acceleration component along the longitudinal (x or roll) axis, measured in meters/sec^2	
Y_Accel		AccelerationType	Required
Description		Body-referenced acceleration component along the lateral (y or pitch) axis, measured in meters/sec^2	
Z_Accel		AccelerationType	Required
Description		Body-referenced acceleration component along the vertical (z or yaw) axis, measured in meters/sec^2.	
Timestamp		DateTimeType	Optional
Description		Indicates the time when the acceleration data was measured or last known to be correct. The element can also indicate the time at which a future velocity of a System, Entity, etc. will be achieved.	

Table A-1-44: Vehicle Interface MA_FlightCapabilityMT - AccelerationLimit UCI Extension Field

AccelerationLimitPair		
Description	Paired value for the specified acceleration limit, provided either as a set of body referenced orientation rates, or a unitless Mach value.	
Base Type	MA_AccelerationLimitPairType	
Message Structure		
Field Name	Field Type	Field Usage
BodyReferenceOrientationRate	MA_BodyReferenceOrientationRateType	Required
Description	Body reference orientation rates to be paired with the specified acceleration limit.	
MachValue	MachType	Required
Description	Unitless Mach value to be paired with the specified acceleration limit.	

Table A-1-45: Vehicle Interface MA_FlightCapabilityMT - AccelerationLimitPair UCI Extension Field

BodyReferenceOrientationRate		
Description	Body reference orientation rates to be paired with the specified acceleration limit.	
Base Type	MA_BodyReferenceOrientationRateType	
Message Structure		
Field Name	Field Type	Field Usage
RollRate	AngleRateType	Required
Description	Body-referenced attitude rate component along the longitudinal (x or roll) axis,	

	measured in radians/sec.	
PitchRate	AngleRateType	Required
Description	Body-referenced attitude rate component along the lateral (y or pitch) axis, measured in radians/sec.	
YawRate	AngleRateType	Required
Description	Body-referenced attitude rate component along the vertical (z or yaw) axis, measured in radians/sec.	
Timestamp	DateTimeType	Optional
Description	Indicates the time when the velocity data was measured or last known to be correct. The element can also indicate the time at which a future velocity of a System, Entity, etc. will be achieved.	

Table A-1-46: Vehicle Interface MA_FlightCapabilityMT - BodyReferenceOrientationRate UCI Extension Field

OrientationRateLimits		
Description	Specified orientation rate limits in body reference or NED coordinate frame.	
Base Type	MA_BodyReferenceOrientationRateType	
Message Structure		
Field Name	Field Type	Field Usage
RollRate	AngleRateType	Required
Description	Body-referenced attitude rate component along the longitudinal (x or roll) axis, measured in radians/sec.	
PitchRate	AngleRateType	Required
Description	Body-referenced attitude rate component along the lateral (y or pitch) axis, measured in radians/sec.	
YawRate	AngleRateType	Required
Description	Body-referenced attitude rate component along the vertical (z or yaw) axis, measured in radians/sec.	
Timestamp	DateTimeType	Optional
Description	Indicates the time when the velocity data was measured or last known to be correct. The element can also indicate the time at which a future velocity of a System, Entity, etc. will be achieved.	

Table A-1-47: Vehicle Interface MA_FlightCapabilityMT - OrientationRateLimits UCI Extension Field

AirspeedPair		
Description	Airspeed pair value for associated orientation rate limit, specified in body reference or NED coordinate frame.	
Base Type	PathSegmentSpeedValueType	
Message Structure		
Field Name	Field Type	Field Usage

Value	SpeedType	Required
Description	Indicates the speed along the PathSegment at the EndPoint.	
Reference	SpeedReferenceEnum	Required
Description	Indicates whether the given vehicle speed is reference to ground speed or true air speed.	

Table A-1-48: Vehicle Interface MA_FlightCapabilityMT - AirspeedPair UCI Extension Field

1.3.2.2MA_FlightCommandMT UCI Extension

MA_FlightCommandMT	
Description	Message extension included to invoke a flight capability to perform a new flight activity or change and existing flight activity. See the XSD File for more info
Ext. Fields	Command : MA_FlightCommandType Capability : MA_FlightCapabilityCommandType HSA_CSA : MA_HSA_CSA_Type WaypointFollowing : MA_WaypointFollowingType CurveFollowing : MA_CurveControlType Speed : PathSegmentSpeedType Heading : MA_DirectionReferenceType SpeedChoice : MA_HSA_SpeedChoiceType SpeedValue : PathSegmentSpeedValueType CurveFitMethod : MA_CurveFitMethodEnum Reference : MA_HeadingReferenceEnum ControlPoint : MA_CurveFollowingControlPointType RelativeAcceleration : MA_AccelerationChoiceType RelativeVelocity : MA_VelocityChoiceType OrientationRate : MA_OrientationRateChoiceType Direction : MA_DirectionChoiceType Course : MA_DirectionReferenceType

Table A-1-49: Vehicle Interface MA_FlightCommandMT UCI Extension

Command	
Description	Indicates an instance of a Flight Capability command. A Capability command message can have multiple command instances to allow a sequence of related commands in a single coherent message. Each command instance reports status via a separate independent Capability CommandStatus message.
Base Type	MA_FlightCommandType

Message Structure		
Field Name	Field Type	Field Usage
Capability	MA_FlightCapabilityCommandType	Required
Description	Indicates a new invocation of a Flight Capability. Generally, if accepted, the command will result in one or more new Flight Activities being created and reported via the Flight Activity message. The request/response interaction terminates as soon as the command is accepted or rejected; this element is not used to interact with an "active" command. Updates and/or additional interaction with the resulting Activity (after the command is accepted) is accomplished via the sibling Activity element.	
Activity	MA_FlightActivityCommandType	Required
Description	Indicates a command to modify an existing Flight Activity (which was previously reported via the Flight Activity message and was marked as "interactive"). The request/response interaction terminates as soon as the modification is accepted or rejected. The modifications are reflected in subsequent Flight Activity messages.	

Table A-1-50: Vehicle Interface MA_FlightCommandMT - Command UCI Extension Field

Capability		
Description	Indicates a new invocation of a Flight Capability. Generally, if accepted, the command will result in one or more new Flight Activities being created and reported via the Flight Activity message. The request/response interaction terminates as soon as the command is accepted or rejected; this element is not used to interact with an "active" command. Updates and/or additional interaction with the resulting Activity (after the command is accepted) is accomplished via the sibling Activity element.	
Base Type	MA_FlightCapabilityCommandType	
Message Structure		
Field Name	Field Type	Field Usage
Loiter	MA_LoiterType	Optional
Description	Indicates the parameters for a loiter.	
MustFly	MustFlyType	Optional
Description	Indicates the parameters for a must fly.	
Formation	MA_FormationType	Repeated
Description	Indicates the relative positions for the entities in the formation.	
FlightControlMode	MA_FlightControlModesChoiceType	Optional
Description	See annotations in child elements and messages/elements that use this type for details.	
AltitudeStackedMarshall	MA_AltitudeStackedMarshallType	Optional
Description	Indicates kinematics for an altitude stacked marshall.	
Launch	MA_LaunchType	Optional
Description	this element identifies the parameters for a launch/take-off task.	
Recovery	MA_RecoveryType	Optional

Description	this element identifies the parameters for a recovery/land task.	
CapabilityID	CapabilityID_Type	Required
Description	Indicates the Capability instance being commanded.	
Ranking	CapabilityCommandRankingType	Required
Description	Indicates the command's rank relative to other commands and rank-related overrides and tailoring.	
TemporalConstraints	CapabilityCommandTemporalConstraintsType	Optional
Description	Indicates temporal constraints for the command. If omitted, the command should be scheduled immediately while satisfying other applicable constraints.	
OverrideRejection	boolean	Optional
Description	Indicates that the Command shall be performed even though it was previously rejected.	
Traceability	TraceabilityType	Optional
Description	C2 sources work at different levels of detail and abstraction, progressing from what to do to how specifically to do it. Inputs from abstract tasking sources and Tasks can be decomposed into discrete, actionable Tasks and/or Capability commands. This element provides traceability to the external tasking or Task from which this Capability command was derived.	
Classification	SecurityInformationType	Optional
Description	This element provides the classification label that the Subsystem should place on messages and other data resulting from the Command.	
CommandID	CommandID_Type	Required
Description	Indicates the unique ID of the Command corresponding to this message. Used to reference the command when providing status.	
CommandState	CommandStateEnum	Required
Description	The state of the command message data. Indicates if a command message is a NEW command, an UPDATE to an existing command, or a CANCEL of an existing command.	

Table A-1-51: Vehicle Interface MA_FlightCommandMT - Capability UCI Extension Field

HSA_CSA		
Description	This message shall be used to provide the ability to command a new flight vector to the Platform.	
Base Type	MA_HSA_CSA_Type	
Message Structure		
Field Name	Field Type	Field Usage
Altitude	AltitudeReferenceType	Optional
Description	Altitude hold value to be achieved and reference to altitude measurement source.	
Speed	PathSegmentSpeedType	Optional
Description	Defines speed type (reference frame) for all speed related fields in this message.	
Direction	MA_DirectionChoiceType	Optional

Description	Defines heading or course type and reference to hold
-------------	--

Table A-1-52: Vehicle Interface MA_FlightCommandMT - HSA_CSA UCI Extension Field

WaypointFollowing		
Description	This message shall be used to provide the ability to send a new waypoint path or segment for the UA to follow.	
Base Type	MA_WaypointFollowingType	
Message Structure		
Field Name	Field Type	Field Usage
Route	MA_RouteType	Required
Description	Indicates the Paths and Segments that make up the Route to be flown.	

Table A-1-53: Vehicle Interface MA_FlightCommandMT - WaypointFollowing UCI Extension Field

CurveFollowing		
Description	Defines the curve control elements including the curve segments, whether to append the curve, and end of curve behavior..	
Base Type	MA_CurveControlType	
Message Structure		
Field Name	Field Type	Field Usage
CurveSegments	MA_NURBS_PointType	Repeated
Description	Specifies the curve segments as a set of NURBS points.	
AppendCurve	EmptyType	Optional
Description	Specifies this curve will be appended to a previous curve command. This is only valid if the vehicle is currently executing a curve.	
EndOfCurveBehavior	MA_EndOfCurveBehaviorEnum	Optional
Description	Specifies the vehicle behavior at the completion of the curve.	

Table A-1-54: Vehicle Interface MA_FlightCommandMT - CurveFollowing UCI Extension Field

Speed		
Description	Defines speed type (reference frame) for all speed related fields in this message.	
Base Type	PathSegmentSpeedType	
Message Structure		
Field Name	Field Type	Field Usage
SpeedChoice	PathSegmentSpeedChoiceType	Optional
Description	Indicates the speed that the vehicle shall traverse the path segment. Speed can be represented as referenced speed (e.g., ground speed in m/s) or as a unitless Mach	

	number.	
SpeedOptimization	SpeedOptimizationEnum	Optional
Description	Indicates that the System will vary its speed autonomously according to the specified optimization algorithm.	

Table A-1-55: Vehicle Interface MA_FlightCommandMT - Speed UCI Extension Field

Heading		
Description	Defines heading type and reference to hold.	
Base Type	MA_DirectionReferenceType	
Message Structure		
Field Name	Field Type	Field Usage
Value	AngleType	Required
Description	Indicates the heading or course value to hold.	
Reference	MA_HeadingReferenceEnum	Required
Description	Reference to type of heading or course given.	

Table A-1-56: Vehicle Interface MA_FlightCommandMT - Heading UCI Extension Field

SpeedChoice		
Description	Indicates the speed that the vehicle shall traverse the path segment. Speed can be represented as referenced speed (e.g., ground speed in m/s) or as a unitless Mach number.	
Base Type	MA_HSA_SpeedChoiceType	
Message Structure		
Field Name	Field Type	Field Usage
SpeedValue	PathSegmentSpeedValueType	Required
Description	Indicates speed in meters per second (m/s).	
MachValue	MachType	Required
Description	Indicates the speed reference for the associated speed value.	

Table A-1-57: Vehicle Interface MA_FlightCommandMT - SpeedChoice UCI Extension Field

SpeedValue		
Description	Indicates speed in meters per second (m/s).	
Base Type	PathSegmentSpeedValueType	
Message Structure		
Field Name	Field Type	Field Usage
Value	SpeedType	Required

Description	Indicates the speed along the PathSegment at the EndPoint.	
Reference	SpeedReferenceEnum	Required
Description	Indicates whether the given vehicle speed is reference to ground speed or true air speed.	

Table A-1-58: Vehicle Interface MA_FlightCommandMT - SpeedValue UCI Extension Field

CurveFitMethod	
Description	Method through which a curve should be fitted to the discretized points in ControlPoint.
Base Type	MA_CurveFitMethodEnum
Message Structure	
Enumeration Literal	Description
B_SPLINE	B-Spline (basis spline) fit.
BEZIER	Bezier curve fit.
CATMULL_ROM	Catmull-Rom spline fit.
NATURAL_CUBIC	Natural Cubic curve fit.

Table A-1-59: Vehicle Interface MA_FlightCommandMT - CurveFitMethod UCI Extension Field

Reference	
Description	Reference to type of heading or course given.
Base Type	MA_HeadingReferenceEnum
Message Structure	
Enumeration Literal	Description
TRUE_NORTH	Heading value according to Geographic North.
MAGNETIC_NORTH	Heading value according to location where the planet's magnetic field points vertically downward.

Table A-1-60: Vehicle Interface MA_FlightCommandMT - Reference UCI Extension Field

ControlPoint		
Description	Control Points for the desired discretized curve or spline path.	
Base Type	MA_CurveFollowingControlPointType	
Message Structure		
Field Name	Field Type	Field Usage
Position	PointChoice4D_Type	Required
Description	Physical location of the referenced item in geospatial coordinates. The use of the timestamp in this context represents the time the positional data was last measured. Position data may be specified as either a geospatial point or relative to a separately	

	defined reference frame.	
PositionUncertainty	UncertaintyType	Optional
Description	Uncertainty of the physical location of the referenced item.	
DomainVelocity	Velocity3D_Type	Optional
Description	Vehicle's velocity in the current operating domain of the system. Air vehicle components are in True Air Speed.	
GroundVelocity	Velocity2D_Type	Optional
Description	Representation of the vehicle's ground velocity as directional speed components.	
RelativeVelocity	MA_VelocityChoiceType	Optional
Description	Represents the vehicle's velocity, in units of m/s, in the separately defined reference frame referenced in the sibling Position element. This element should only be used when the RelativePoint choice is selected and the seperately defined reference frame is itself moving.	
DomainAcceleration	Acceleration3D_Type	Optional
Description	The vehicle's acceleration in the current operating domain of the system.	
Orientation	OrientationType	Optional
Description	Euler Angle Sequence describing the orientation of the vehicle in the order yaw, pitch, roll. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle.	
OrientationRate	MA_OrientationRateChoiceType	Optional
Description	Euler Angle Sequence describing the rate of orientation change of the vehicle in the order yaw, pitch, roll. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle.	
RelativeAccelertaion	MA_AccelerationChoiceType	Optional
Description		

Table A-1-61: Vehicle Interface MA_FlightCommandMT - ControlPoint UCI Extension Field

RelativeAccelertaion		
Description		
Base Type	MA_AccelerationChoiceType	
Message Structure		
Field Name	Field Type	Field Usage
BodyReferenceAcceleration	MA_BodyReferenceAccelerationType	Required
Description	Body reference accelerations.	
NED_Acceleration	Acceleration3D_Type	Required
Description	Inertial, NED coordinate frame accelerations.	

Table A-1-62: Vehicle Interface MA_FlightCommandMT - RelativeAccelertaion UCI Extension Field

RelativeVelocity

Description	Represents the vehicle's velocity, in units of m/s, in the separately defined reference frame referenced in the sibling Position element. This element should only be used when the RelativePoint choice is selected and the seperately defined reference frame is itself moving.	
Base Type	MA_VelocityChoiceType	
Message Structure		
Field Name	Field Type	Field Usage
BodyReferenceVelocity	MA_BodyReferenceVelocityType	Required
Description	Velocity in body reference frame.	
NED_Velocity	Velocity3D_Type	Required
Description	Velocity in North East Down reference frame.	

Table A-1-63: Vehicle Interface MA_FlightCommandMT - RelativeVelocity UCI Extension Field

OrientationRate		
Description	Euler Angle Sequence describing the rate of orientation change of the vehicle in the order yaw, pitch, roll. The angles are given in a locally level, North-East-Down coordinate system centered on the vehicle.	
Base Type	MA_OrientationRateChoiceType	
Message Structure		
Field Name	Field Type	Field Usage
BodyReferenceOrientationRate	MA_BodyReferenceOrientationRateType	Required
Description	Orientation rate in body reference.	
NED_OrientationRate	OrientationRateType	Required
Description	Orientation rate in inertial reference.	

Table A-1-64: Vehicle Interface MA_FlightCommandMT - OrientationRate UCI Extension Field

Direction		
Description	Defines heading or course type and reference to hold	
Base Type	MA_DirectionChoiceType	
Message Structure		
Field Name	Field Type	Field Usage
Heading	MA_DirectionReferenceType	Optional
Description	Defines heading type and reference to hold.	
Course	MA_DirectionReferenceType	Optional
Description	Defines course type and reference to hold.	

Table A-1-65: Vehicle Interface MA_FlightCommandMT - Direction UCI Extension Field

Course		
Description	Defines course type and reference to hold.	
Base Type	MA_DirectionReferenceType	
Message Structure		
Field Name	Field Type	Field Usage
Value	AngleType	Required
Description	Indicates the heading or course value to hold.	
Reference	MA_HeadingReferenceEnum	Required
Description	Reference to type of heading or course given.	

Table A-1-66: Vehicle Interface MA_FlightCommandMT - Course UCI Extension Field

1.3.2.3MA_FlightCommandStatusMT UCI Extension

MA_FlightCommandStatusMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	MessageData : MA_FlightCommandStatusMDT CannotComplyDetails : MA_CannotComplyDetailsType Severity : MA_FaultSeverityEnum

Table A-1-67: Vehicle Interface MA_FlightCommandStatusMT UCI Extension

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_FlightCommandStatusMDT	
Message Structure		
Field Name	Field Type	Field Usage
CannotComplyDetails	MA_CannotComplyDetailsType	Optional
Description	Provides detail for why FA cannot comply with the flight command.	
Activity	ResultingActivityType	Repeated
Description	Indicates an Activity that satisfies the associated Command. This element is required when the sibling State element indicates ACCEPTED.	
CommandID	CommandID_Type	Required
Description	Indicates the unique ID of the command corresponding to this status message.	
CommandProcessingState	CommandProcessingStateEnum	Required
Description	Indicates the state of the command. The command state machine assumes that the Command and CommandStatus messaging interaction is only used to	

	communicate the desired Command to the Subsystem or Service. The state machine and messaging terminates as soon as the host accepts or rejects the Command. See enumeration annotations for further details.	
CommandProcessingStateReason	CannotComplyType	Optional
Description	Indicates details regarding the reason for and/or a contributing factor to the sibling CommandProcessingState element. This element is only expected when the Command is in a potentially undesirable state such as rejected, canceled or accepted with less than optimum results expected.	

Table A-1-68: Vehicle Interface MA_FlightCommandStatusMT - MessageData UCI Extension Field

CannotComplyDetails		
Description	Provides detail for why FA cannot comply with the flight command.	
Base Type	MA_CannotComplyDetailsType	
Message Structure		
Field Name	Field Type	Field Usage
ValidationResult	MA_ValidationResultEnum	Repeated
Description	In case of flight command rejection, provides the reason for Flight Autonomy rejection. Further details can be provided by Flight Autonomy through the ValidationResultReason. List size for this element is based on "Select All That Apply" condition.	
ValidationResultReason	MA_ValidationResultReasonType	Repeated
Description	In case of flight command rejection, provides additional context for Flight Autonomy rejection.	

Table A-1-69: Vehicle Interface MA_FlightCommandStatusMT - CannotComplyDetails UCI Extension Field

Severity	
Description	The severity of the fault. This enumeration is modified for Mission Autonomy adaptation.
Base Type	MA_FaultSeverityEnum
Message Structure	
Enumeration Literal	Description
NOMINAL	A fault that carries the nominal level indicates that fault does not impact the current mission but may indicate something that should be checked in maintenance. Faults should be tagged as nominal if they are targeting maintenance. An example might be air filter hasn't been changed in 5000 hours or queue length reached 60% of max.
ADVISORY	A fault that is important but is not a Caution or Warning; this can be a configuration change, a condition that is contributing to a degraded aircraft or subsystem operation, or an anomaly that is being automatically controlled or corrected by a higher level system.

CAUTION	A fault that is indicative of a condition that, without a timely and correct response, could result in equipment damage or failure of mission.
WARNING	A fault that is indicative of a condition requiring immediate response or attention to prevent loss of aircraft, or failure of mission.
FAILED	A fault that carries a failed level indicates the failure of a component which may preclude the use of a capability. Faults should be tagged with failed if the component state will be marked faulted. Failed components may or may not prevent the completion of the mission.

Table A-1-70: Vehicle Interface MA_FlightCommandStatusMT - Severity UCI Extension Field

1.3.2.4MA_FlightActivityMT UCI Extension

MA_FlightActivityMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	VehicleCommandState : MA_VehicleCommandStateType Activity : MA_FlightActivityType Speed : PathSegmentSpeedValueType Heading : MA_DirectionReferenceType

Table A-1-71: Vehicle Interface MA_FlightActivityMT UCI Extension

VehicleCommandState		
Description	Indicates the currently commanded parameters per the Flight Autonomy's execution of the request FlightCommand.	
Base Type	MA_VehicleCommandStateType	
Message Structure		
Field Name	Field Type	Field Usage
Altitude	AltitudeReferenceType	Optional
Description	Commanded Altitude Type	
Heading	MA_DirectionReferenceType	Optional
Description	Intended heading of the aircraft.	
Orientation	OrientationType	Optional
Description	Roll, pitch, yaw of aircraft.	
OrientationRate	OrientationRateType	Optional
Description	Change in orientation of the aircraft	
PowerLevel	PercentType	Optional
Description	Indicates average throttle setting of all engines as a percentage.	
Speed	PathSegmentSpeedValueType	Optional

Description	Defines speed type (reference frame) for all speed related fields in this message.	
Acceleration	Acceleration3D_Type	Optional
Description	Defines speed type (reference frame) for all speed related fields in this message.	

Table A-1-72: Vehicle Interface MA_FlightActivityMT - VehicleCommandState UCI Extension Field

Activity		
Description	Indicates the dynamic operational characteristics of an Activity of a System's Flight Capability.	
Base Type	MA_FlightActivityType	
Message Structure		
Field Name	Field Type	Field Usage
VehicleCommandState	MA_VehicleCommandStateType	Required
Description	Indicates the currently commanded parameters per the Flight Autonomy's execution of the request FlightCommand.	
ActualEndPoint	MA_EndPointType	Required
Description		
ActualCompletionTime	DateTimeType	Required
Description		
CurveFollowingControl	MA_CurveFollowingControlType	Required
Description		
ActivityID	ActivityID_Type	Required
Description	Indicates the unique ID of the Activity.	
CapabilityID	CapabilityID_Type	Repeated
Description	Indicates a Capability which the Activity is an instance of. Generally an Activity is an instance of a single Capability. An Activity can be associated with multiple Capabilities when, for example, a Subsystem has distinct Capabilities for automated and manual. In this case, if a manual command is issued while an automated Activity that achieves the desired result already exists, a single Activity represents both Capabilities.	
Interactive	boolean	Required
Description	Indicates whether or not the Activity can be edited/modified via a [Capability]Command...Activity message.	
ActivityState	ActivityStateEnum	Required
Description	Indicates the current state of the Activity its state machine. See enumeration annotations for further details.	
Basis	ActivityBasisEnum	Optional
Description	Indicates the basis of the Activity. See enumerated type annotations for further details. If omitted, a basis of ACTUAL should be assumed.	
ActivityRank	ComparableRankingType	Optional
Description	Indicates the rank of the Activity amongst all other Activities of the Capability and/or Subsystem. This isn't necessarily the same as the Rank associated with the sibling	

	ActivitySource. This element is meant to convey the Subsystem determined relative ranking of all Activities of a Capability category (ESM, EA, etc.). For Subsystems with multiple Capability categories this element is meant to convey relative ranking across all Capability categories. In addition to the rank of the ActivitySource, an Activity's rank can be influenced by Capability priority (see CapabilitySettings), RF_ControlCommands and other factors that influence the Subsystem's internal scheduling and resource allocation.	
ActivityReason	CannotComplyType	Optional
Description	Indicates details regarding the reason for and/or a contributing factor to the sibling ActivityState element. This element is only expected when the Activity is in a potentially undesirable state such as constrained, failed or completed with less than optimum results.	
Source	ActivitySourceType	Repeated
Description	Indicates an originating "source" of the Activity; the command or other item which invoked this specific Activity of a Capability. A single Activity can fully or partially satisfy Activity sources.	
EstimatedStartTime	DateTimeType	Optional
Description	Indicates the estimated time when the Activity started or is expected to start.	
EstimatedCompletionTime	DateTimeType	Optional
Description	Indicates the estimated time when the Activity will complete.	
EstimatedPercentComplete	PercentType	Optional
Description	Indicates the current estimated percent complete of the Activity.	

Table A-1-73: Vehicle Interface MA_FlightActivityMT - Activity UCI Extension Field

Speed		
Description	Defines speed type (reference frame) for all speed related fields in this message.	
Base Type	PathSegmentSpeedValueType	
Message Structure		
Field Name	Field Type	Field Usage
Value	SpeedType	Required
Description	Indicates the speed along the PathSegment at the EndPoint.	
Reference	SpeedReferenceEnum	Required
Description	Indicates whether the given vehicle speed is reference to ground speed or true air speed.	

Table A-1-74: Vehicle Interface MA_FlightActivityMT - Speed UCI Extension Field

Heading	
Description	Intended heading of the aircraft.
Base Type	MA_DirectionReferenceType

Message Structure		
Field Name	Field Type	Field Usage
Value	AngleType	Required
Description	Indicates the heading or course value to hold.	
Reference	MA_HeadingReferenceEnum	Required
Description	Reference to type of heading or course given.	

Table A-1-75: Vehicle Interface MA_FlightActivityMT - Heading UCI Extension Field

1.3.2.5MA_PositionReportDetailedMT UCI Extension

MA_PositionReportDetailedMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	PositionReportData : MA_PositionReportDataType MessageData : MA_PositionReportDetailedMDT

Table A-1-76: Vehicle Interface MA_PositionReportDetailedMT UCI Extension

PositionReportData		
Description	If multiple instances are given, each should be of a different navigation solution type as indicated by the child element.	
Base Type	MA_PositionReportDataType	
Message Structure		
Field Name	Field Type	Field Usage
PositionSource	PositionSourceID_ChoiceType	Required
Description	The source of the positional data.	
ComponentID	ComponentID_Type	Optional
Description	The ID of the specific component within the subsystem that is the source of this data. This can be used if multiple redundant INS/GPS units are represented as a single subsystem.	
NavigationSolutionState	NavigationSolutionStateEnum	Required
Description	The navigation solution state indicates what sensor inputs the navigation unit is using to generate its kinematic solution.	
FigureOfMerit	BytePositiveType	Optional
Description	A FOM is typically calculated as the root-sum-square of the estimated errors contributing to the solution accuracy. Example criteria include:	

	a. GPS receiver state (e.g., carrier tracking, code tracking, acquisition) b. Carrier to noise ratio c. Satellite geometry (DOP value) d. Satellite range accuracy (URA value) e. Ionospheric measurement or modeling error f. Receiver aiding used g. Kalman filter error estimates. The resultant FOM is presented as a numerical value from 1 to 9, where 1 indicates the best navigation performance.	
Kinematics	MA_DetailedKinematicsType	Required
Description	The kinematic state of the vehicle.	
KinematicsError	DetailedKinematicsErrorType	Required
Description	Indicates the covariance matrix of the vehicle, containing detailed kinematic errors of the position, velocity, and orientation. See the DetailedKinematicsErrorType element annotations for details.	
SolutionCorrections	NavigationSolutionCorrectionsType	Optional
Description	This element contains the corrections applied by the measurement unit to the reported values.	

Table A-1-77: Vehicle Interface MA_PositionReportDetailedMT - PositionReportData UCI Extension Field

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_PositionReportDetailedMDT	
Message Structure		
Field Name	Field Type	Field Usage
PositionReportData	MA_PositionReportDataType	Repeated
Description	If multiple instances are given, each should be of a different navigation solution type as indicated by the child element.	

Table A-1-78: Vehicle Interface MA_PositionReportDetailedMT - MessageData UCI Extension Field

1.3.2.6MA_FaultMT UCI Extension

MA_FaultMT	
Description	This represents a fault for the system. Intended to be used by services such as a health monitoring service for centralized fault management and fault isolation of a system. Allows

	announcing a fault state change (raised or cleared) from a service as opposed to a capability or component of a subsystem. Also conveys a service relationship in addition to a subsystem relationship. Extended to include a modified Fault Severity Enumeration for Mission Autonomy.
	See the XSD File for more info
Ext. Fields	MessageData : MA_FaultMDT FaultInformation : MA_SubsystemFaultType Severity : MA_FaultSeverityEnum

Table A-1-79: Vehicle Interface MA_FaultMT UCI Extension

MessageData		
Description	Reports a fault for the system.	
Base Type	MA_FaultMDT	
Message Structure		
Field Name	Field Type	Field Usage
FaultInformation	MA_SubsystemFaultType	Repeated
Description	Contains the fault information for the message.	

Table A-1-80: Vehicle Interface MA_FaultMT - MessageData UCI Extension Field

FaultInformation		
Description	Contains the fault information for the message.	
Base Type	MA_SubsystemFaultType	
Message Structure		
Field Name	Field Type	Field Usage
FaultID	FaultID_Type	Required
Description	The unique ID for this fault. Every fault from a Subsystem, Capability, SupportCapability, Component, or Service should have a unique ID.	
Severity	MA_FaultSeverityEnum	Optional
Description	The severity of the fault. This enumeration is modified for Mission Autonomy adaptation.	
FaultState	FaultStateEnum	Optional
Description	The state of the fault.	
FaultData	FaultDataType	Repeated
Description	Additional fault data which contains key/value pairs with format and unit indicated for the data value. This data supports fault isolation and helps to minimize false alarms.	
DetectionTime	DateTimeType	Optional
Description	Indicates the time the fault was detected; it does not indicate the time the fault was transmitted.	

FaultCode	VisibleString256Type	Required
Description	Indicates the "code" or name of the fault.	
FaultCodeCount	int	Optional
Description	Indicates the number of occurrences of the associated FaultCode and a set FaultState.	
FaultDescription	VisibleString256Type	Optional
Description	Indicates a human readable description of the fault and/or its cause. This could be different than DescriptiveLabel in the sibling FaultID field. For example, DescriptiveLabel could be "Engine 1 overheat" and the FaultDescription could be "Lubrication oil" or "Exhaust nozzle" indicating the underlying cause of the fault.	
SubsystemID	SubsystemID_Type	Optional
Description	The Subsystem that the fault occurred within.	
ServiceID	ServiceID_Type	Optional
Description	The Service that this fault occurred within.	
CapabilityID	CapabilityID_Type	Repeated
Description	Indicates the unique ID of a Capability related to the fault.	
SupportCapabilityID	SupportCapabilityID_Type	Repeated
Description	Indicates the unique ID of a SupportCapability related to the fault.	
ComponentID	ComponentID_Type	Repeated
Description	Indicates the unique ID of a Component related to the fault. When multiple components are identified, it should be inferred that each Component has been individually determined to be related to the fault. In situations where the Subsystem has not yet been able to isolate the fault to individual components, the sibling AmbiguityGroup element should be used.	
AmbiguityGroup	SubsystemFaultAmbiguityGroupType	Repeated
Description	Indicates that a fault has been narrowed down to a set of components, but not yet fully isolated. Separate AmbiguityGroups can be provided when it is desirable to associate specific tests with a subset of the set of components.	

Table A-1-81: Vehicle Interface MA_FaultMT - FaultInformation UCI Extension Field

Severity	
Description	The severity of the fault. This enumeration is modified for Mission Autonomy adaptation.
Base Type	MA_FaultSeverityEnum
Message Structure	
Enumeration Literal	Description
NOMINAL	A fault that carries the nominal level indicates that fault does not impact the current mission but may indicate something that should be checked in maintenance. Faults should be tagged as nominal if they are targeting maintenance. An example might be air filter hasn't been changed in 5000 hours or queue length reached 60% of max.

ADVISORY	A fault that is important but is not a Caution or Warning; this can be a configuration change, a condition that is contributing to a degraded aircraft or subsystem operation, or an anomaly that is being automatically controlled or corrected by a higher level system.
CAUTION	A fault that is indicative of a condition that, without a timely and correct response, could result in equipment damage or failure of mission.
WARNING	A fault that is indicative of a condition requiring immediate response or attention to prevent loss of aircraft, or failure of mission.
FAILED	A fault that carries a failed level indicates the failure of a component which may preclude the use of a capability. Faults should be tagged with failed if the component state will be marked faulted. Failed components may or may not prevent the completion of the mission.

Table A-1-82: Vehicle Interface MA_FaultMT - Severity UCI Extension Field

1.3.2.7MA_TaskMT UCI Extension

MA_TaskMT	
Description	This message specifies a Task that a System will perform against a specific entity or geographical location. Tasks include flight, collection, strike, electronic attack, directed energy, laser designation, cargo delivery and communication relay. They are abstracted to a level of C2 and autonomy derived from UCI operational vignettes. Tasks enable services, operators and other actors to do automated, dynamic, machine-to-machine allocation of Tasks to Systems and subsequent mission planning. If necessary, authorization of Users to Task Systems can be controlled using the Authorization message. See the XSD File for more info
Ext. Fields	TaskType : MA_TaskType TaskPlurality : MA_TaskPluralityEnum ConstraintID : MA_ConstraintID_Type HSA_CSA : MA_HSA_CSA_Type WaypointFollowing : MA_WaypointFollowingType CurveFollowing : MA_CurveControlType Speed : PathSegmentSpeedType Heading : MA_DirectionReferenceType SpeedChoice : MA_HSA_SpeedChoiceType SpeedValue : PathSegmentSpeedValueType CurveFitMethod : MA_CurveFitMethodEnum

Table A-1-83: Vehicle Interface MA_TaskMT UCI Extension

TaskType	
Description	Identifies the type of this Task instance.
Base Type	MA_TaskType

Message Structure		
Field Name	Field Type	Field Usage
AirSample	AirSampleTaskType	Required
Description	Air sample includes direct sampling of the air (SAMPLE) and remote sensing with spectral analysis (SPECTROMETER) with the intent of detecting NBC events.	
AMTI	AMTI_TaskType	Required
Description	Indicates a Task to collect Air Moving Target Indicator (AMTI) data.	
AO	AO_TaskType	Required
Description	Indicates a Task to perform an optical emission such as laser designation.	
CAP	MA_CAP_TaskType	Required
Description	Indicates a Task to perform Combat Air Patrol.	
CargoDelivery	CargoDeliveryTaskType	Required
Description	Indicates a Task to transfer cargo between locations.	
COMINT	COMINT_TaskType	Required
Description	Indicates a Task to provide COMINT.	
CommRelay	CommRelayTaskType	Required
Description	Indicates a Task to provide communications relay support.	
CounterSpace	CounterSpaceTaskType	Required
Description	Indicates a Task to employ a CounterSpace capability.	
Escort	MA_EscortTaskType	Required
Description	Indicates a Task to escort another Entity or System.	
EA	EA_TaskType	Required
Description	Indicates a Task to provide electronic attack support to another System. It guides/constrains the EA System by specifying where it should fly, what it should protect and what the threat is.	
ESM	ESM_TaskType	Required
Description	Indicates a Task to collect ESM data.	
Flight	MA_FlightTaskType	Required
Description	Indicates a Task to effect the flight path/plan of the System.	
Jettison	MA_JettisonTaskType	Required
Description	Indicates a Task to a jettisonable weapon from the system.	
OrbitChange	OrbitChangeTaskType	Required
Description	Indicates a task to perform an orbit change via a spacecraft maneuver.	
OrbitalSurveillance	OrbitalSurveillanceTaskType	Required
Description	Indicates an Orbital Surveillance Task.	
OrbitalSurveillanceSensor	OrbitalSurveillanceSensorTaskType	Required
Description	Indicates a task to perform orbital surveillance sensor tasking.	

PO	PO_TaskType	Required
Description	Indicates a Task to collect Passive Optical data, imagery and video as well as perform PO search and track capabilities.	
Refuel	RefuelTaskType	Required
Description	Indicates a Task for one System to refuel another.	
SAR	SAR_TaskType	Required
Description	Indicates a Task to collect a Synthetic Aperture Radar (SAR) image.	
SMTI	SMTI_TaskType	Required
Description	Indicates a Task to collect Moving Target Indicator (MTI) data.	
Strike	StrikeTaskType	Required
Description	Indicates a Task to kinetically attack/strike, with a weapon that can be released from the System.	
SystemDeployment	SystemDeploymentTaskType	Required
Description	Indicates a task to perform a deployment or release of a system at a specified location.	
TacticalOrder	TacticalOrderTaskType	Required
Description	Indicates a task to perform a tactical order.	
WeatherRadar	WeatherRadarTaskType	Required
Description	Indicates a task to collect weather radar data.	

Table A-1-84: Vehicle Interface MA_TaskMT - TaskType UCI Extension Field

TaskPlurality	
Description	Indicates whether the task is meant to be executed by a single entity or as a joint operation. A join operation implies forming a group prior to execution. A single entity could be a single system or a single pre-formed group.
Base Type	MA_TaskPluralityEnum
Message Structure	
Enumeration Literal	Description
UNSPECIFIED	Indicates the task plurality is unspecified, and should be defaulted or determined by the receiving system.
JOINT	Indicates the task is to be executed as a joint task. A group will be formed when this task is assigned.
SINGLE_ENTITY	Indicates the task is to be executed by a single entity. A single entity could be a single system or a single pre-existing group.

Table A-1-85: Vehicle Interface MA_TaskMT - TaskPlurality UCI Extension Field

ConstraintID	
Description	Indicates any Constraints that apply to the Task (e.g FormationConstraint). A FormationConstraint implies that if a group is formed to execute this task, the referenced Constraint(s) should be applied to that group.

Base Type	MA_ConstraintID_Type	
Message Structure		
Field Name	Field Type	Field Usage
UUID	UniversallyUniqueIdentifierType	Required
Description	Universally Unique Identifier (UUID). A UUID is a 128-bit number (32 hexadecimal digits, 16 bytes) that is conformant to a version and variant of IETF RFC 4122.	
DescriptiveLabel	VisibleString256Type	Optional
Description	Human readable text label with content meaningful to human operators.	

Table A-1-86: Vehicle Interface MA_TaskMT - ConstraintID UCI Extension Field

HSA_CSA		
Description	This message shall be used to provide the ability to command a new flight vector to the Platform.	
Base Type	MA_HSA_CSA_Type	
Message Structure		
Field Name	Field Type	Field Usage
Altitude	AltitudeReferenceType	Optional
Description	Altitude hold value to be achieved and reference to altitude measurement source.	
Speed	PathSegmentSpeedType	Optional
Description	Defines speed type (reference frame) for all speed related fields in this message.	
Direction	MA_DirectionChoiceType	Optional
Description	Defines heading or course type and reference to hold	

Table A-1-87: Vehicle Interface MA_TaskMT - HSA_CSA UCI Extension Field

WaypointFollowing		
Description	This message shall be used to provide the ability to send a new waypoint path or segment for the UA to follow.	
Base Type	MA_WaypointFollowingType	
Message Structure		
Field Name	Field Type	Field Usage
Route	MA_RouteType	Required
Description	Indicates the Paths and Segments that make up the Route to be flown.	

Table A-1-88: Vehicle Interface MA_TaskMT - WaypointFollowing UCI Extension Field

CurveFollowing	
Description	Defines the curve control elements including the curve segments, whether to

	append the curve, and end of curve behavior..	
Base Type	MA_CurveControlType	
Message Structure		
Field Name	Field Type	Field Usage
CurveSegments	MA_NURBS_PointType	Repeated
Description	Specifies the curve segments as a set of NURBS points.	
AppendCurve	EmptyType	Optional
Description	Specifies this curve will be appended to a previous curve command. This is only valid if the vehicle is currently executing a curve.	
EndOfCurveBehavior	MA_EndOfCurveBehaviorEnum	Optional
Description	Specifies the vehicle behavior at the completion of the curve.	

Table A-1-89: Vehicle Interface MA_TaskMT - CurveFollowing UCI Extension Field

Speed		
Description	Defines speed type (reference frame) for all speed related fields in this message.	
Base Type	PathSegmentSpeedType	
Message Structure		
Field Name	Field Type	Field Usage
SpeedChoice	PathSegmentSpeedChoiceType	Optional
Description	Indicates the speed that the vehicle shall traverse the path segment. Speed can be represented as referenced speed (e.g., ground speed in m/s) or as a unitless Mach number.	
SpeedOptimization	SpeedOptimizationEnum	Optional
Description	Indicates that the System will vary its speed autonomously according to the specified optimization algorithm.	

Table A-1-90: Vehicle Interface MA_TaskMT - Speed UCI Extension Field

Heading		
Description	Defines heading type and reference to hold.	
Base Type	MA_DirectionReferenceType	
Message Structure		
Field Name	Field Type	Field Usage
Value	AngleType	Required
Description	Indicates the heading or course value to hold.	
Reference	MA_HeadingReferenceEnum	Required
Description	Reference to type of heading or course given.	

Table A-1-91: Vehicle Interface MA_TaskMT - Heading UCI Extension Field

SpeedChoice		
Description	Indicates the speed that the vehicle shall traverse the path segment. Speed can be represented as referenced speed (e.g., ground speed in m/s) or as a unitless Mach number.	
Base Type	MA_HSA_SpeedChoiceType	
Message Structure		
Field Name	Field Type	Field Usage
SpeedValue	PathSegmentSpeedValueType	Required
Description	Indicates speed in meters per second (m/s).	
MachValue	MachType	Required
Description	Indicates the speed reference for the associated speed value.	

Table A-1-92: Vehicle Interface MA_TaskMT - SpeedChoice UCI Extension Field

SpeedValue		
Description	Indicates speed in meters per second (m/s).	
Base Type	PathSegmentSpeedValueType	
Message Structure		
Field Name	Field Type	Field Usage
Value	SpeedType	Required
Description	Indicates the speed along the PathSegment at the EndPoint.	
Reference	SpeedReferenceEnum	Required
Description	Indicates whether the given vehicle speed is reference to ground speed or true air speed.	

Table A-1-93: Vehicle Interface MA_TaskMT - SpeedValue UCI Extension Field

CurveFitMethod	
Description	Method through which a curve should be fitted to the discretized points in ControlPoint.
Base Type	MA_CurveFitMethodEnum
Message Structure	
Enumeration Literal	Description
B_SPLINE	B-Spline (basis spline) fit.
BEZIER	Bezier curve fit.
CATMULL_ROM	Catmull-Rom spline fit.

NATURAL_CUBIC	Natural Cubic curve fit.
----------------------	--------------------------

Table A-1-94: Vehicle Interface MA_TaskMT - CurveFitMethod UCI Extension Field

1.3.2.8MA_RoutePlanMT UCI Extension

MA_RoutePlanMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	MessageData : MA_RoutePlanMDT

Table A-1-95: Vehicle Interface MA_RoutePlanMT UCI Extension

MessageData		
Description	Indicates the data being conveyed by the message.	
Base Type	MA_RoutePlanMDT	
Message Structure		
Field Name	Field Type	Field Usage
RoutePlanID	RoutePlanID_Type	Required
Description	Indicates a unique ID assigned to the RoutePlan. Minor updates to the Plan can be indicated by an update of the value in the child Version element. More significant updates, replacements, etc. might warrant a new Plan with a different child UUID value.	
PlanCommandID	RoutePlanCommandID_ChoiceType	Optional
Description	Indicates the unique ID of the *PlanCommand which the RoutePlan originated from.	
Plan	MA_RoutePlanType	Required
Description	Contains the details of the route plan.	
ForPlanningUseOnly	boolean	Required
Description	When TRUE, indicates the *Plan is to be used only for intermediate planning purposes. It will eventually be replaced/superseded by another *Plan or be discarded/abandoned. *Plans with this flag should only be used when explicitly referenced by other messages. These *Plans should never: trigger functionality, be subject to approvals, be activated, etc.	
PlanInputs	RoutePlanInputsType	Optional
Description	Indicates inputs that were given in the *PlanCommand (or equivalent automated replan) to create the Plan.	

Table A-1-96: Vehicle Interface MA_RoutePlanMT - MessageData UCI Extension Field

1.3.2.9MA_SystemManagementRequestMT UCI Extension

MA_SystemManagementRequestMT

Description	See the annotation in the associated message for an overall description of the message and this type. See the XSD File for more info
Ext. Fields	

Table A-1-97: Vehicle Interface MA_SystemManagementRequestMT UCI Extension

1.3.2.10 MA_SystemManagementRequestStatusMT UCI Extension

MA_SystemManagementRequestStatusMT	
Description	See the annotation in the associated message for an overall description of the message and this type. See the XSD File for more info
Ext. Fields	

Table A-1-98: Vehicle Interface MA_SystemManagementRequestStatusMT UCI Extension

1.3.2.11 MA_ControlRequestMT UCI Extension

MA_ControlRequestMT	
Description	This message requests control of a UCI System or its Capabilities/Tasks per the A-GRA Custody Construct. UCI Systems are collections of capabilities that can be pseudo-independently shared/controlled by multiple users. Control of a UCI System or its Capabilities/Tasks can be requested using this message. The System/Service that currently has control (mission control) or is the primary controller (capability control) is responsible for responding to this message. The controlling System/Service informs the controlled System/Service of control changes via the ControlSourcesCommand. See the XSD File for more info
Ext. Fields	MessageData : MA_ControlRequestMDT ControlAssignment : MA_ControlAssignmentType RequestType : MA_ControlRequestEnum

Table A-1-99: Vehicle Interface MA_ControlRequestMT UCI Extension

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_ControlRequestMDT	
Message Structure		
Field Name	Field Type	Field Usage

ControlAssignment	MA_ControlAssignmentType	Required
Description	Provides the requested assignment of control.	
RequestType	MA_ControlRequestEnum	Required
Description	Indicates whether the control request is meant to acquire, release, or steal control.	
RequestID	RequestID_Type	Required
Description	Indicates the unique ID of the request message.	
RequestState	RequestStateEnum	Required
Description	The state of the request message data. Indicates if a request message is a NEW request, an UPDATE to a request, or a CANCEL of a request.	

Table A-1-100: Vehicle Interface MA_ControlRequestMT - MessageData UCI Extension Field

ControlAssignment		
Description	Provides the requested assignment of control.	
Base Type	MA_ControlAssignmentType	
Message Structure		
Field Name	Field Type	Field Usage
ControlChoiceType	MA_ControlChoiceType	Required
Description	Indicates the type of control. See enumeration annotations for a description of control types.	
Controller	SystemServiceType	Required
Description	Indicates the identity of the requested controller; the identity of the non-Operator identifier which will be granted control if the request is approved.	
Controllee	ControlleeType	Required
Description	Indicates the identity of an item to be controlled.	

Table A-1-101: Vehicle Interface MA_ControlRequestMT - ControlAssignment UCI Extension Field

RequestType	
Description	Indicates whether the control request is meant to acquire, release, or steal control.
Base Type	MA_ControlRequestEnum
Message Structure	
Enumeration Literal	Description
ACQUIRE	Indicates control is to be acquired.
RELEASE	Indicates control is to be released.
STEAL	Indicates control is to be stolen, if allowed by the hierarchy in the custody configuration.
ASSIGN_STEAL	Indicates control is to be acquired without any permissions if allowed by the hierarchy in the custody configuration.

Table A-1-102: Vehicle Interface MA_ControlRequestMT - RequestType UCI Extension Field

1.3.2.12 MA_ControlRequestStatusMT UCI Extension

MA_ControlRequestStatusMT	
Description	This message is sent in response to a MA_ControlRequest. See the XSD File for more info
Ext. Fields	MessageData : MA_ControlRequestStatusMDT

Table A-1-103: Vehicle Interface MA_ControlRequestStatusMT UCI Extension

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_ControlRequestStatusMDT	
Message Structure		
Field Name	Field Type	Field Usage
ApprovalRequestProcessingState	ApprovalStatusEnum	Required
Description	Indicates the state of the specified MA_ControlRequest.	
ApprovalRequestProcessingStateReason		Optional
Description	Indicates the reason for the approval response to the specified MA_ControlRequest.	
RequestID	RequestID_Type	Required
Description	Indicates the unique ID of the request message corresponding to this status message.	
RequestProcessingState	RequestProcessingStateEnum	Required
Description	Indicates the status of the request message. Indicates how the receiver of the request has responded to the request message.	
RequestProcessingStateReason	CannotComplyType	Optional
Description	Indicates details regarding the reason for and/or a contributing factor to the sibling RequestProcessingState element. This element is only expected when the Request is in a potentially undesirable state such as rejected, canceled or accepted with less than optimum results expected.	

Table A-1-104: Vehicle Interface MA_ControlRequestStatusMT - MessageData UCI Extension Field

1.3.2.13 MA_ControlAssignmentMT UCI Extension

MA_ControlAssignmentMT	
Description	See the annotation in the associated message for an overall description of the message and this type.

	See the XSD File for more info
Ext. Fields	MessageData : MA_ControlAssignmentMDT ControlAssignment : MA_ControlAssignmentType

Table A-1-105: Vehicle Interface MA_ControlAssignmentMT UCI Extension

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_ControlAssignmentMDT	
Message Structure		
Field Name	Field Type	Field Usage
ControlAssignment	MA_ControlAssignmentType	Required
Description	Provides the granted assignment of control.	

Table A-1-106: Vehicle Interface MA_ControlAssignmentMT - MessageData UCI Extension Field

ControlAssignment		
Description	Provides the granted assignment of control.	
Base Type	MA_ControlAssignmentType	
Message Structure		
Field Name	Field Type	Field Usage
ControlChoiceType	MA_ControlChoiceType	Required
Description	Indicates the type of control. See enumeration annotations for a description of control types.	
Controller	SystemServiceType	Required
Description	Indicates the identity of the requested controller; the identity of the non-Operator identifier which will be granted control if the request is approved.	
Controllee	ControlleeType	Required
Description	Indicates the identity of an item to be controlled.	

Table A-1-107: Vehicle Interface MA_ControlAssignmentMT - ControlAssignment UCI Extension Field

1.3.2.14 MA_MissionPlanActivationCommandMT UCI Extension

MA_MissionPlanActivationCommandMT	
Description	See the annotation in the associated message for an overall description of the message and this type. See the XSD File for more info

Ext. Fields

Table A-1-108: Vehicle Interface MA_MissionPlanActivationCommandMT UCI Extension

1.3.2.15 MA_MissionPlanActivationCommandStatusMT UCI Extension**MA_MissionPlanActivationCommandStatusMT****Description**

See the annotation in the associated message for an overall description of the message and this type.

See the XSD File for more info

Ext. Fields

Table A-1-109: Vehicle Interface MA_MissionPlanActivationCommandStatusMT UCI Extension

1.3.2.16 MA_FlightCapabilityStatusMT UCI Extension**MA_FlightCapabilityStatusMT****Description**

See annotations in child elements and messages/elements that use this type for details.

See the XSD File for more info

Ext. Fields

Capability : MA_FlightCapabilityType

MessageData : MA_FlightCapabilityMDT

Table A-1-110: Vehicle Interface MA_FlightCapabilityStatusMT UCI Extension

Capability**Description**

Provides a list of the specific Flight Capabilities as defined by the FlightCapabilityEnum.

Base Type

MA_FlightCapabilityType

Message Structure**Field Name****Field Type****Field Usage****CapabilityType**

MA_FlightCapabilityEnum

Repeated

Description

Indicates the type of Flight capability. See enumeration annotations for further details. List size for this element is based on "Select All That Apply" condition.

FlightCapabilityPerformanceProfile

MA_FlightCapabilityPerformanceProfileType

Optional

Description

Provides per-control mode performance parameters. Enables autonomy commands to properly conform to envelope limits or limit summary values before sending a FlightCommand to the vehicle.

AcceptedInterface

CapabilityControlInterfacesEnum

Repeated

Description

Indicates an accepted control interface for the Capability. See enumeration annotations for further details. List size for this element is based on "Select All That Apply" condition.

CommsRequired	boolean	Optional
Description	Indicates whether (TRUE) or not (FALSE) communications are required to use the Capability. Omitted is equivalent to FALSE. For example, an unmanned system with no onboard storage would require communications to disseminate the outputs of the Capability.	
CapabilityID	CapabilityID_Type	Required
Description	Indicates the unique ID of this Capability. This ID is used to invoke the Capability via Command messages, report its activity via Activity messages, etc.	
DomainPairing	EnvironmentPairingEnum	Repeated
Description	Indicates the domain/environment which the Capability is employed from and the "target" domain/environment. See enumerated type annotations for further details. List size for this element is based on "Select All That Apply" condition.	
ProductOutput	ProductOutputType	Repeated
Description	Indicates a Product that can be an output of the Capability.	
PackageOperation	PackageOperationEnum	Optional
Description	Indicates if and how the operation of the Capability can be coordinated with a partner Capability of another System in a multi-System Package. Omitting this element is equivalent to the enumerate NO_PACKAGE_OPERATION. The actual Package configuration including per-Capability partnering between Systems is given in the PackageStatus message. For example, a System that can perform cooperative geolocation could advertise with this element in the ESM_Capability message. A System that can navigate in a swarm with other Systems could advertise with this element in the FlightCapability message.	

Table A-1-111: Vehicle Interface MA_FlightCapabilityStatusMT - Capability UCI Extension Field

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_FlightCapabilityMDT	
Message Structure		
Field Name	Field Type	Field Usage
Capability	MA_FlightCapabilityType	Repeated
Description	Provides a list of the specific Flight Capabilities as defined by the FlightCapabilityEnum.	
AvailableMDF	MDF_Type	Repeated
Description	Indicates a Mission Data File (MDF) that is available for use. All MDFs currently available for use should be listed. An MDF is a collection of data, organized into files, that is used to tailor, configure or otherwise affect the behavior of a Capability. MDFs can vary from mission to mission or even by phases of a single mission. They are generally developed in pre-mission planning and are unique for each Subsystem model. Their content is not standardized by UCI, but the mechanism for advertising their availability and activating them is. Their content is abstracted by the UCI concept of Capability.	

Table A-1-112: Vehicle Interface MA_FlightCapabilityStatusMT - MessageData UCI Extension Field

1.3.2.17 MA_SystemNotificationMT UCI Extension

MA_SystemNotificationMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	MessageData : MA_SystemNotificationMDT NotificationDetail : CannotComplyType

Table A-1-113: Vehicle Interface MA_SystemNotificationMT UCI Extension

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_SystemNotificationMDT	
Message Structure		
Field Name	Field Type	Field Usage
SystemSubjectIDs	MA_PackageSystemType	Required
Description	Indicates the subject of the Notification, the thing which the Notification is about.	
SystemPerspective	NotificationPerspectiveEnum	Repeated
Description	Indicates a perspective (characteristic, behavior and/or change) of the Subject that is the basis of the Notification. List size for this element is based on "Select All That Apply" condition.	
AssociatedMessage	AssociatedMessageType	Repeated
Description	Indicates a message that is associated with the Notification.	
NotificationDetail	CannotComplyType	Optional
Description	Indicates details regarding the reason for and/or a contributing factor to the unsuccessful processing of the sibling AssociatedMessage element. This element is only expected when the Associated Message processing is not successful	
NotificationID	NotificationID_Type	Required
Description	Indicates the unique ID of the Notification.	
NotificationState	NotificationStateEnum	Required
Description	Indicates the current state within an enumerated state machine that characterizes the life cycle of a notification.	
Timestamp	DateTimeType	Required
Description	The time at which the Notification was originally triggered (ex: event time).	
Source	NotificationSourceType	Required
Description	Indicates the originating source of the Notification.	
Severity	NotificationSeverityEnum	Required

Description	Indicates the overall severity of the notification. Informational notifications are meant to provide informational context of events. Indicates the existence of a hazardous condition requiring immediate action to prevent loss of life, equipment damage, or abortion of the mission. Critical notifications indicate the need for immediate attention.	
AppliesTo	SubjectType	Repeated
Description	Indicates an Asset that the Notification applies to and/or is affected by. Optional when the Notification characterizes the Subject without regard for Assets it might affect. Omission of this element doesn't necessarily mean the Notification isn't applicable to an Asset.	
NotificationNarrative	VisibleString1024Type	Optional
Description	Human readable message for display to an operator.	

Table A-1-114: Vehicle Interface MA_SystemNotificationMT - MessageData UCI Extension Field

NotificationDetail		
Description	Indicates details regarding the reason for and/or a contributing factor to the unsuccessful processing of the sibling AssociatedMessage element. This element is only expected when the Associated Message processing is not successful	
Base Type	CannotComplyType	
Message Structure		
Field Name	Field Type	Field Usage
Reason	CannotComplyEnum	Required
Description	Indicates the reason why an action (Task, [Capability]Command, [SupportingCapability]Command, [Capability]Activity, etc.) was rejected, interrupted, unallocated, failed and/or generally can't be completed satisfactorily. See enumeration annotations for further details.	
Description	VisibleString1024Type	Optional
Description	Indicates a human-readable text description of the reason for the "can't comply".	
AssociatedID	ID_Type	Optional
Description	Indicates an ID corresponding to another object that is associated with or the cause of the "can't comply".	

Table A-1-115: Vehicle Interface MA_SystemNotificationMT - NotificationDetail UCI Extension Field

1.3.2.18 MA_ResponseMT UCI Extension

MA_ResponseMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	MessageData : MA_ResponseMDT

Table A-1-116: Vehicle Interface MA_ResponseMT UCI Extension

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_ResponseMDT	
Message Structure		
Field Name	Field Type	Field Usage
ResponseID	ResponseID_Type	Required
Description	Indicates the unique ID of the Response. Minor updates to the Response can be indicated by an update of the value in the child Version element. More significant updates, replacements, etc. might warrant a new instance with a different child UUID value.	
ResponseType	ResponseTypeEnum	Repeated
Description	Indicates the types of Response options included in the message. List size for this element is based on "Select All That Apply" condition.	
ResponseManagementConstraints	RequirementConstraintsType	Optional
Description	Indicates monitoring constraints on the Response. The constraint type used here is common to all Requirement types: Effect, Action, Task, [Capability]Command, Response. Requirement constraints can be 1) given in a [Requirement] message, 2) overridden as a planning input in a [Requirement]PlanCommand message, 3) further refined/overridden during plan creation and reported in a [Requirement]Plan message to inform subsequent planning or execution. See sibling Option element annotation for an overall description of Response structure. Constraints given here aren't conveyed to instantiated Response options for a particular triggering event. The constraints given here apply to Response management (planning, allocation and trigger monitoring prior to a triggering event). Constraints to apply to instantiated Response options, if any, are given elsewhere in the message.	
ResponseManagementGuidance	RequirementGuidanceType	Optional
Description	Indicates guidance on planning and C2 of the Response.	
Option	MA_ResponseOptionDetailsType	Repeated
Description	Indicates an Option of the Response. Responses are designed to be flexible. A Response is a set of prioritized Options to address an anticipated triggering event, events or event sequence in the battlespace. Responses are allocated to Systems via a ResponsePlan. When a ResponsePlan is activated, monitoring for the triggering events in the Options begins. When a triggering event happens, the applicable Option(s) are evaluated.	
RequirementsTemplate	RequirementsTemplateType	Repeated

Description	The sibling Option element associates triggering events with a desired Response one of which is to use this element, a Requirements template, to generate or change one or more Requirements messages/objects (Effect, Action, Task, [Capability]Command). An instance of this element is a RequirementsTemplate; it indicates the types of Requirements to generate or update and their details. Each RequirementsTemplate has a unique ID and therefore can be used for multiple Options.	
Metadata	RequirementMetadataType	Optional
Description	Indicates traceability and other metadata associated with the Response.	

Table A-1-117: Vehicle Interface MA_ResponseMT - MessageData UCI Extension Field

1.3.2.19 MA_MissionPlanExecutionStatusMT UCI Extension

MA_MissionPlanExecutionStatusMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	

Table A-1-118: Vehicle Interface MA_MissionPlanExecutionStatusMT UCI Extension

1.3.2.20 MA_TaskCommandStatusMT UCI Extension

MA_TaskCommandStatusMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	MessageData : MA_TaskCommandStatusMDT

Table A-1-119: Vehicle Interface MA_TaskCommandStatusMT UCI Extension

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_TaskCommandStatusMDT	
Message Structure		
Field Name	Field Type	Field Usage
Activity	ResultingActivityType	Repeated
Description	Indicates an Activity that satisfies the associated Command. This element is required when the sibling State element indicates ACCEPTED.	
CommandID	CommandID_Type	Required
Description	Indicates the unique ID of the command corresponding to this status message.	

CommandProcessingState	CommandProcessingStateEnum	Required
Description	Indicates the state of the command. The command state machine assumes that the Command and CommandStatus messaging interaction is only used to communicate the desired Command to the Subsystem or Service. The state machine and messaging terminates as soon as the host accepts or rejects the Command. See enumeration annotations for further details.	
CommandProcessingStateReason	CannotComplyType	Optional
Description	Indicates details regarding the reason for and/or a contributing factor to the sibling CommandProcessingState element. This element is only expected when the Command is in a potentially undesirable state such as rejected, canceled or accepted with less than optimum results expected.	

Table A-1-120: Vehicle Interface MA_TaskCommandStatusMT - MessageData UCI Extension Field

1.3.2.21 MA_TaskCommandMT UCI Extension

MA_TaskCommandMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info
Ext. Fields	MessageData : MA_TaskCommandMDT

Table A-1-121: Vehicle Interface MA_TaskCommandMT UCI Extension

MessageData		
Description	See annotations in child elements and messages/elements that use this type for details.	
Base Type	MA_TaskCommandMDT	
Message Structure		
Field Name	Field Type	Field Usage
Command	MA_TaskCommandType	Repeated
Description	Indicates an instance of a Task Capability command. A Capability command message can have multiple command instances to allow a sequence of related commands in a single coherent message. Each command instance reports status via a separate, independent Capability CommandStatus message.	

Table A-1-122: Vehicle Interface MA_TaskCommandMT - MessageData UCI Extension Field

1.3.2.22 MA_ActionStatusMT UCI Extension

MA_ActionStatusMT	
Description	See annotations in child elements and messages/elements that use this type for details. See the XSD File for more info

Ext. Fields

Table A-1-123: Vehicle Interface MA_ActionStatusMT UCI Extension

1.4 Mission and Flight Autonomy Capabilities V5

Table A-1-124: Mission and Flight Autonomy Capabilities V5

ID	Capability	Category	Definition	Allocation
1	Flight Autonomy	Overview	Flight Autonomy includes Air Vehicle (AV) aka Autonomous Collaborative Platform (ACP) specific "aviate," flight, and safety critical functions. Aviating includes autopilot functions and stable vehicle control to safely execute commanded paths, taxi, take-off, and landing. Flight and safety critical functions verify, bound, and recover safe flight. The command validator enables a separation of concerns between highly-assured flight and safety critical functions and mission behaviors. For example, it shields the autopilot from commands that would exceed operation limits or fly into terrain. Flight Critical refers to loss of the aircraft while Safety Critical refers to the loss of human life.	Flight Autonomy
2	Mission Autonomy	Overview	Mission Autonomy is responsible for decision making, coordination, and overall mission execution for the ACP based on mission planned or dynamic intent commanded by a Command and Control (C2) entity, ie., a human. MA commands are vetted by the FA command validator for safety compliance and MA itself does not command flight controls. Mission Autonomy should be updated frequently with new or refined skills, based on mission debrief and other sources of feedback.	Mission Autonomy
3	Flight/Team Lead Role	Tasking	Notionally, i.e., NOT A REQUIREMENT, a flight of ACPs could contain 16 ACPs and could have 10+ flights of various sizes operating in an area. It quickly becomes intractable for humans, especially QBs, to directly manage that many ACPs. The flight lead concept is an enabling point of entry for managing multiple ACPs. The flight lead is a logical notion in that any of the flight could perform "Flight Lead Functions." Due to the possibility of QB/ACP Link congestion, the flight lead needs to be a comms gateway between the flight and the QB. It is also possible that the flight lead provides voice or data satcom relay for the QBs. Other flight lead functions include: receiving and decomposing	Mission Autonomy

ID	Capability	Category	Definition	Allocation
			Tasks/Objectives, maintaining the flight, coordinating tactics, etc	
4	Aviate	Navigation and Deconfliction	The Autonomy should be able to fly routes, as defined by waypoints, HSA, holding patterns, and splines/trajectories. Command of the aircraft and fine controls such as spline/trajectory must be executed by Flight Autonomy. The A-GRA defines the parameters for all types of flight control modes. Mission Autonomy may request these functions to be executed although they may be rejected if they cannot be performed within the flight envelope. For some of these items, timing and speed are critical and MA should be able to meet Time of Arrival type goals and comply with speed restrictions and route "bounds." FA must ensure the safety of these commands coming from MA. See flight Modes for a list of types of modes.	Both
5	Task Prioritization/Command Queue/Task Allocation (Safety, Nav)	Tasking	This function is responsible for maintaining a high level list of commands, and sub-task breakdown as applicable, and allocating those commands/tasks to various flight members. This function needs to handle simultaneous competing commands/objectives.	Mission Autonomy
6	Vehicle Fuel Management	Basic Flight Operations	The ACP should perform fuel management for subsystem control. This includes real-time monitoring of fuel state and abnormal fuel flow based on atmospheric conditions and engine settings. FA does not determine bingo and joker fuel points. FA responses to low fuel are defined by emergency procedures if MA is unresponsive, which may vary between peace and war times. MA makes decisions based on fuel state, unless otherwise commanded or prioritized by the QB or C2.	Flight Autonomy
7	Mission Fuel Management	Basic Flight Operations	MA should perform fuel management for mission execution. MA will receive fuel state from FA, and then determines Bingo and Joker fuel based upon mission parameters. MA may recommend new actions to avoid fuel failure that will align with mission priorities. Mission Autonomy should calculate "play time" real time recovery routing and flight profile. Mission planners will set pre-planned fuel states for MA	Mission Autonomy

ID	Capability	Category	Definition	Allocation
			to manage (normal recovery fuel, min fuel, emergency fuel, bingo, joker, etc.)	
7	Navigation (Hold, Dynamic Route, Time,)	Basic Flight Operations	Ability to produce interdependent multi-ship routes determining where to go and how to get there using various types of navigation information. Key considerations of the Navigation function are developing flexible/dynamic routes in real-time given an environment change, producing 4D (3D+Time) routes to ensure coordination, navigating as a flight, and developing coordinated holds, CAPs, and other types of racetrack patterns. The resulting routes are achieved via multiple, currently three (3), types of commands for FA skills.	Mission Autonomy
8	Geozones	Navigation and Deconfliction	Geozones are non safety critical defined areas. They allow the MA and/or mission planner to place "soft" limits on the autonomy. They can be developed and removed by the autonomy as well as C2 entities. Geozones can contain a multitude of information in them, see A-GRA, such as weather, speed restrictions, keepin/keep-out, active time, and priority.	Mission Autonomy
9	Overlapping Geozone Management	Formation and Teaming	In mission planning, geozones may overlap. For example, broad keep-in zones may consist of smaller, speed-restricted zones. Spatially overlapping keep-in and keep-out zones may have varying activation times and priorities. Mission Autonomy should decompose these overlapping mission geozones into a set of non-conflicting "primitives" to be followed at various 4D points. This includes the ability for C2 to add, delete, or change any of the geozones, overlapping or not. Geozone Management includes 4 primary Subfunctions: - Store Geozones (OMS OpZones) in a database for Service queries - Generate Geozones for mission deconfliction (i.e. altitude block deconfliction, moving geozones for formations) - Activate/deactivate Geozone Restrictions for ACPs (i.e. mission area is 'keep in' zone until RTB then Restriction lifted) - Develop a "union set" of overlapping Geozones for Mission Autonomy to follow	Mission Autonomy

ID	Capability	Category	Definition	Allocation
10	Geofence	Navigation and Deconfliction	FA/VMS should be able to store and enforce geofences, seen as "hard" limits. The primary intent is to pre-define a "no-fly" limit such as an operational altitude floor. FA is responsible for any 'geofence' adherence. Other uses for geofence shall only be considered on a case-by-case basis at the discretion of the program office. The geofence is pre-mission-planned and loaded within the FA/VMS. Specific examples of unacceptable uses of Geofences include but are not limited to: - Test range boundaries - FAA aircraft deconfliction corridors - Political boundaries As a rule of thumb most operational sorties will not use Geofences beyond an altitude floor.	Flight Autonomy
11	Formation Flight or Loose Formation Flight	Formation and Teaming	In general, formation flight for ACPs is to be considered loose. These formations are used for transiting airspace, providing coordinated/deconflicted team flight. MA can plan and execute formations. FA is responsible for collision avoidance and would ignore MA commands as needed to maintain safety.	Mission Autonomy
12	Tight Formation Flight	Navigation and Deconfliction	Certain ACP tasks may require tight formations. As a mission behavior, tight formations will necessitate close coordinated turning, entry and exit procedures, etc. Both loose and tight formations are assumed to operate outside of safe separation bubbles. However, presence of adequate FA/VMS flight safety protection is crucial for assuring safe separation during tight formations.	Mission Autonomy
13	Mission Contingency Management (Weather, Emergencies)	Contingencies	Contingency Management is responsible for dynamic task reallocation and mission re-planning when a contingency situation is detected. A contingency is an unanticipated event that impacts the ability to complete a planned task such as payload failures, pop up threats, weather and zone modifications. Non-safety critical contingencies are handled by MA.	Mission Autonomy
14	Vehicle Contingency Management	Contingencies	FA needs to autonomously handle a set of safety-critical, vehicle specific, and rapid attention contingences. Flight Autonomy should notify MA of the contingences, remedy/status, and any changes to vehicle capability, such as reduced speed/range, sensor failures, etc. FA would execute "Platform-1 Emergency Procedures" type actions for some EPs. Contingencies that may have various alternate	Flight Autonomy

ID	Capability	Category	Definition	Allocation
			responses such as Bingo Fuel (which may require C2 confirmation) are handled by MA, not FA .	
15	Team Management	Formation and Teaming	Team management is the ability to form groups and sub-groups. This requires the ability to add, subtract, and change group/sub-group members. It is also requires the ability to make new groups/sub-groups to include real-time and in the mission plan.	Mission Autonomy
16	Tactical Commands	Tactical Employment	Various tactical commands are enabled by Mission Autonomy. Some examples of these commands include target allocation, CAP, and commit. These are high level tactical behaviors that MA is responsible for decomposing into various FA and Mission Systems commands. The execution of these commands is built upon FA primitives. Mission Autonomy is responsible for managing the priority and planning for these tasks. Some tactical commands may be high level resulting in multiple Mission Autonomy and Flight Autonomy sub-tasks, such as "sanitize airspace." At times, conflicting tactical commands will be deconflicted by priority. MA will need to optimize teams' mission outcomes with respect to these priorities.	Mission Autonomy
17	Formation/Flight Lead	Formation and Teaming	The Formation/Flight Lead may be an actual aircraft or a virtual point in the sky. Manned formations fly off of a lead aircraft. This lead aircraft helps maintain formation space and provides a reference for things like formation turns.	Mission Autonomy
18	Route Management	Navigation and Navigation and Deconfliction	Route management defines a route comprised of waypoints, HSA, and/or trajectories for FA to execute a path. Route management includes coordinating time-of-arrival, specific heading arrival/departure, CAP management, following FAA and other flying rules, geozone adherence, etc. Creation and planning of routes should be a capability leveraged by MA and informed by state data provided by FA, Mission Systems, etc.	Mission Autonomy
19	Preplanned Zeroize	Contingencies	This may be platform- and/or mission-dependent based on a given data store configuration. MA can be scoped to command some zeroization. FA's role is to execute the actual zeroization of data.	Both

ID	Capability	Category	Definition	Allocation
20	Auto Taxiing	Admin	MA sends its taxi routes composed of waypoints to FA. FA validates the AV ability to perform taxi routes with appropriate taxi speed, turn rate, etc. and responds to MA positively or negatively regarding the route. As waypoints are validated, MA commands activation of the achievable route. FA provides updates on route completion. FA/VMS is assumed to have the ability to do emergency braking for protection against obstructions.	Both
21	Mission Planning	Mission Planning	MA ingests initial and alternate mission plans pre-mission which can be updated mid-mission directly via the C2 interface or on a contingency basis. Mission plans include routes, contingency planning, tactical behaviors, weather/winds, geozones, etc. An ACP can hold multiple mission plans and multiple routes per mission plan. Each mission plan should contain a default route and RTB. An ACP should have a mission plan activated at all times, even during ad-hoc tasking. See A-GRA for more details.	Mission Autonomy
22	Auto Takeoff and Landing	Admin	FA: Takeoff and Landing are safety critical functions that need to be handled by Flight Autonomy. MA: Selection of ATLC direction/etc. can be done on mission side as long as it's a pre-approved ATLC profile. Flight Autonomy Functions: Automatic Landing Wave off (e.g. Crosswinds) Rejected Take Off and Abort commands Automatic Takeoff to a (Time, waypoint, state based, land gear up)- Exit Criteria Emergency Divert on takeoff/Landing Automatic Landing from (Time, waypoint, state based) - Entrance Criteria Flight Autonomy puts in landing configuration and final line up with runway to include: Glide slope Flaring Crabbing Go Around	Flight Autonomy
23	Ground and Air Deconfliction	Navigation and Deconfliction	Deconfliction occurs 10s of seconds to minutes out from potential collision. The autonomy provides air and ground deconfliction according to FAA and other governing rules. Deconfliction options can be prioritized as part of the mission plan.	Mission Autonomy

ID	Capability	Category	Definition	Allocation
24	Sense and Avoid / Cooperative Avoidance	Navigation and Deconfliction	These behaviors require tight-loop control that is only in Flight Autonomy. Mission Autonomy may passively observe detected vehicles to create deconflicted plans. FA behaviors: ADS-B Collision Avoidance Accept current aircraft position Monitor other aircraft position Provide information on avoidance/ads-b data/ avoidance route to Mission Autonomy Trajectory tracking Take evasive maneuvers	Flight Autonomy
25	Terrain Avoidance	Navigation and Deconfliction	These behaviors require tight-loop control that is only in Flight Autonomy: Hold limited DTED Use own-ship altitude Avoid terrain and return to straight and level Trajectory tracking	Flight Autonomy
26	Airspeed	Basic Flight Operations	This capability includes setting the airspeed source (e.g., GPS, Air data) and units (e.g., KAIS, Knots).	Flight Autonomy
27	Emergency Divert or Return to Base	Contingencies	Auto-route to preplanned location in response to an aircraft emergency state where MA is unresponsive. The emergency procedures to perform this action may differ depending on peace time or war time conditions.	Flight Autonomy
28	Divert or Return to Base	Contingencies	Generate own route or C2 directed command to return to base or other divert location to include ability to understand current state of various basing locations, understand flight/individual fuel limitation and flying qualities. Can use various aircraft/environmental states and C2 inputs to make determinations.	Mission Autonomy
29	Flight Envelope Management	Basic Flight Operations	The FA shall protect the aircraft from unsafe commands. This includes simple things such as g-limits, rate-limits, and speed limits. It also includes more complex concepts such as state energy management, i.e. prevent the aircraft from getting into an unrecoverable part of the flight regime such as stall	Flight Autonomy

ID	Capability	Category	Definition	Allocation
30	Command Validation	Validation	Maintaining safe flight requires Flight Autonomy to reject unsafe commands with reasoning per A-GRA definition. MA needs to understand returned command validation response. Notional groupings for types of validations include: Valid, Invalid, Feasible, Infeasible Command validation ensures correct commands are coming into the safety critical FA. Some examples of validation include: Invalid commands (wrong format) Valid/Un-executable (wrong parameters) Command Flight Envelope validation Command Rejection adjustment (best effort with reason) Energy Management (Stall, etc.) Respecting Geofences	Flight Autonomy
31	Mission Command Validation	Validation	Prior to sending commands to FA, the MA should ensure that proper commands are sent to the FA as much as possible with available data. Also, the MA needs to accept command rejection from FA and adjust accordingly.	Mission Autonomy
32	Flight Modes	Basic Flight Operations	Modes that the FA may utilize to optimize performance while accepting MA requests. Some examples may include: Best Cruise Speed/Alt/Best Loiter Fast as possible Airspeed, Altitude Types Best Climb	Flight Autonomy
33	Mission Route Capability	Navigation and Deconfliction	Mission Autonomy should receive commands to add, delete, and modify routes inside mission plans directly from C2 or on a contingency basis. Accept Routes Change waypoints out of routes Add Waypoints Delete waypoints Loiter/Hold Referenced to Ground Track	Mission Autonomy
34	Subsystem state data	Contingencies	FA should monitor and report subsystem data to Mission Autonomy as defined in the A-GRA.	Flight Autonomy